# 알고리즘 개념교안

version.25.07.29

해당 교안을 무단으로 복제, 업로드, 배포, 도용할 경우

저작권법 제 97조의 저작재산권침해죄에 해당하며

5년이하의 징역 또는 5천만원 이하의 벌금에 처해질 수 있습니다.

# 목차


시간복잡도와 빅오표기법 **8**
- 시간복잡도 8
- 빅오표기법 9
- 상수시간 시간복잡도 O(1) 10
- 문제로 연습하는 시간복잡도 12
  - Q1. 다음 코드의 시간복잡도는? 12
  - Q2. 다음 코드의 시간복잡도는? 13
  - Q3. 다음 코드의 시간복잡도는? 14
  - Q4. 다음 코드의 시간복잡도는? 15
  - Q5. 다음 코드의 시간복잡도는? 16
  - [참고] 재귀함수에서의 메인로직은 무엇인가요? 17
- 시간복잡도의 존재이유 : 효율적인 코드의 척도 18
- 가능한 시간복잡도의 기준 19

자료구조의 시간복잡도 **20**
- 배열(Array) 20
- 연결리스트(doubly linked list) 20
- 배열(Array) 21
- 배열(vector) 21
- 스택(stack) 21
- 큐(queue) 21
- 연결리스트(doubly linked list) 22
- 맵(map) 22

공간복잡도 **23**
- 1. 문제의 최대범위 23
- 2. 문제의 메모리제한 24

누적합 **27**
- Q1. 잃어버린 승철이의 뇌 28

구현 **31**
- Q1. 도파 문자열 다루기 31
- Q2. 오름차순과 평균값 32

문제를 푸는 방법 **34**
- 문제 해석 34
- 손코딩 35
- 문제푸는 과정 정리 35

그래프이론 **37**
- 정점과 간선 37
- indegree outdegree 39
- 가중치 39

트리 **39**
- 트리의 특징 40

루트노드 41
내부노드 41
리프노드 41
트리의 높이와 레벨 42
이진트리**(BT, Binary Tree)** **43**
균형이진트리의 높이 차이 계산 43
이진탐색트리**(BST, Binary Search Tree)** **44**
Q. 이진탐색트리의 시간복잡도는 얼마일까요? 45
그래프 구현과 탐색 **1.** 인접행렬 **47**
문제로 알아가는 인접행렬 50
인접행렬을 기반으로 탐색하기 53
그래프 구현과 탐색 **2.** 인접리스트 **54**
Q. 왜 list가 아닌 vector로 구현할까? 56
list의 시간복잡도 56
vector의 시간복잡도 56
Q. 인접리스트를 기반으로 탐색하기 57
인접행렬과 인접리스트의 차이 58
공간복잡도 58
시간복잡도 : 간선 한개 찾기 58
시간복잡도 : 모든 간선찾기 59
그래프가 희소할 때는 인접리스트, 조밀할 때는 인접행렬을 써라 59
정리 60
그래프 구현과 탐색 **3.** 맵과 방향벡터 **60**
4방향탐색과 방향벡터 61
Q1. {0, 0}좌표에서 dy, dx를 만들어 4방향(위, 오른쪽, 아래, 왼쪽)을 탐색하며 좌표를 출력하시오 62
Q2. {0, 0}좌표에서 dy, dx를 만들어 8방향(위, 오른쪽, 아래, 왼쪽 및 대각선방향포함)을 탐색하며 좌표를 출력하시오 62
Q3. 3 * 3 맵을 입력받아야 함. 이 맵은 1과 0으로 이루어져있고 {0, 0}은 무조건 1임을 보장한다. {0, 0}부터 4방향을 기준으로 한칸씩 탐색해나가며 방문한 정점은 다시 방문하지 않으며 방문하는 좌표를 출력하는 코드. 0은 갈 수 없는 지역. 1은 갈 수 있는 지역을 구현하시오 63
연결된 컴포넌트**(connected component)** **64**
깊이우선탐색 **66**
DFS의 수도코드 66
구현방법1 : 돌다리를 두들겨 보다 69
구현방법2 : 못 먹어도 GO 70
Q. 종화는 방구쟁이야! 71
디버깅 팁 74
너비우선탐색**(BFS, Breadth First Search)** **75**
BFS의 수도코드1 77
BFS의 수도코드2 77
Q. 만약 앞의 그래프의 가중치가 한칸이 아니고 2칸이라면 어떻게 될까요? 80
Q. 시작지점이 다수라면? 80

Q. BFS는 왜 가중치가 같은 그래프내에서만 최단거리 알고리즘으로 쓰이나요? 가중치가 다른 그래프에서는 안되나요? 81
Q. 당근마킷 엔지니어 승원이 82
**DFS와 BFS비교** **85**
트리순회 **86**
후위순회 86
전위순회 86
중위순회 87
레벨순회 87
Q. 아래의 그래프가 주어졌을 때 preorder, inorder, postorder를 구현하라 87
정리 90
완전탐색 **92**
반복문을 활용한 완전탐색 92
Q. 2400!! 2400!! 2400!! 93
재귀함수를 활용한 완전탐색 94
Q.승철이의 문단속 94
백트래킹 **96**
완전탐색과의 차이 96
가지치기 96
Q. N과 N개의 자연수가 주어진다. 여기서 몇개의 숫자를 골라 합을 mod 11을 했을 때 나오는 가장 큰수를 구하라 97
완전탐색 **-** 원복 **100**
Q. 긍정왕 홍철이의 구걸 여행 104
이진수 **108**
이진수를 이용한 불리언**(boolean)** 배열 **110**
비트연산자 기초 **+, -** **110**
+연산의 이해 111
-연산의 이해 111
비트연산자 기초 **&, |** **112**
비트연산자 기초 **<<, >>** **113**
왼쪽 시프트 연산자 << 114
오른쪽 시프트 연산자 >> 115
비트연산자 기초 **^** **116**
비트연산자 기초 **~** **117**
비트연산자 활용법 **119**
1. idx번째 비트끄기 : S &= ~(1 << idx) 120
2. idx번째 비트 XOR 연산 : S ^= (1 << idx) 121
3. 최하위 켜져있는 비트 찾기 idx = (S & -S) 123
4. 크기가 n인 집합의 모든 비트를 켜기 (1 << n) - 1 124
5. idx번째 비트를 켜기 S |= (1 << idx) 125
6. idx번째 비트가 켜져있는지 확인하기 if(S & (1 << idx)) 126
비트마스킹 **:** 경우의수**,** 매개변수 **127**
비스마스킹을 이용한 경우의 수 128
비트마스킹을 이용한 매개변수 전달 129

비트마스킹의 한계 **130**
그리디 **132**
그리디를 구현하는 방법 132
Q. 가장 지폐를 적게 내는 경우의 수 133
그리디의 조건 134
1. 최적부분구조 134
2. 탐욕 선택속성 135
직접증명과 간접증명 136
√2가 무리수임을 증명해보기 136
현실적인 방법 137
팁 138
Q. 큰돌은 욕심많은 도서관 사서야!!! 138
Q. 골동품 수집가 큰돌은 욕심쟁이야!!! 144
Q. 큰돌 교수님의 과제는 너무 어려워!!! 148
라인스위핑 **151**
Q. 큰돌이는 못말리는 화가야!!! 152
투포인터 **154**
Q. 큰돌이는 숫자매니아야!!! 155
이분탐색 **159**
이분탐색의 동작 원리 159
이분탐색의 시간복잡도 증명해보기 161
Q. 큰돌의 비밀 수첩 162
Q. 큰돌은 형광펜 질투쟁이야!!! 164
최대증가부분수열 **167**
Q. 큰돌의 LIS O(N^2) 168
Q. 큰돌의 LIS trace O(N^2) 170
Q. 큰돌의 LIS O(NlogN) 171
**DP**의 조건 **175**
**DP**를 시도할 때 **176**
**DP =** 완전탐색 **+** 메모이제이션 **177**
메모이제이션 **177**
Q. 큰돌의 사과나무 178
코드 180
**DP**의 구조 **180**
1. 초기화 (Initialization) 181
2. 기저 사례 (Base Case) 181
3. 메모이제이션 (Memoization) 181
4. 로직 (Logic) 182
**DP**의 종류 **182**
피보나치 수열 : 탑다운 방식 182
피보나치 수열 : 바텀업 방식 183
경우의 수를 푸는 **DP** **183**
Q. 큰돌의 숫자 놀이 184
코드 185

코드설명 186
펜윅트리 188
원리 188
코드 189
update() 190
sum(idx) 190
최단거리 알고리즘 191
1. 다익스트라 알고리즘 (Dijkstra's Algorithm) 191
원리 192
Q. 시작점에서 다른 모든 교차로까지의 최단 경로 찾기 192
코드구조 194
무한대 초기값 -> 초기값은 0 194
완화 195
느긋한 삭제 195
시간복잡도 195
2. 플로이드 워셜(Floyd-Warshall) 197
원리 197
Q. 모든 도시 간 최단 경로 197
코드 198
시간복잡도 199
3.벨만-포드(Bellman-Ford) 199
원리 200
Q. 음수 사이클 감지 200
코드 201
코드 분석 202
시간복잡도 203
4. 최단거리 알고리즘 비교 203
강의를 마무리하며 203
1.유니온파인드 205
find(x) 205
merge() 206
Q. 큰돌의 친구 그룹 만들기 206
코드 208
경로압축 209
트리의 깊이 최적화 210
시간 복잡도 211
2.LCS 212
예시 212

# 1주차 개념

# 시간복잡도와 빅오표기법

## 시간복잡도

복잡도는 시간복잡도와 공간복잡도로 나뉘어지는데 먼저 시간복잡도에 대해 알아보겠습니다.

**시간복잡도란 입력크기에 대해 어떠한 알고리즘이 실행되는데 걸리는 시간이며 주요로직의 반복횟수를 중점으로 측정됩니다.**

아니 시간이라고?

그렇다면 시간복잡도를 측정하기 위해서 항상 시간을 재야 할까요?

만약 어떠한 로직이 있고 그 로직에 걸리는 시간을 재려면 이렇게 재곤 해야 합니다.

```
console.time("test")
let sum = 0;
for(let i = 0; i < 1000000; i++){
  sum += 1;
}
console.timeEnd("test")
// test: 1.575ms
```

하지만 이러한 시간이라는 것은 컴퓨터 사양 등 여러가지 요소에 영향을 받곤 합니다.

그래서 시간복잡도를 설명할 때는 시간이 아니라 어떠한 알고리즘이 주어진 입력크기를 기반으로 어떠한 로직이 몇번 반복되었는가를 중점으로 설명합니다.

예를 들어 다음코드는 어떠한 시간복잡도를 가질까요?

```
for(int i = 0; i < 10; i++){
      for(int j =0; j < n; j++){
            for(int k = 0; k < n; k++){
                  if(true) cout << k << '\n';
            }
      }
}
for(int i = 0; i < n; i++){
      if(true) cout << i << '\n';
}
```

if(true) cout << k << '\n';라는 로직이 이만큼 반복되는 것을 알 수 있습니다.

10 * n^2 + n

이것이 바로 시간복잡도입니다.

## 빅오표기법

앞의 코드는 다음과 같은 시간복잡도를 가진다고 했죠?

10 * n^2 + n

이를 빅오 표기법으로 나타내면 다음과 같습니다.

O(n^2)

빅오 표기법(Big - O notation) 이란 **복잡도에 가장 영향을 많이 끼치는 항의 상수인자를 빼고 나머지 항을 없애서 복잡도를 나타내는 표기법**입니다.

참고로 다른 표기법들도 있지만 가장 많이 쓰이는 것은 빅오표기법입니다.

예를 들어 다음과 같은 시간복잡도에서 복잡도에 가장 많은 영향을 끼치는 항은 무엇일까요?

10 * n^2 + n

바로 n의 제곱항이고 다른 것은 그에 비해 미미하기 때문에 이것만 신경쓰면 된다는 것입니다.

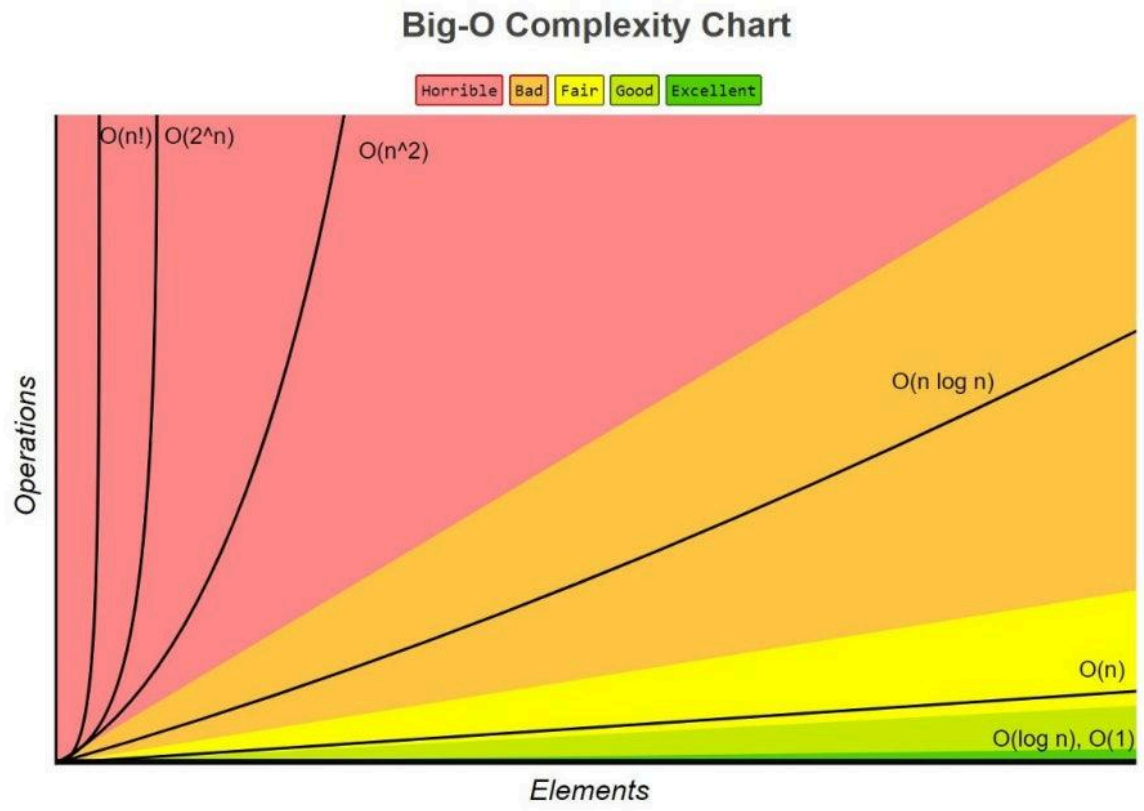
아니, n도 신경써야 되지 않나요? 라고 생각할 수 있지만 입력크기가 커질 때의 연산이 커지는 것을 볼까요?

예를 들어 n이 1, 3, 9, 12로 증가한다고 할 때 n^2은 1, 9, 81, 144로 증가하죠? n ^ 2이 월등히 크게 증가하는 것을 볼 수 있습니다.

예를 들어서 10 * n ^2 + n^ 2정도의 시간복잡도를 빅오표기법으로 표현하면 어떻게 될까요? n^2이 되게 됩니다.

자 다음 그림을 외워볼까요? 이를 외우고 있어야 나중에 빅오표기법을 나타낼 때 쉽게 표현할 수 있습니다.

n! > 2 ^ n > n ^2 > nlogn > n > logn > 1 순입니다.




## 상수시간 시간복잡도 O(1)

상수시간 시간복잡도는 입력크기와 상관없이 일정한 시간복잡도를 가지는 것을 말하며 O(1)의 시간복잡도를 씁니다.

예를 들어 다음과 같은 부분들이 모두 O(1)의 시간복잡도를 가집니다.

**입력과 출력**

ex) cin, cout, scanf, printf

**곱하기**

```
a[2] *= 2;
```

이외에도 곱하기, 나누기, 나머지연산, 빼기 등은 O(1)을 가집니다.

**간단한 비교 if문**

```
if(a[2] == 2)
```

**배열의 인덱스 참조**

```
int a[3] = {1, 2, 3};
int b = a[2];
```

## 문제로 연습하는 시간복잡도

자 그러면 문제를 통해 배워보도록 하겠습니다.

### Q1. 다음 코드의 시간복잡도는?

```
#include<bits/stdc++.h>
using namespace std;
int n;
int main(){
	cin >> n;
	int a = 0;
	for(int i = 0; i < n; i++){
		for(int j = 0; j < i; j++){
			a += i + j;
		}
	}
	cout << a << '\n';
	return 0;
}
```

답 : O(n ^ 2)

## Q2. 다음 코드의 시간복잡도는?

```
#include<bits/stdc++.h>
using namespace std;
int N, M;
void solve(int N, int M){
	int a = 1;
	for (int i = 0; i < N; i++) {
		a *= i;
	}
	for (int j = 0; j < M; j++) {
		a *= j;
	}
	cout << a << "\n";
}
int main(){
	cin >> N >> M;
	solve(N, M);
	return 0;
}
```

답 : O(N + M)

## Q3. 다음 코드의 시간복잡도는?

```
#include<bits/stdc++.h>
using namespace std;
int n, a[1004], cnt;
int go(int l, int r){
      if(l == r) return a[l];
      int mid = (l + r) / 2;
      int sum = go(l, mid) + go(mid + 1, r);
      return sum;
}
int main(){
      cin >> n;
      for(int i = 1; i <= n; i++){
            a[i - 1] = i;
      }
      int sum = go(0, n - 1);
      cout << sum << '\n';
}
/*
입력
10

출력
45
*/
```

답 : O(n)

## Q4. 다음 코드의 시간복잡도는?

로그(log)는 **지수 함수의 역함수**이다. 어떤 수를 나타내기 위해 고정된 밑을 몇 번 곱하여야 하는지를 나타냅니다.

```
#include<bits/stdc++.h>
using namespace std;
int N;
void solve(int N){
	int a = 0, i = N;
	while (i > 0) {
		a += i;
		i /= 2;
	}
	cout << a << '\n';
}
int main(){
	cin >> N;
	solve(N);
	return 0;
}
```

답 : O(logN)

## Q5. 다음 코드의 시간복잡도는?

```
#include<bits/stdc++.h>
using namespace std;
int N, cnt;
void solve(int N){
	cnt++;
	cout << cnt << '\n';
	if(N == 0) return;
	for(int i = 0; i < 3; i++){
		solve(N - 1);
	}
	return;
}
int main(){
	cin >> N;
	solve(N);
	return 0;
}
```

등비수열의 합.

① $\frac{a(r^n-1)}{r-1}$ $(a:\text{초항} \ / \ r:\text{공비} \ / \ n:\text{더하는것의 개수})$

② $\frac{a}{1-r}$ $(a:\text{초항} \ / \ r:\text{공비})$

답 : O(3 ^ n)

## [참고] 재귀함수에서의 메인로직은 무엇인가요?

**메인로직은 해당 함수에서 시간복잡도가 어느정도 큰 주요 로직이라고 보시면 됩니다.**

```
#include<bits/stdc++.h>
using namespace std;
int N, cnt;
void solve(int N){
	cnt++;
	cout << cnt << '\n';
	if(N == 0) return;
	for(int i = 0; i < 3; i++){
		solve(N - 1);
	}
	return;
}
int main(){
	cin >> N;
	solve(N);
	return 0;
}
```

이 solve라는 함수에서 주요한 로직은

```
   for(int i = 0; i < 3; i++){
		solve(N - 1);
	}
```

이부분입니다.

정확히는... 두가지의 로직이 합쳐져 있죠?

로직1 : 주어진 N에 대해 solve(N - 1)을 3번 재귀 호출

로직2 : 각 호출마다 cnt를 증가시키고 출력합니다.

여기서 메인로직이란 반복적으로(재귀적으로) 호출하는 부분이 시간복잡도가 크기 때문에 로직1이 됩니다.

**다만 재귀함수에서 시간복잡도를 산정할 때는 여러가지 로직 중에서 재귀적으로 호출되는 함수부분과**

**다른 부분을 분리시켜서 봐야 하고 재귀적으로 호출되지 않는 부분중에 가장 시간복잡도가 큰 부분이 중요한 메인로직이 됩니다.**

그리고 재귀함수의 시간복잡도는 해당 메인 로직 * 반복횟수로 결정됩니다.

```
void solve(int N){
	cnt++;
	cout << cnt << '\n';
	if(N == 0) return;
	for(int i = 0; i < 3; i++){
		solve(N - 1);
	}
	return;
}
```

이코드에서 반복되는 for문을 제외하고 가장 큰 부분은 cout 부분 출력 = O(1)이기 때문에(로직2가 메인로직이 된다.)

```
cout << cnt << '\n';
```

총 시간복잡도는 O(1) * 반복되는 횟수 = 3^n이 되어 3^n이 됩니다.

자 그러면 이코드의 시간복잡도는 얼마일까요?

```
void solve(int N){
    cnt++;
    for(int i = 0; i < N; i++){
        cout << cnt << '\n';
    }
    if(N == 0) return;
    for(int i = 0; i < 3; i++){
        solve(N - 1);
    }
    return;
}
```

O(N) * 3^n이 되겠죠?

반복적으로 호출되는 부분 제외 -> 가장 큰부분 = for문 x 반복횟수가 되기 때문입니다.

## 시간복잡도의 존재이유 : 효율적인 코드의 척도

자 그러면 이 시간복잡도는 왜 필요할까요? 바로 효율적인 코드로 개선하는데 쓰이는 척도가 됩니다.

**시간복잡도 O(n^)을 O(n)으로 줄였다.**

라고 해봅시다. 이는 어떤 버튼을 누르고 화면이 나타나는데 9초이며 이 시간복잡도가 n^2인데 n으로 개선시킨다면 3초가 될 수도 있는 것이죠.

자 그러면 알고리즘 문제를 풀 때를 보죠. n의 크기가 10만이라고 해봅시다. 그러면 n ^ 2의 시간복잡도면 100억입니다. 문제에 따라 다르긴 하지만 보통 100억이면 잘 안됩니다.

그럴 때는 "아 조금 더 작은 시간복잡도의 알고리즘을 써야 하겠구나. 로직을 개선해볼까? O(nlogn) 이하의 알고리즘을 써볼까?" 라고 하는 생각의 기준점이 되는 것입니다.

## 가능한 시간복잡도의 기준

문제마다 다르지만 100억이상 -> 절대 안됨 / 10억이상 -> 거의 안됨 / 1억 -> 어느정도는 됨 이렇게 생각하고 들어가는게 좋습니다.

+ 문제풀 때는 최고차항의 계수도 고려해야 합니다. 문제 범위기준 -> 시간복잡도 판단하는데요. 예를 들어 15000 * 15000 / 2 -> 시간복잡도 -> 어 ? 1억이네 될 수도 있겠네..? 이렇게 판단해야 합니다.

이 부분은 문제를 풀 때 매우 매우 중요합니다. 항상 문제를 보고 문제로 부터 어떻게 풀어야 하는지 로직을 생각한 후 그 로직의 시간복잡도를 얼추 계산하고 그리고 나서 시간초과가 뜰 것 같은면 다른 낮은 시간복잡도를 가진 로직을 빠르게 생각하는 것이 문제를 푸는데 있어 기본입니다.

# 자료구조의 시간복잡도

지금까지 배운 시간복잡도를 기반으로 자료구조를 쓸 때 적용해보겠습니다.어떠한 로직을 생각하고 그 로직에 알맞는 자료구조를 선택할 때 기준이 되는 것이 자료구조의 시간복잡도입니다.

예를 들어 어떠한 요소들의 모음에서 k번째 요소를 계속해서 "참조"해야 하는 로직이 있습니다. 그렇다면 이에 걸맞는 자료구조는 무엇일까요? 바로 배열입니다.

## 배열(Array)

- 참조 : O(1)

- 탐색 : O(n)

## 연결리스트(doubly linked list)

- 참조 : O(n)

- 탐색 : O(n)

- 삽입 / 삭제 : O(1)

앞의 시간복잡도처럼 연결리스트를 쓰게 된다면 참조에 매번 O(k)의 시간복잡도가 드는 반면 배열은 O(1)의 시간복잡도가 걸리기 때문입니다.

이러한 생각을 자연스레 하려면 자주 쓰는 자료구조들의 시간복잡도가 대략적으로 얼마인지를 알고 있어야 합니다.

보통 시간복잡도를 고려할 때 평균 또는 최악의 시간복잡도를 기반으로 생각을 하는데 문제를 풀 때는 최악의 시간복잡도를 기반으로 해야 합니다.

예를 들어 어떠한 코딩테스트나 알고리즘 문제가 있다고 해봅시다. 이 문제에 나의 코드를 제출했는데 시간초과가 떴습니다. 이 상황은 어떠한 상황일까요?

바로 제가 제출한 코드가 문제 내부에 있는 (외부로 주어지는 테스트케이스 말고) 테스트케이스에 나의 코드가 시간초과가 된 상황입니다. 평균적으로 나의 자료구조는 시간복잡도가 O(1)이고 최악은 O(n)일 때 해당 테스트케이스에서는 O(1)이 아니라 O(n)인 상황이 발생해버린 것이죠.

자, 그러면 최악의 시간복잡도를 기반으로 대표적인 자료구조들을 설명해보겠습니다.자주 일어나는 연산만을 기반으로 설명하며 자주 일어나지 않은 연산은 제외해서 설명합니다.

예를 들어 정적 배열은 삭제 연산이 자주 일어나지 않기 때문에 제외하고 설명합니다.

```
int a[1004];
```

참고로 앞의 코드와 같은 정적배열을 구축했을 때 k번째 해당 요소를 삭제해야 하는 로직이 발생했을 때 실제로 삭제하는 것이 아니라 삭제했다는 표기를 위해 a[k] = -1; 이런식으로 어떠한 값을 설정하는 것으로 삭제를 구현합니다.

## 배열(Array)

- 참조 : O(1)

- 탐색 : O(n)

## 배열(vector)

- 참조 : O(1)

- 탐색 : O(n)

- 맨 끝, 앞에 삽입/삭제 : O(1)

- 중간에 삽입 / 삭제 : O(n)

## 스택(stack)

- n번째 참조 : O(n)

- 가장 앞부분 참조 : O(1)

- 탐색 : O(n)

- 삽입 / 삭제(n번째 제외) : O(1)

## 큐(queue)

- n번째 참조 : O(n)

- 가장 앞부분 참조 : O(1)

- 탐색 : O(n)

- 삽입 / 삭제(n번째 제외) : O(1)

## 연결리스트(doubly linked list)

- 참조 : O(n)

- 탐색 : O(n)

- 삽입 / 삭제 : O(1)

## 맵(map)

- 참조 : O(logn)

- 탐색 : O(logn)

- 삽입 / 삭제 : O(logn)

이 외에 자료구조들의 시간복잡도는 다음과 같습니다. 앞서 설명한 것은 반드시 외워주시고 아래의 표는 참고삼아 봐주시면 됩니다.

**Common Data Structure Operations**

| Data Structure | Time Complexity | | | | | | | | Space Complexity |
|---|---|---|---|---|---|---|---|---|---|
| | Average | | | | Worst | | | | Worst |
| | Access | Search | Insertion | Deletion | Access | Search | Insertion | Deletion | |
| Array | Θ(1) | Θ(n) | Θ(n) | Θ(n) | O(1) | O(n) | O(n) | O(n) | O(n) |
| Stack | Θ(n) | Θ(n) | Θ(1) | Θ(1) | O(n) | O(n) | O(1) | O(1) | O(n) |
| Queue | Θ(n) | Θ(n) | Θ(1) | Θ(1) | O(n) | O(n) | O(1) | O(1) | O(n) |
| Singly-Linked List | Θ(n) | Θ(n) | Θ(1) | Θ(1) | O(n) | O(n) | O(1) | O(1) | O(n) |
| Doubly-Linked List | Θ(n) | Θ(n) | Θ(1) | Θ(1) | O(n) | O(n) | O(1) | O(1) | O(n) |
| Skip List | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | O(n) | O(n) | O(n) | O(n) | O(n log(n)) |
| Hash Table | N/A | Θ(1) | Θ(1) | Θ(1) | N/A | O(n) | O(n) | O(n) | O(n) |
| Binary Search Tree | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | O(n) | O(n) | O(n) | O(n) | O(n) |
| Cartesian Tree | N/A | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | N/A | O(n) | O(n) | O(n) | O(n) |
| B-Tree | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | O(log(n)) | O(log(n)) | O(log(n)) | O(log(n)) | O(n) |
| Red-Black Tree | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | O(log(n)) | O(log(n)) | O(log(n)) | O(log(n)) | O(n) |
| Splay Tree | N/A | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | N/A | O(log(n)) | O(log(n)) | O(log(n)) | O(n) |
| AVL Tree | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | O(log(n)) | O(log(n)) | O(log(n)) | O(log(n)) | O(n) |
| KD Tree | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | Θ(log(n)) | O(n) | O(n) | O(n) | O(n) | O(n) |

# 공간복잡도

**공간복잡도는 "입력크기에 대해 어떠한 알고리즘이 실행되는데 필요한 메모리 공간의 양 "을 가리킵니다.**

이는 정적변수로 선언된 것 말고도 동적으로 재귀적인 함수로 인해 공간을 계속해서 필요로 할 경우도 포함하며 배열이든 맵이든 셋이든 요소들을 담을 공간이면 다 적용됩니다.

다음 코드에서 int형 배열 1000만짜리 1개, 1004개짜리 1개를 선언했죠? 둘 다 포함해서 공간복잡도를 계산합니다.

```
int a[10000000];
void f(){
      int b[1004];
}
int main(){

}
```

자 예를 들어 입력이 N이 들어왔고 거기에 따라 N개 짜리 int형 요소를 담아야 할 공간이 필요하다면 4N바이트의 공간이 필요하겠죠?

이를 빅오표기법으로 나타내면 O(N)이 됩니다. 상수는 버리니까요.

```
int a[N];
```

그러나 사실 이러한 O(N) 등의 공간복잡도의 개념은 "문제"를 푸는데 잘 사용되지 않습니다.

보통은 문제의 최대범위, 메모리 제한 이 2가지를 기반으로 공간복잡도의 상한선을 잡고 메모리초과가 나지 않게 배열의 크기를 선언합니다.

| | | | |
|---|---|---|---|
| 14021082 | zagabi | 17071 | 메모리 초과 |

이를 잘 설정하지 않을 경우 앞의 그림과 같이 메모리초과가 뜨게 될 것입니다.

## 1. 문제의 최대범위

먼저 문제에서 주어진 최대범위를 기반으로 배열을 미리 만들어야 합니다.

예를 들어 다음 문제에서 int타입 N의 최대범위는 얼마죠? 1,000,000입니다. 무려 100만이죠? 그렇다면 무조건 100만짜리 배열을 선언해야 합니다.

입력

첫째 줄에 수의 개수 N(1 ≤ N ≤ 1,000,000)과 M(1 ≤ M ≤ 10,000), K(1 ≤ K ≤ 10,000) 가 주어진다. M은 수의 변경이 일어나는 횟수이고, K는 구간의 합을 구하는 횟수이다. 그리고 둘째 줄부터 N+1번째 줄까지 N개의 수가 주어진다. 그리고 N+2번째 줄부터 N+M+K+1번째 줄까지 세 개의 정수 a, b, c가 주어지는데, a가 1인 경우 b(1 ≤ b ≤ N)번째 수를 c로 바꾸고 a가 2인 경우에는 b(1 ≤ b ≤ N)번째 수부터 c(b ≤ c ≤ N)번째 수까지의 합을 구하여 출력하면 된다.

그렇다면 다음과 같이 선언을 해야 합니다.

```
int a[1000000];
```

즉, 문제에서 나올 수 있는 최대범위이상에 해당하는 배열은 무조건 만들어야 하는 것이죠.

## 2. 문제의 메모리제한

다음은 문제의 메모리제한을 보는 것이입니다. 이제 문제에 필요한 배열은 만들었겠다... 해당 부분 말고 다른 로직들의 경우 메모리제한을 기반으로 설정을 합니다.

다음 문제의 메모리 제한은 512MB네요. 아 그러면 계산해볼까요? 512MB는 512,000,000 바이트네요? 그러면 int a[128000000];을 선언할 수 있겠네요?

연구소 성공

| 시간 제한 | 메모리 제한 | 제출 | 정답 | 맞힌 사람 | 정답 비율 |
|---|---|---|---|---|---|
| 2 초 | 512 MB | 69853 | 40210 | 22145 | 55.015% |

자, 근데 이 문제는 2차원배열이 필요하니 쪼개야 합니다. 적당히 10000 * 10000으로 해서 집어넣어볼게요. 즉, 1억짜리 배열을 만들어봅시다.

```
#include<bits/stdc++.h>
using namespace std;
int a[10000][10000], visited[10][10], n, m, ret;
vector<pair<int, int>> virusList, wallList;
const int dy[] = {-1, 0, 1, 0};
const int dx[] = {0, 1, 0, -1};
void dfs(int y, int x){
    for(int i = 0; i < 4; i++){
```

```
        int ny = y + dy[i];
        int nx = x + dx[i];
        if(ny < 0 || ny >= n || nx < 0 || nx >= m || visited[ny][nx] ||
a[ny][nx] == 1) continue;
        visited[ny][nx] = 1;
        dfs(ny, nx);
    }
    return;
}
int solve(){
    fill(&visited[0][0], &visited[0][0] + 10 * 10, 0);
    for(pair<int, int> b : virusList){
        visited[b.first][b.second] = 1;
        dfs(b.first, b.second);
    }

    int cnt = 0;
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            if(a[i][j] == 0 && !visited[i][j])cnt++;
        }
    }
    return cnt;
}
int main(){
    cin >> n >> m;
    for(int i = 0; i <n; i++){
        for(int j = 0; j < m; j++){
            cin >> a[i][j];
            if(a[i][j] == 2) virusList.push_back({i, j});
            if(a[i][j] == 0) wallList.push_back({i, j});
        }
    }
    for(int i = 0; i < wallList.size(); i++){
        for(int j = 0; j < i; j++){
            for(int k = 0; k < j; k++){
                a[wallList[i].first][wallList[i].second] = 1;
                a[wallList[j].first][wallList[j].second] = 1;
                a[wallList[k].first][wallList[k].second] = 1;
                ret = max(ret, solve());
                a[wallList[i].first][wallList[i].second] = 0;
                a[wallList[j].first][wallList[j].second] = 0;
                a[wallList[k].first][wallList[k].second] = 0;
            }
        }
```

```
    }
    cout << ret << "\n";
    return 0;
}
```

| 제출 번호 | 아이디 | 문제 | 결과 | 메모리 | 시간 | 언어 | 코드 길이 | 제출한 시간 |
|---|---|---|---|---|---|---|---|---|
| 51220076 | zagabi | 14502 | 맞았습니다!! | 392648 KB | 40 ms | C++17 / 수정 | 1708 B | 38초 전 |

자, 이렇게 맞는다는 것을 볼 수 있습니다. 무려 1억짜리 배열을 만들었는데도 불구하고 통과하는 것을 볼 수 있습니다.

여기서 메모리제한의 경우 매번 신경쓰기가 어렵습니다. 그래서 다음과 같이 생각하는게 좋습니다.

**int타입 배열 기준, 보통은 3000만까지는 된다.**

정확히는 메모리제한 - int타입 기준으로 다음과 같습니다.

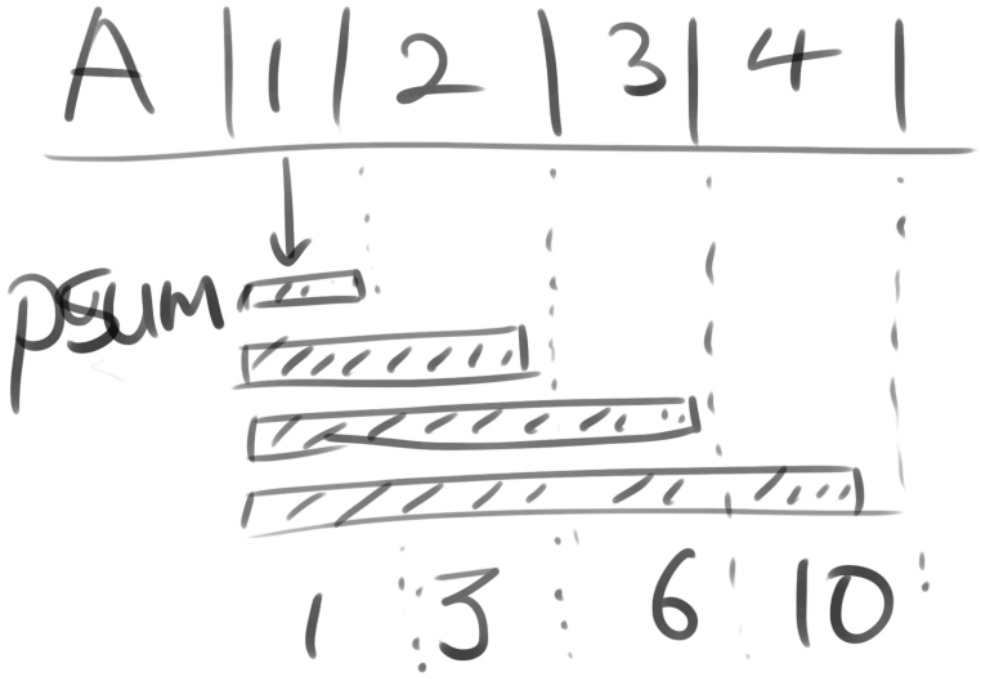
- 64MB : 1500만
- 128MB : 3000만
- 256MB : 6000만
- 512MB : 1억2천만

보통 최저로 메모리를 128MB까지는 주기 때문에 보통은 3000만이다. 라고 보시면 됩니다.

## 누적합

누적합이란 요소들의 누적된 합의 의미로 어떠한 배열을 기반으로 앞에서 부터 요소들의 누적된 합을 저장해 새로이 배열을 만들어서 이를 활용하는 것을 말합니다.

이는 앞에서부터 더하는 prefix sum과 뒤에서 부터 더하는 suffix sum 이 있지만 코딩테스트에는 prefix sum만 나오니 prefix sum만을 배우면 됩니다.




위그림처럼 누적합이란 누적해서 더해진 값들의 배열을 의미합니다. 1, 2, 3, 4라는 배열의 누적합은 1, 3, 6, 10이란 배열이 됩니다.

이렇게 만들어놓으면 뭐가 좋을까요? 바로 구간쿼리에 대응하기가 쉽습니다. 예를 들어 0번째 요소부터 3번째요소까지 다 더해라고 했을 때 누적합배열 3번째요소를 이용하면 되니까요.

문제를 풀 때 "구간"에 대한 많은 "쿼리"가 나올 때 생각해야 될 것은 트리 또는 누적합입니다. 여기서 트리는 세그먼트, 펜윅트리를 뜻합니다. 이 때 그 구간 안에 있는 요소들이 변하지 않는 정적 요소라면 누적합을 쓰면 됩니다.

## Q1. 잃어버린 승철이의 뇌

승철이는 뇌를 잃어버렸다. 학교에 갔더니 선생님이 자연수로 이루어진 N개의 카드를 주며 M개의 질문을 던진다. 그 질문은 나열한 카드 중 A번째부터 B번째까지의 합을 구하는 것이다. 뇌를 잃어버렸기 때문에 승철이는 이 문제를 풀 수 없다. 문제를 풀 수 있는 프로그램을 작성해보자.

**입력**

수의 개수 N, 합을 구해야 하는 횟수 M, 그 이후 N개의 수가 주어진다. 수는 100 이하의 자연수. 그 이후 M개의 줄에는 합을 구해야 하는 구간 A, B가 주어진다.

**출력**

M개의 줄에 A부터 B까지의 합을 구하라.

**범위**

1 <= N <= 100,000

1 <= M <= 100,000

1 <= A <= B <= N

**예제입력**

8 3

1 2 3 4 5 6 7 8

1 4

1 5

3 5

**에제출력**

10

15

12

이문제를 무식하게 구현했을 때 어떻게 될까요?

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
int a[100004], b, c, psum[100004], n ,m;
int main(){
	cin >> n >> m;
	for(int i = 1; i <= n; i++){
		cin >> a[i];
	}
	for(int i = 0 ; i < m; i++){
		cin >> b >> c;
		int sum = 0;
		for(int j = b; j <= c; j++) sum += a[j];
		cout << sum << '\n';
	}
	return 0;
}
```

이런꼴이 되겠죠? 위의 코드는 시간복잡도는 O(10만 * 10만) 이며 A와 B가 주어졌을 때 한 연산당 시간복잡도는 O(10만) 입니다. 즉, 단순하게 접근하면 시간초과가 나는 것이죠.

이럴 때 "누적합"을 써야 합니다. 쿼리가 주어졌고 배열은 변함없는 정적요소이기 때문이죠.

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
int a[100004], b, c, psum[100004], n ,m;
int main(){
	ios_base::sync_with_stdio(false); cin.tie(NULL); cout.tie(NULL);
	cin >> n >> m;
	for(int i = 1; i <= n; i++){
		cin >> a[i];
		psum[i] = psum[i - 1] + a[i];
	}
	for(int i = 0 ; i < m; i++){
```

```
            cin >> b >> c;
            cout << psum[c] - psum[b - 1] << "\n";
        }
        return 0;
}
```

**코드설명**

```
psum[i] = psum[i - 1] + a[i];
```

누적합을 만들 때는 반드시 "1번째 요소부터" 만드는 것이 좋습니다. 왜냐면 i = i - 1 이부분이 있기 때문에 0부터 시작한다면 -1이 되기 때문입니다.

```
psum[c] - psum[b - 1]
```

그 다음 이렇게 빼기만 하면 A부터 B까지 구해라 라는 것을 쉽게 O(1)만에 할 수 있습니다.

# 구현

사실 구현은 아주 쉬운 알고리즘입니다. 말그대로 문제 그대로 구현을 하면 됩니다. 예를 들어 배열을 회전하라 스택에 넣어라 라고 한다면 말그대로 rotate 함수를 만들어서 호출하고 stack.push 등을 하면 되는 것입니다.

## Q1. 도파 문자열 다루기

다음과 같이 문자열을 선언했다고 하고 아래의 문제를 풀어보겠습니다.

```
string dopa = "umzunsik";
```

Q1. 앞에서부터 3개의 문자열을 출력하라

Q2. 해당 문자열을 거꾸로 해서 출력하라.

Q3. 거꾸로된 해당 문자열 끝에 "umzunsik"이란 문자열을 추가하라.

처음에는 잘 안될 수도 있습니다.

이 때는 주석을 달고 코드를 구축하면 됩니다.

코드는 다음과 같습니다.

```
#include<bits/stdc++.h>
using namespace std;
string dopa = "abcde";
int main(){
	cout << dopa << "\n";
	//문자열에서 부분배열
	cout << dopa.substr(0, 3) << "\n";
	// 반대로 하는 것.
	reverse(dopa.begin(), dopa.end());
	cout << dopa << "\n";
	// 추가해서 출력하는 것.
```

```
        dopa += " umzunsik";
        cout << dopa << '\n';
        return 0;
}
/*
abcde

abc
edcba
edcba umzunsik
*/
```

**함수설명**

- reverse : 원래의 문자열을 바꿔버립니다. begin과 end를 통해 전체를 바꿔도 되고 dopa.begin(), dopa.begin() + 3 이런식으로 부분만 바꿔버릴 수도 있습니다.
- substr : 시작지점으로부터 몇개의 문자열을 뽑아냅니다.

## Q2. 오름차순과 평균값

숫자로 이루어진 배열이 주어졌을 때, 배열의 요소들을 오름차순으로 정렬하고, 배열의 평균값을 계산하는 프로그램을 작성하세요.

**입력**

첫 번째 줄에 배열의 크기 N (1 ≤ N ≤ 100)이 주어집니다. 두 번째 줄에 N개의 정수가 주어집니다. 각 정수는 -1000 이상 1000 이하입니다.

**출력**

첫 번째 줄에 정렬된 배열을 출력하고, 두 번째 줄에 배열의 평균값을 출력하세요. 평균값은 소수점 두 번째 자리까지 출력하세요.

**예시입력**

5

3 1 4 1 5

**예시출력**

1 1 3 4 5

2.80

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N;
    cin >> N;

    vector<int> numbers(N);
    for(int i = 0; i < N; ++i) {
        cin >> numbers[i];
    }

    // 배열을 정렬합니다
    sort(numbers.begin(), numbers.end());

    // 평균을 계산합니다
    double sum = 0;
    for(int i = 0; i < N; ++i) {
        sum += numbers[i];
    }
    double average = sum / N;

    // 정렬된 배열을 출력합니다
    for(int i = 0; i < N; ++i) {
        cout << numbers[i] << " ";
    }
    cout << "\n";

    // 평균값을 출력합니다
    cout << fixed << setprecision(2) << average << "\n";

    return 0;
}
```

# 문제를 푸는 방법

우리는 문제를 풀 때 다음과 같은 과정을 통해 문제를 풀어야 합니다.

문제해석 -> 손코딩 -> 코드를 작성 O -> 반례체크

즉, 바로 코드를 작성하면 안됩니다.

물론 숙달이 되었다면 바로 코드를 작성해도 되지만 초창기에는 손코딩을 항상 먼저 하고 그 다음 코드를 작성해야 합니다.

## 문제 해석

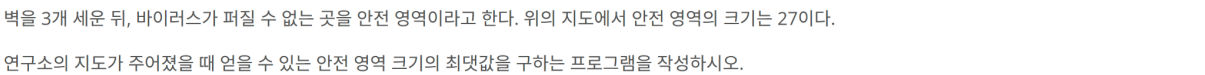
예를 들어 연구소 문제를 볼까요?

인체에 치명적인 바이러스를 연구하던 연구소에서 바이러스가 유출되었다. 다행히 바이러스는 아직 퍼지지 않았고, 바이러스의 확산을 막기 위해서 연구소에 벽을 세우려고 한다.

연구소는 크기가 N×M인 직사각형으로 나타낼 수 있으며, 직사각형은 1×1 크기의 정사각형으로 나누어져 있다. 연구소는 빈 칸, 벽으로 이루어져 있으며, 벽은 칸 하나를 가득 차지한다.

일부 칸은 바이러스가 존재하며, 이 바이러스는 상하좌우로 인접한 빈 칸으로 모두 퍼져나갈 수 있다. 새로 세울 수 있는 벽의 개수는 3개이며, 꼭 3개를 세워야 한다.

예를 들어, 아래와 같이 연구소가 생긴 경우를 살펴보자.



벽을 3개 세운 뒤, 바이러스가 퍼질 수 없는 곳을 안전 영역이라고 한다. 위의 지도에서 안전 영역의 크기는 27이다.

연구소의 지도가 주어졌을 때 얻을 수 있는 안전 영역 크기의 최댓값을 구하는 프로그램을 작성하시오.

### 입력

첫째 줄에 지도의 세로 크기 N과 가로 크기 M이 주어진다. (3 ≤ N, M ≤ 8)

둘째 줄부터 N개의 줄에 지도의 모양이 주어진다. 0은 빈 칸, 1은 벽, 2는 바이러스가 있는 위치이다. 2의 개수는 2보다 크거나 같고, 10보다 작거나 같은 자연수이다.

빈 칸의 개수는 3개 이상이다.

이 문제를 다음과 같이 해석할 수 있습니다

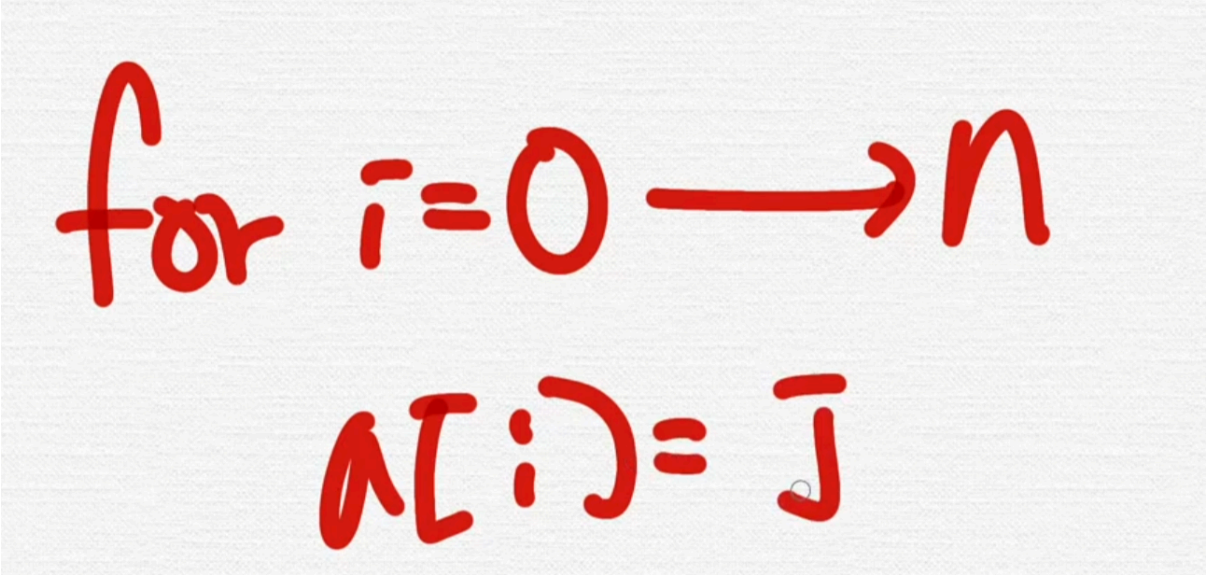
벽을 세우고 -> 바이러스를 퍼뜨린 다음 -> 안전 영역을 구하여 -> 그 중 최대 크기를 출력합니다.

이처럼 문제를 도식화하는 것이 중요합니다.

그 다음 부터는 그 도식화한 부분들에 맞춰 구현을 하면 되는 것이죠. 벽을 세우는 것? 효율적으로 세울까? 무식하게 모든 경우의 수를 다 따져가면서 할까? (brute force) 등 어떤 알고리즘 개념을 적용할 건지는 그 이후에 고민하는 것입니다.

## 손코딩

손코딩은 다음과 같이 본인이 알아볼 수 있게만 하면 됩니다. 굳이 손코딩으로 for(int i = 0; i < ... 이렇게 적을 필요는 없습니다.




## 문제푸는 과정 정리

이 강의를 통해 우리는 이 4가지의 플로우를 숙달하는 것을 목표로 삼습니다.

1. 단순구현이라면 구현하고 -> 문제의 최대, 최소 범위를 파악합니다.

2. 최대범위 기반, 시간복잡도상 -> 무식하게 풀 수 있다면 무식하게 모든 경우의 수를 생각하면서 풀고 -> 이 때 구현할 부분들은 구현하자

3. 그게 아니라면 다른 알고리즘을 생각하자.

4. 제출하기전, 반례를 항상 생각하자. (외부 TC와 내부TC는 항상 다르다. 내부 TC를 생각해보자.)

**반드시 먼저 코드를 작성하지 말고 해석하고! 손코딩 먼저 하자!**

# 2주차 개념

이번 2주차는 그래프이론과 DFS(깊이우선탐색), BFS(너비우선탐색) 그리고 트리순회인 preorder, inorder, postorder를 다루겠습니다.

# 그래프이론

그래프이론은 오일로경로, SCC, 단절점 등의 어려운 개념들도 많고 넓은 범위를 다루는 이론이지만

오늘은 기초중의 기초를 배워보겠습니다.

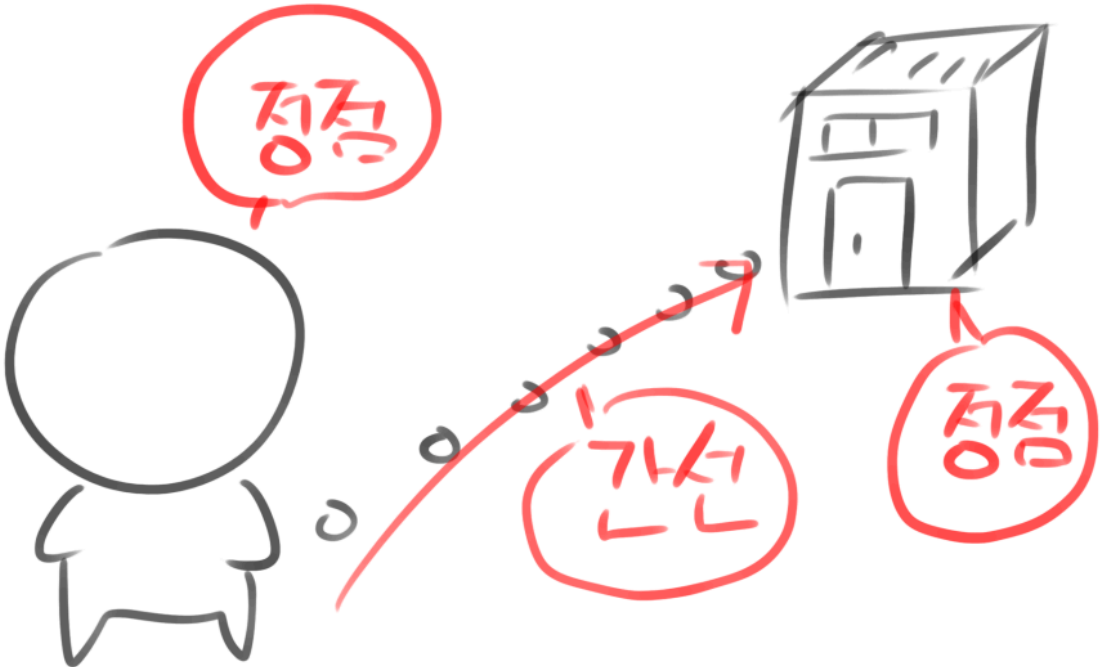
## 정점과 간선

정점(vertex)는 노드라고도 불리며 그래프를 형성하는 기본 단위입니다.

정점은 분할할 수 없는 객체이자 "점"으로 표현되는 위치, 사람, 물건 등이 될 수 있습니다.

간선(Edge)은 정점을 잇는 선을 의미합니다. 관계, 경로 등이 될 수 있습니다.

예를 들어 **"어떠한 위치나 어떠한 사람"**으로부터 "**무언가를 통해 간다"**라고 했을 때 **"어떠한 위치나 어떠한 사람"** 정점(Vertex)이 되고 **"무언가를 통해 간다"**는 간선(Edge)이 됩니다.



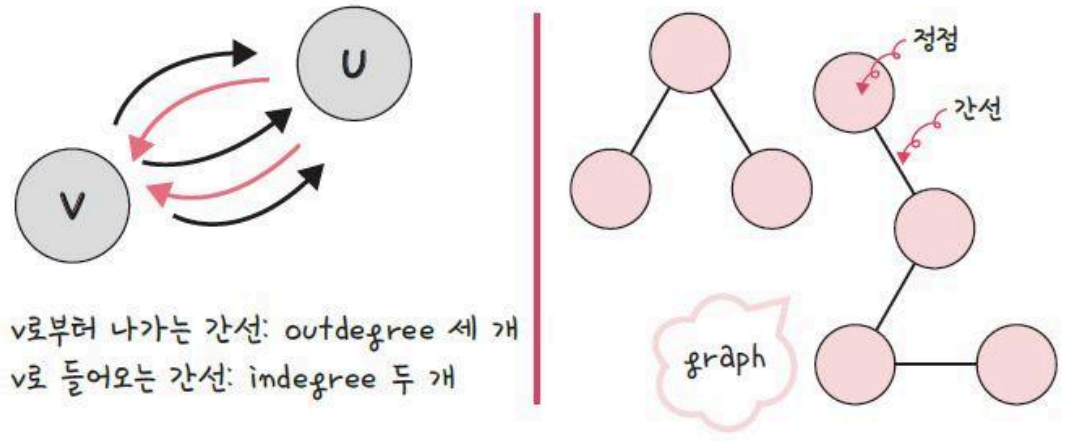
필자가 위 그림처럼 어떤 아파트로 간다고 해봅시다. 저와 아파트는 하나의 정점이고 거기로 가는 길은 간선이 됩니다.

필자가 어떤 사람을 좋아한다고 해봅시다. 필자와 어떤 사람은 정점이 이러한 마음은 간선이 됩니다. 자 이 때 제가 만약에 어떤 사람을 좋아하는데 어떤 사람은 저를 좋아하지 않는 "짝사랑"이라고 해봅시다. 이것은 어떠한 간선일까요? 바로 단방향 간선입니다. 저의 마음만이 그 사람으로 가는 길밖에 없는 슬픈 관계이지요.

하지만 그 어떤 사람도 저를 좋아한다면 어떻게 될까요? 바로 양방향 간선이 됩니다. 저의 마음도 그 어떤 사람의 마음도 갈 수 있는 "사랑"이라 불리는 상태인 것이죠.

## indegree outdegree

▼ 그림 5-12 그래프




정점으로 나가는 간선을 해당 정점의 outdegree라고 하며 들어오는 간선을 해당정점의 indegree라고 합니다. 해당 정점은 outdegree는 3개 indegree는 2개인 상태입니다. 또한 정점은 약자로 V또는 U라고 하며 보통 어떤 정점으로부터 시작해서 어떤 정점까지를 간다를 "U에서부터 V로 간다" 라고 표현을 합니다.

## 가중치

정점과 정점사이에 드는 비용을 뜻합니다. 1번노드와 2번노드까지 가는 비용이 한칸이라면 1번노드에서 2번노드까지의 가중치는 한칸입니다.

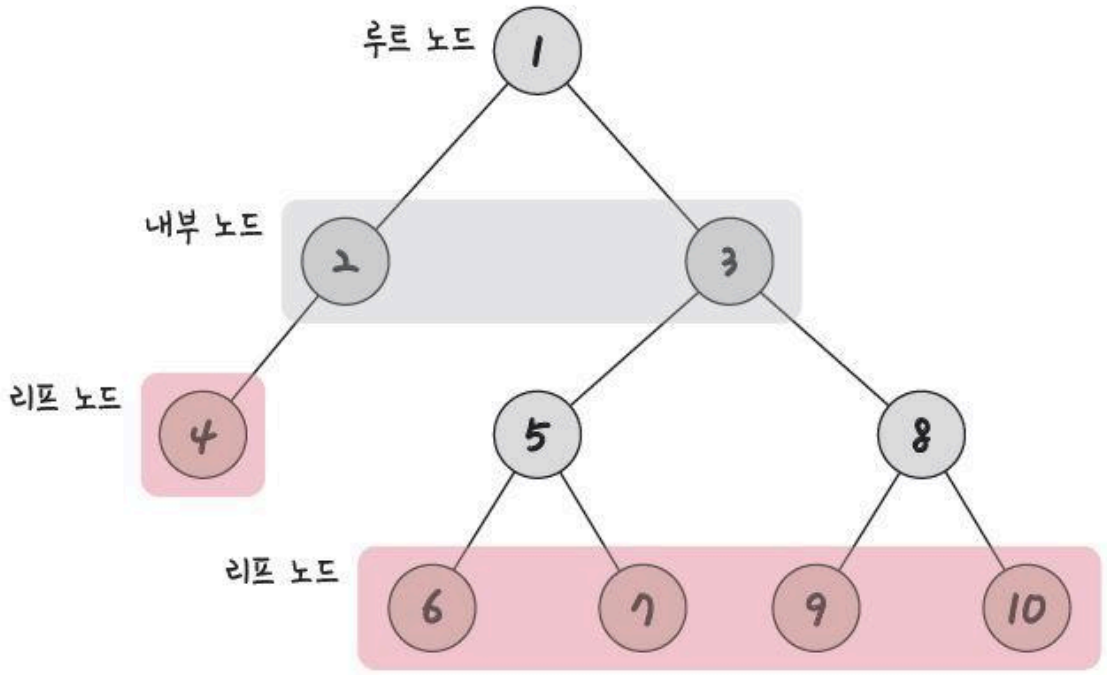
예를 들어 제가 성남이라는 정점에서 네이버라는 정점까지 걸리는 택시비가 13000원이라면 성남에서 네이버까지 가는 가중치는 13000원이 됩니다.

지금까지 설명한 정점과 간선들로 이루어진 집합을 "그래프(Graph)"라고 합니다.

## 트리

트리는 자식노드와 부모노드로 이루어진 계층적인 구조를 가지며 무방향 그래프의 일종이자 사이클이 없는 자료구조를 의미합니다.

▼그림 5-13 트리의 특징




## 트리의 특징

트리는 그래프의 일종이며 다음 특징을 가집니다.

1. 부모, 자식 계층 구조를 가집니다. 지금 보면 5번 노드는 6번 노드와 7번 노드의 부모 노드이고, 6번노드와 7번 노드는 5번 노드의 자식 노드입니다. 같은 경로 상에서 어떤 노드보다 위에 있으면 부모,아래에 있으면 자식 노드가 됩니다.

2. V - 1 = E라는 특징이 있습니다. 간선 수는 노드 수 - 1입니다.

3. 임의의 두 노드 사이의 경로는 '유일무이'하게 '존재'합니다. 즉, 트리 내의 어떤 노드와 어떤 노드까지의 경로는 반드시 있으며 하나밖에 없습니다.

## 루트노드

가장 위에 있는 노드를 뜻합니다. 보통 트리문제가 나오고 트리를 탐색할 때 루트노드를 중심으로 탐색을 하면 문제가 쉽게 풀리는 경우가 많습니다.

## 내부노드

루트노드와 리프노드 사이에 있는 노드를 뜻합니다.

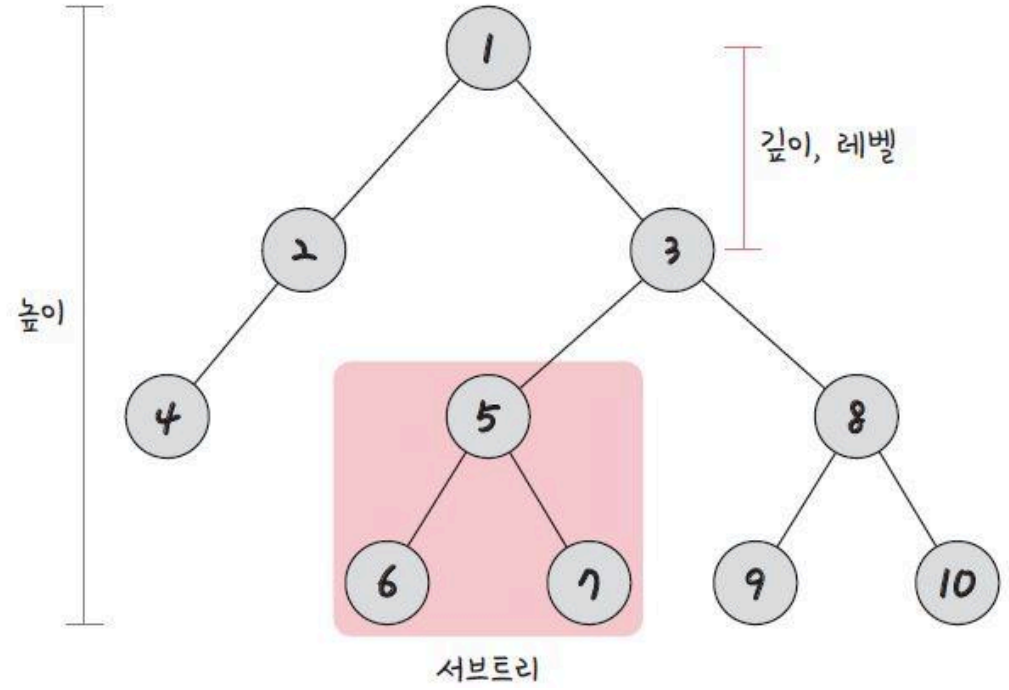
## 리프노드

실제 알고리즘 고인물과 카톡한 내용입니다. 아래그림처럼 리프노드는 자식노드가 없는 노드를 뜻합니다.

## 트리의 높이와 레벨

다음 그림은 트리의 높이와 레벨을 설명한 그림입니다.

▼ 그림 5-15 트리의 높이와 레벨




- 깊이: 트리에서의 깊이는 각각의 노드마다 다르며 루트 노드에서 특정 노드까지 최단거리로 갔을 때의 거리를 말합니다. 예를 들어 4번 노드의 깊이는 2입니다.

- 높이: 트리의 높이는 루트 노드부터 리프 노드까지의 거리 중 가장 긴 거리를 의미하며 앞 그림의 트리 높이는 3입니다.

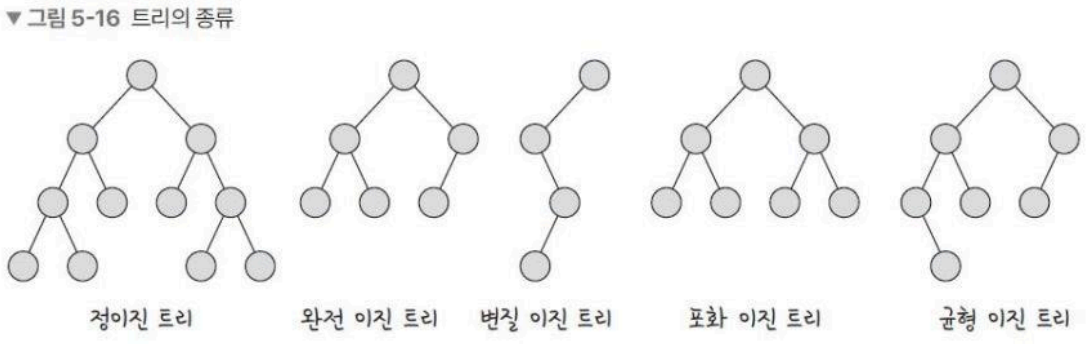
- 레벨: 트리의 레벨은 주어지는 문제마다 조금씩 다르지만 보통 깊이와 같은 의미를 지닙니다. 1번 노드를 0레벨이라고 하고 2번 노드, 3번 노드까지의 레벨을 1레벨이라고 할 수도 있고, 1번 노드를 1레벨이라고도 할 수 있습니다. 예를 들어 1번 노드를 1레벨이라고 한다면 2번 노드와 3번 노드는 2레벨이 됩니다.

- 서브트리: 트리 내의 하위 집합을 서브트리라고 합니다. 트리 내에 있는 부분집합이라고도 보면 됩니다. 지금 보면 5번, 6번, 7번 노드가 이 트리의 하위 집합으로 "저 노드들은 서브트리다"라고 합니다.

[참고] 트리로 이루어진 집합을 숲(forest)이라고 합니다.

# 이진트리(BT, Binary Tree)

각각의 노드의 자식노드 수가 2개 이하로 구성되어있는 트리를 의미하며 이를 다음과 같이 분류합니다.




- 정이진 트리(full binary tree): 자식 노드가 0 또는 2개인 이진 트리를 의미합니다.

- 완전 이진 트리(complete binary tree): 왼쪽에서부터 채워져 있는 이진 트리를 의미합니다. 마지막 레벨을 제외하고는 모든 레벨이 완전히 채워져 있으며 마지막 레벨의 경우 왼쪽부터 채워져 있습니다.

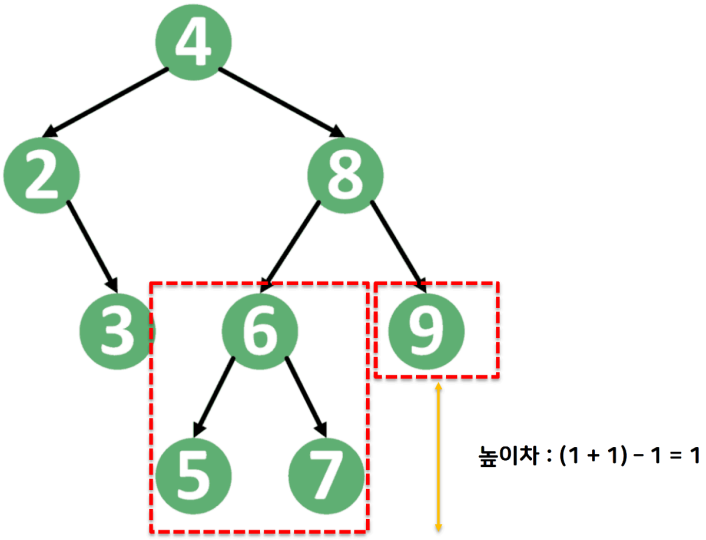
- 변질 이진 트리(degenerate binary tree): 리프노드 제외, 자식 노드가 하나밖에 없는 이진 트리를 의미합니다.

- 포화 이진 트리(perfect binary tree): 모든 노드가 꽉 차 있는 이진 트리를 의미합니다.

- 균형 이진 트리(balanced binary tree): 균형 이진 트리는 트리의 모든 노드에 대해, 그 노드의 왼쪽 하위 트리와 오른쪽 하위 트리의 높이 차이가 1 이하인 이진 트리입니다. 이 말은, 트리의 어느 한쪽이 지나치게 치우치지 않도록 구조가 유지된다는 것을 의미합니다. 이러한 균형 덕분에, 탐색, 삽입, 삭제 등의 연산에서 시간 복잡도가 O(log n)으로 보장될 수 있습니다.

## 균형이진트리의 높이 차이 계산

높이차이는 각 하위 트리의 전체 높이를 기준으로 계산합니다. 예를 들어 한 노드의 왼쪽 하위 트리의 높이가 1이고, 오른쪽 하위 트리의 높이가 0이라면, 이 노드의 하위 트리 높이 차이는 1이 됩니다.

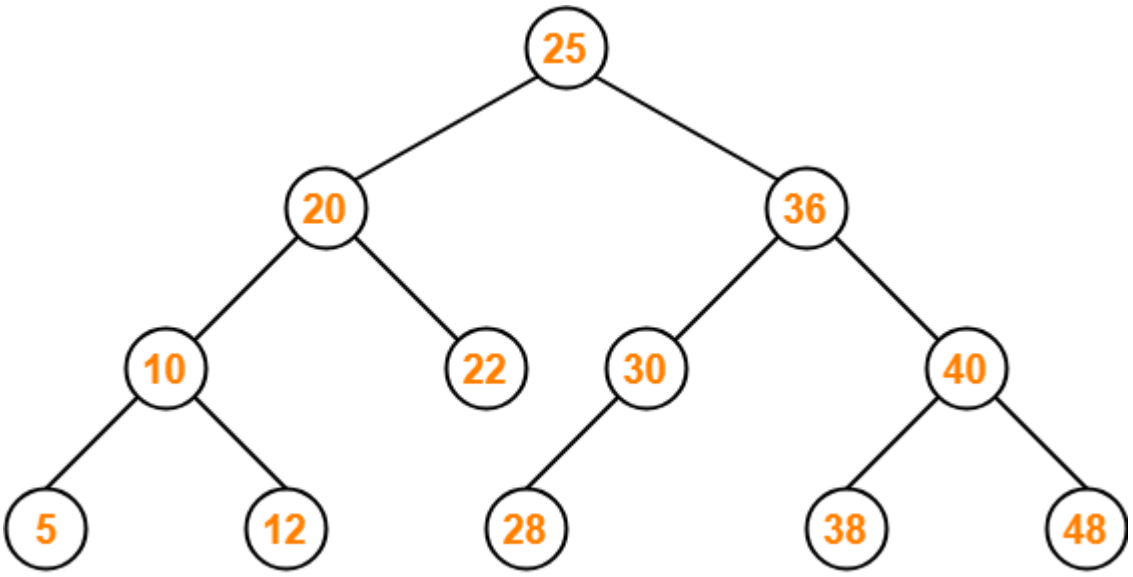



이 차이가 1 이하로 유지되도록 트리를 구성하면, 그 트리는 균형 이진 트리가 됩니다.

[참고] 레드-블랙 트리(Red-Black Tree)는 균형 이진 트리의 한 예로, 자가 균형을 유지하기 위한 특정한 규칙들을 갖춘 이진 탐색 트리입니다. 이 규칙들 덕분에 레드-블랙 트리는 최악의 경우에도 효율적인 연산을 보장하며, C++의 std::map과 std::set 같은 데이터 구조의 내부 구현에 사용됩니다.

## 이진탐색트리(BST, Binary Search Tree)

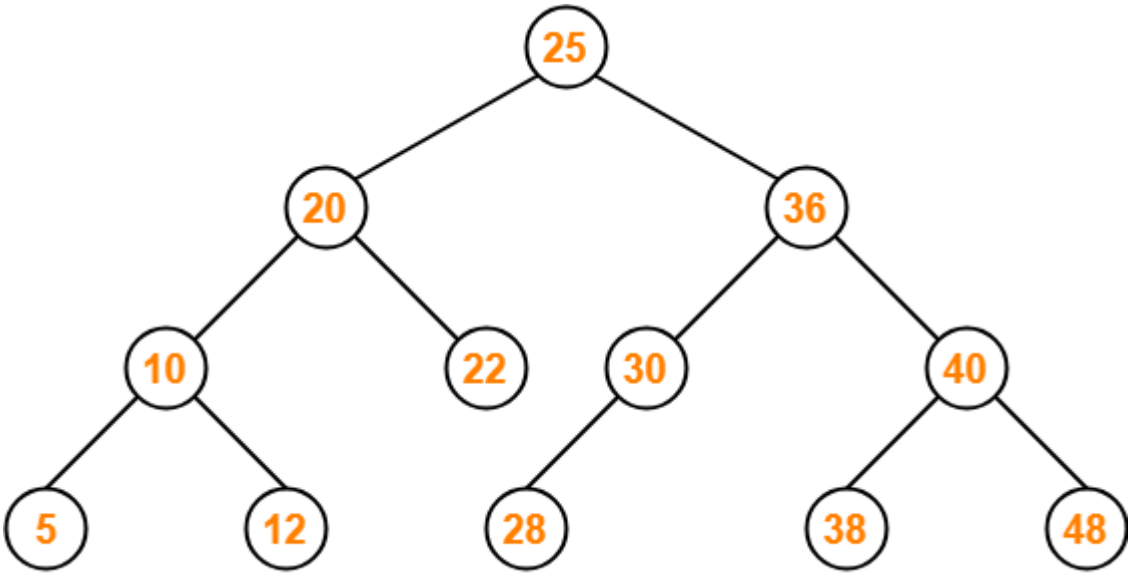
다음 그림은 이진탐색트리(BST) 입니다. 이진트리의 일종으로 노드의 오른쪽 하위 트리에는 "노드의 값보다 큰 값"이 있는 노드만 포함되고 왼쪽 하위트리에는 "노드의 값보다 작은값"이 들어있는 트리를 말합니다.



이 때 왼쪽 및 오른쪽 하위 트리도 해당 특성을 가집니다. 이렇게 두면 "검색"을 하기에 용이합니다. 왼쪽에는 작은 값, 오른쪽에는 큰 값이 이미 정해져있기 때문에 예를 들어 10을 찾으려고 한다면 25의 왼쪽 노드들만 찾으면 된다는 것은 자명하기 때문입니다. 이렇게 한다면 전체탐색을 하지 않아도 되겠죠?

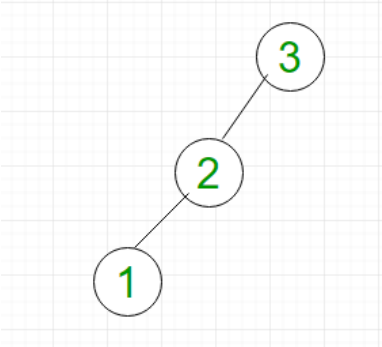
## Q. 이진탐색트리의 시간복잡도는 얼마일까요?

만약 이렇게 균형잡히게 분포가 되었다면 탐색, 삽입, 삭제, 수정 모두 O(logN)입니다.

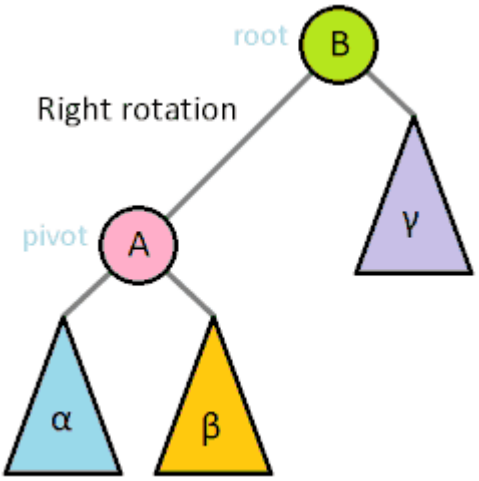



그러나. 이는 삽입 순서에 따라 달라집니다. 예를 들어 1, 2, 3 이렇게 삽입이 되어 이진탐색트리가 완성이 되었다면. 이렇게 "선형적"인 자료구조가 완성되어버립니다.

참고로 선형적이란 직선, 비선형적은 직선이 아닌 모양을 의미합니다.

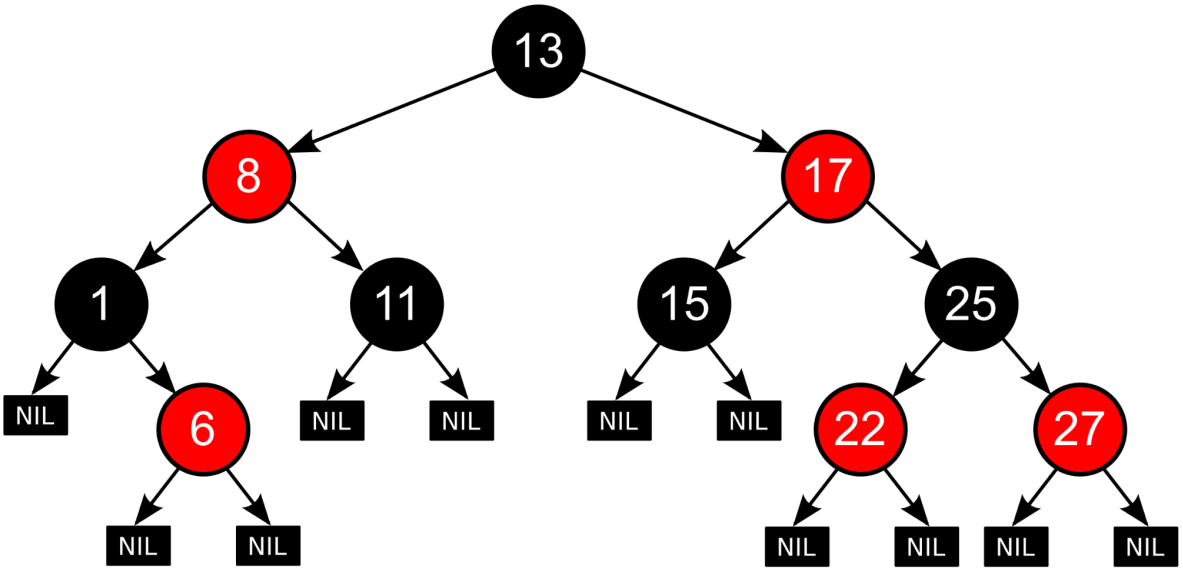



즉, 이진탐색트리는 삽입순서에 따라 영향을 받습니다. 그러나 삽입순서가 어떻게 되든 트리의 노드들을 회전시키는 등의 방법을 통해서 "균형잡히게 만든" 이진탐색트리에서 발전된 트리로는 AVL트리, 레드블랙트리가 있습니다.




[AVL 트리]

map이라는 자료구조는 삽입, 탐색, 삭제, 수정의 시간복잡도가 모두 O(logN)임을 보장받는데 그 이유가 균형잡힌 트리인 레드블랙트리를 기반으로 구현되어있기 때문입니다.




[레드블랙트리]

## 그래프 구현과 탐색 1. 인접행렬

자, 지금까지 그래프에 대해서 얘기를 해봤는데 어떻게 이를 컴퓨터에게 알려줄까요?

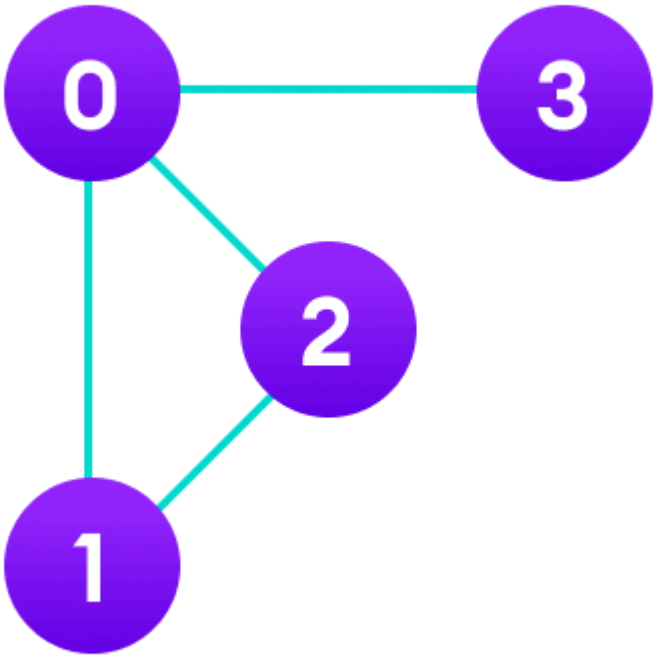
컴퓨터에게 내가 이러한 그래프를 그렸다고 알려줄 표현방법으로는 인접행렬과 인접리스트가 있습니다.

[참고] 인접해있다 = 연결되어있다. 라고 보면 됩니다.

먼저 인접행렬에 대해 설명해보겠습니다.

인접행렬이란 그래프에서 정점과 간선의 관계를 나타내는 bool 타입의 정사각형 행렬을 의미합니다. 정사각형 행렬의 각 요소가 0 또는 1이라는 값으로 가짐을 의미하는데요. 0은 두 정점 사이의 경로가 없음을 의미하며 1은 두 정점 사이의 경로가 있음을 의미합니다.

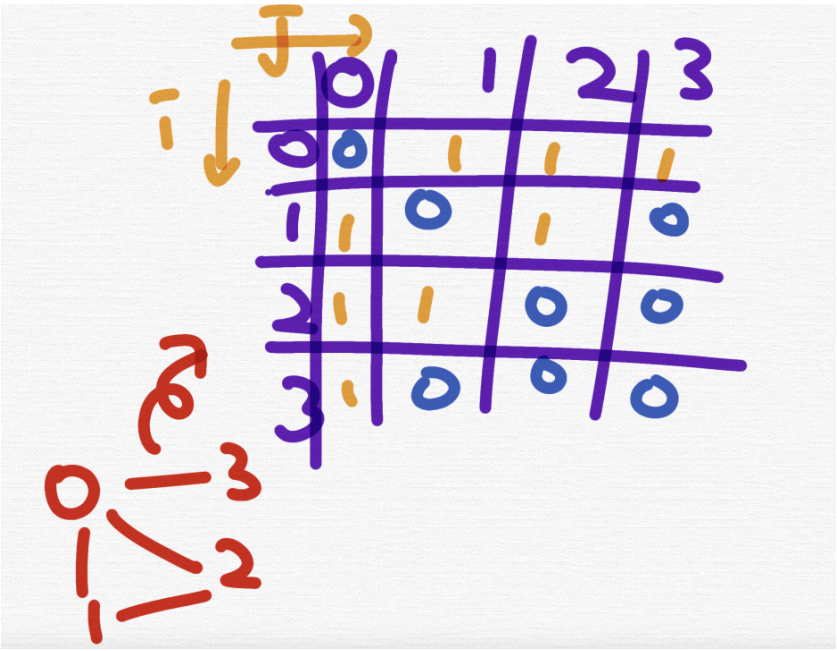
예를 들어 아래와 같은 그래프가 있습니다. 이 그래프는 무방향 그래프로 설명되어있는데 무방향 그래프 := 양방향 그래프로 봐도 됩니다.




이를 다음과 같은 행렬로 표현하는 것을 말합니다.

1 - 1, 2 - 2를 보면 0으로 되어있는 것을 볼 수 있는데 자기자신을 나타내는 것이며 해당 정점의 사이클이 없을 때는 0, 사이클이 있을 때는 1로 표기를 합니다.

0 - 1 이 연결되어있기 때문에 a[0][1] = 1이 됩니다. 그러나 1 - 3은 연결되어있지 않기 때문에 a[1][3] = 0이 됩니다.



이걸 코드로 표현하면 다음과 같습니다.

```
bool a[4][4] = {
    {0, 1, 1, 1},
    {1, 0, 1, 0},
    {1, 1, 0, 0},
    {1, 0, 0, 0},
};
```

그렇다면 이를 기반으로 어떻게 코드를 구축할까요?

보통은 다음코드처럼 2중 for문을 통해 i에서 j로 가는 경로가 있다면 어떠한 로직 또는 해당 정점으로 부터 탐색하는 로직을 구축합니다. 참고로 BFS와 DFS는 하나의 예에 불과합니다. 이것들 말고도 이 정점으로부터 탐색하는 다양한 알고리즘을 쓸 수 있습니다.

```
bool a[V][V];
for(int i = 0;i < V; i++){
    for(int j = 0; j < V; j++){
        if(a[i][j]){
            //출력하는 로직
            cout << i << "부터 " << j << "까지 경로가 있습니다.\n";
            // 해당 정점으로 부터 탐색하는 로직
            BFS(i);
            DFS(i);
        }
    }
}
```

자 그러면 문제를 풀면서 인접행렬을 더 알아가보도록 하겠습니다.

## 문제로 알아가는 인접행렬

**Q1. 3번노드에서 5번노드로 가는 단방향 경로가 있고 이를 인접행렬로 표현한다면 어떻게 될까요?**

답 :

```
a[3][5] = 1;
```

**Q2. 3번노드에서 5번노드로 가는 양방향 경로가 있고 이를 인접행렬로 표현한다면 어떻게 될까요?**

답 :

```
a[3][5] = 1;
a[5][3] = 1;
```

Q3. 정점의 갯수가 20개인 그래프가 있습니다. 이를 인접행렬로 표현한다고 하고 메모리를 최소로 쓴다고 했을 때 배열을 어떻게 만들어야 할까요?

답 :

```
bool a[20][20];
```

Q4. 노드 1, 2, 3이 있으며, 1번 노드에서 2번 노드로 가는 단방향 경로와 2번 노드에서 3번 노드로 가는 단방향 경로가 있습니다. 이를 인접행렬로 표현한다면 어떻게 될까요?

답:

```
   1  2  3
1 [0, 1, 0]
2 [0, 0, 1]
3 [0, 0, 0]
```

Q5. 1 ~ 4번노드로 이루어진 4개의 노드가 있습니다. 1번 노드에서 2번 노드로 가는 경로, 2번 노드에서 3번 노드로 가는 경로, 3번 노드에서 4번 노드로 가는 경로, 그리고 4번 노드에서 1번 노드로 가는 경로가 있을 때, 인접행렬은 어떻게 나타날까요?

답:

```
   1  2  3  4
1 [0, 1, 0, 0]
2 [0, 0, 1, 0]
3 [0, 0, 0, 1]
4 [1, 0, 0, 0]
```

Q6. 1 ~ 5번노드로 이루어진 5개의 노드가 있습니다. 모든 노드가 서로 연결되어 있는 완전 그래프의 인접행렬은 어떻게 될까요?

답:

```
   1  2  3  4  5
1 [0, 1, 1, 1, 1]
2 [1, 0, 1, 1, 1]
3 [1, 1, 0, 1, 1]
4 [1, 1, 1, 0, 1]
5 [1, 1, 1, 1, 0]
```

[참고] 완전 그래프에서는 자기 자신을 가리키는 경로가 없으므로 대각선은 0이고, 나머지 모든 요소는 1입니다.

Q7. 노드 1, 2, 3이 있으며, 1번 노드에서 3번 노드로 가는 단방향 경로가 있고, 3번 노드에서 2번 노드로 가는 단방향 경로가 있습니다. 또한, 2번 노드에서 다시 1번 노드로 가는 경로가 있습니다. 이 경우 인접행렬은 어떻게 될까요?

답:

```
   1  2  3
1 [0, 0, 1]
2 [1, 0, 0]
3 [0, 1, 0]
```

## 인접행렬을 기반으로 탐색하기

1번.

정점은 0번 부터 9번까지 10개의 노드가 있다. 1 - 2 / 1 - 3 / 3 - 4 라는 경로가 있다. (1번과 2번, 1번과 3번, 3번과 4번은 연결되어있다.) 이를 인접행렬로 표현하라.

2번.

0번부터 방문안한 노드를 찾고 해당 노드부터 방문, 연결된 노드를 이어서 방문해서 출력하는 재귀함수를 만들어라. 여기서 정점을 방문하고 다시 방문하지 않게 만들어야 한다.

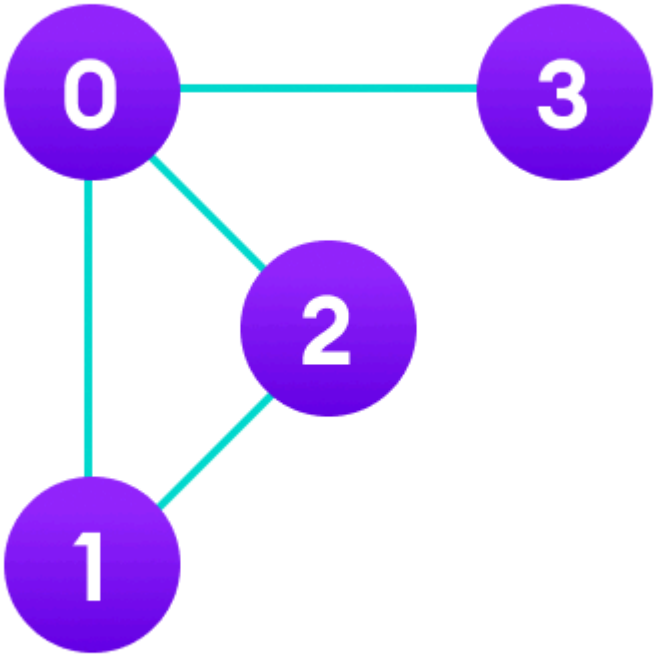
정답코드

```
#include<bits/stdc++.h>
using namespace std;
const int V = 10;
bool a[V][V], visited[V];
void go(int from){
	visited[from] = 1;
	cout << from << '\n';
	for(int i = 0; i < V; i++){
		if(visited[i]) continue;
		if(a[from][i]){
			go(i);
		}
	}
	return;
}
int main(){
	a[1][2] = 1; a[1][3] = 1; a[3][4] = 1;
	a[2][1] = 1; a[3][1] = 1; a[4][3] = 1;
	for(int i = 0;i < V; i++){
		for(int j = 0; j < V; j++){
			if(a[i][j] && visited[i] == 0){
				go(i);
```

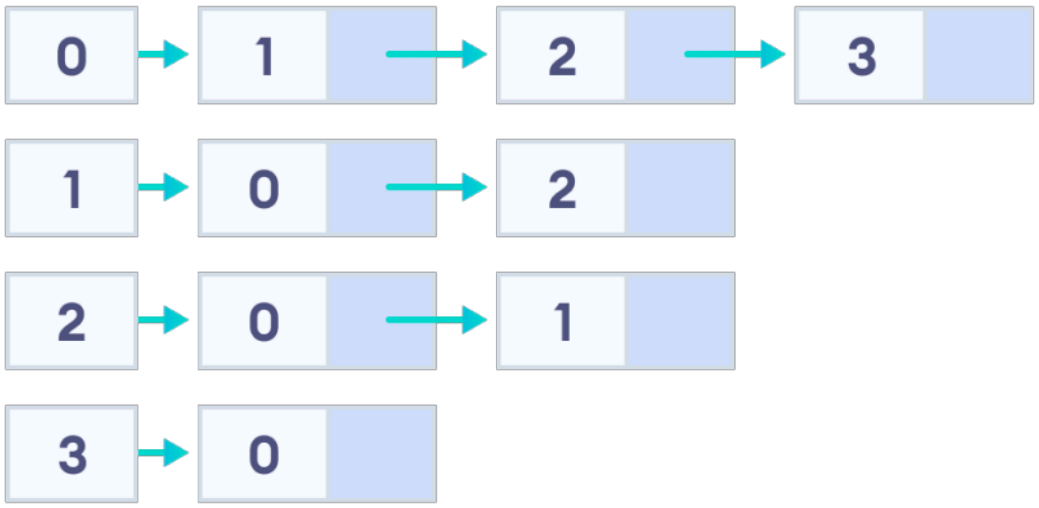
```
                    }
                }
            }
}
```

## 그래프 구현과 탐색 2. 인접리스트

인접 리스트(adjacency list)는 그래프에서 정점과 간선의 관계를 나타내는 연결리스트를 의미합니다.
예를 들어 아래와 같은 그래프가 있습니다.




이 그래프를 아래와 같이 나타낼 수 있습니다.




이를 코드로 나타내면 다음과 같습니다.

```
#include<bits/stdc++.h>
using namespace std;
const int V = 4;
vector<int> adj[V];
int main(){
    adj[0].push_back(1);
    adj[0].push_back(2);
    adj[0].push_back(3);

    adj[1].push_back(0);
    adj[1].push_back(2);

    adj[2].push_back(0);
    adj[2].push_back(1);

    adj[3].push_back(0);

    for(int i = 0; i < 4; i++){
        cout << i << " :: ";
        for(int there : adj[i]){
            cout << there << " ";
        }
        cout << '\n';
    }
    // 이렇게도 할 수 있다.
```

```
    for(int i = 0; i < 4; i++){
        cout << i << " :: ";
        for(int j = 0; j < adj[i].size(); j++){
            cout << adj[i][j] << " ";
        }
        cout << '\n';
    }

}
/*
0 :: 1 2 3
1 :: 0 2
2 :: 0 1
3 :: 0
*/
```

## Q. 왜 list가 아닌 vector로 구현할까?

앞의 코드는 vector로 구현했는데 인접리스트인 list로 구현해도 되고 vector로 구현해도 됩니다.

```
// vector
vector<int> adj[1004];
// list
list<int> adj[1004];
```

그러나 보통 vector를 주로 많이 씁니다. 이는 list 와 vector의 연산 시간복잡도 때문입니다.

### list의 시간복잡도

- n번째 인덱스에 삽입, 삭제 : O(1)

- 마지막 요소에 삽입, 삭제 : O(1)

- 특정요소 탐색 : O(n)

- n번째 요소 참조 : O(n)

### vector의 시간복잡도

- n번째 인덱스에 삽입, 삭제 : O(n)

- 마지막 요소에 삽입, 삭제 : O(1)

- 특정요소 탐색 : O(n)

- n번째 요소 참조 : O(1)

인접리스트를 기반으로 로직을 구현할 때 많이 사용되는 연산은 마지막 요소에 삽입과 해당 배열의 n번째 요소를 탐색하는 연산입니다. 그렇기 때문에 vector로 구현하는게 좋습니다.

## Q. 인접리스트를 기반으로 탐색하기

1번과 2번을 만족시키는 코드를 만들어내야 합니다.

1번.

정점은 0번 부터 9번까지 10개의 노드가 있다. 1 - 2 / 1 - 3 / 3 - 4 라는 경로가 있다. (1번과 2번, 1번과 3번, 3번과 4번은 연결되어있다.)이를 인접리스트로 표현한자?

2번.

0번부터 방문안한 노드를 찾고 해당 노드부터 방문, 연결된 노드를 이어서 방문해서 출력하는 재귀함수를 만들고 싶다면 어떻게 해야할까? 또한, 정점을 방문하고 다시 방문하지 않게 만드려면 어떻게 해야할까?

정답코드

```
#include<bits/stdc++.h>
using namespace std;
const int V = 10;
vector<int> adj[V];
int visited[V];
void go(int idx){
    cout << idx << '\n';
    visited[idx] = 1;
    for(int there : adj[idx]){
        if(visited[there]) continue;
        go(there);
    }
    return;
}
int main(){
    adj[1].push_back(2);
```

```
    adj[2].push_back(1);

    adj[1].push_back(3);
    adj[3].push_back(1);

    adj[3].push_back(4);
    adj[4].push_back(3);
    for(int i = 0; i < V; i++){
        if(adj[i].size() && visited[i] == 0) go(i);
    }
}
/*
1
2
3
4
*/
```

## 인접행렬과 인접리스트의 차이

자아 지금까지 인접행렬과 인접리스트를 배워봤습니다. 둘 중에 어떤 것을 써야 할까요?

일단 차이를 배워보도록 하겠습니다.

### 공간복잡도

- 인접행렬 : O(V^2)

- 인접리스트 : O(V + E)

```
// 인접행렬
bool adj[V][V];
// 인접리스트
vector<int> adj[V];
```

### 시간복잡도 : 간선 한개 찾기

- 인접행렬 : O(1)

- 인접리스트 : O(V)

```
// 인접행렬
```

```
for(int i = 0;i < V; i++){
    for(int j = 0; j < V; j++){
        if(a[i][j]){
        }
    }
}
// 인접리스트
for(int j = 0; j < adj[i].size(); j++){
    cout << adj[i][j] << " ";
}
```

## 시간복잡도 : 모든 간선찾기

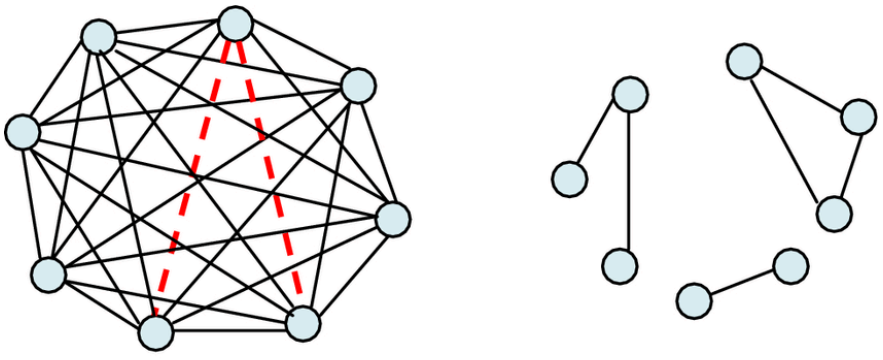
- 인접행렬 : O(V^2)

- 인접리스트 : O(V + E)

## 그래프가 희소할 때는 인접리스트, 조밀할 때는 인접행렬을 써라.

그래프가 희소할 때는 인접리스트, 조밀 할 때는 인접행렬이 좋습니다. 그래프가 희소할 때(sparse)할 때는 인접행렬이 인접리스트보다 메모리를 더 많이 써야 합니다. 간선이 없어서 인접행렬의 대부분의 요소가 0인데도 불구하고 해당 부분을 포함해 2차원 배열을 만들어야 되기 때문입니다.

그래프가 조밀할 때 (dense)할 때는 인접행렬이 인접 리스트보다 더 좋습니다. 어차피 다 연결되어있기 때문에 메모리적 효율성은 동일해지고 정점i에서 정점j까지의 간선이 있는 확인하는 속도가 더 빠르기 때문에 인접행렬이 더 빠릅니다.



(a) (b)

[a : dense / b : sparse]

### 정리

보통은 문제에서 sparse한 그래프가 많이 나오기 때문에 인접리스트를 쓰면 됩니다. 다만 문제 또는 코딩인터뷰에서 인접행렬로 주어진다면 그대로 인접행렬로 푸는 것이 좋습니다.

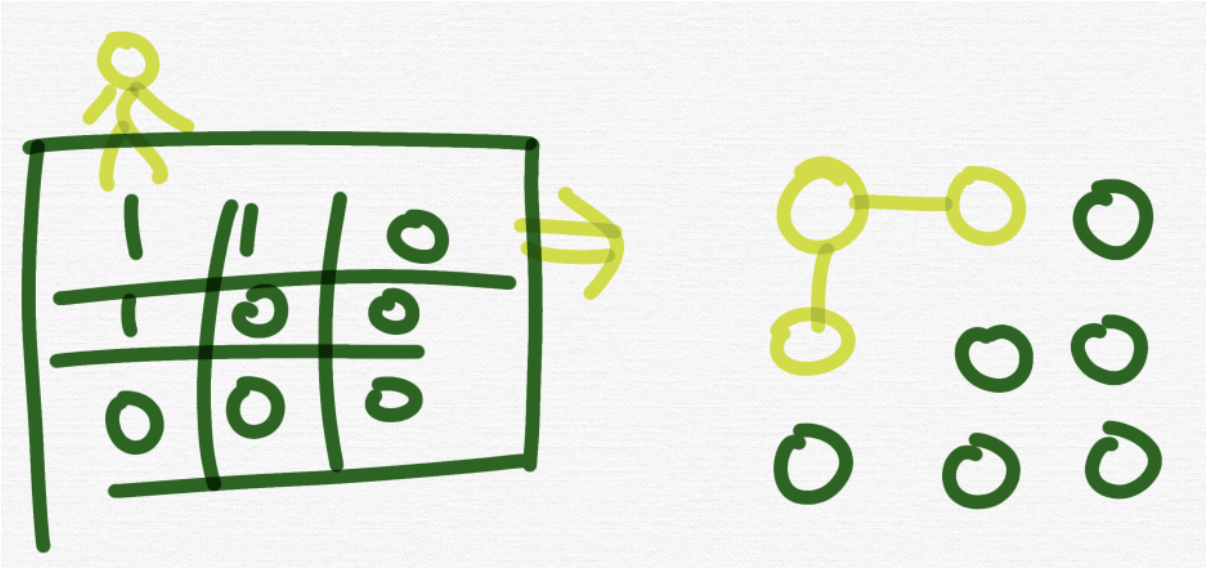
## 그래프 구현과 탐색 3. 맵과 방향벡터

코딩인터뷰나 알고리즘 문제를 보면 지금까지 배웠던 인접행렬이나 인접리스트로 문제가 주어지는 경우도 있지만 맵으로도 주어지는 경우가 있습니다.

N×M크기의 배열로 표현되는 미로가 있다.

| | | | | | |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 | 1 |

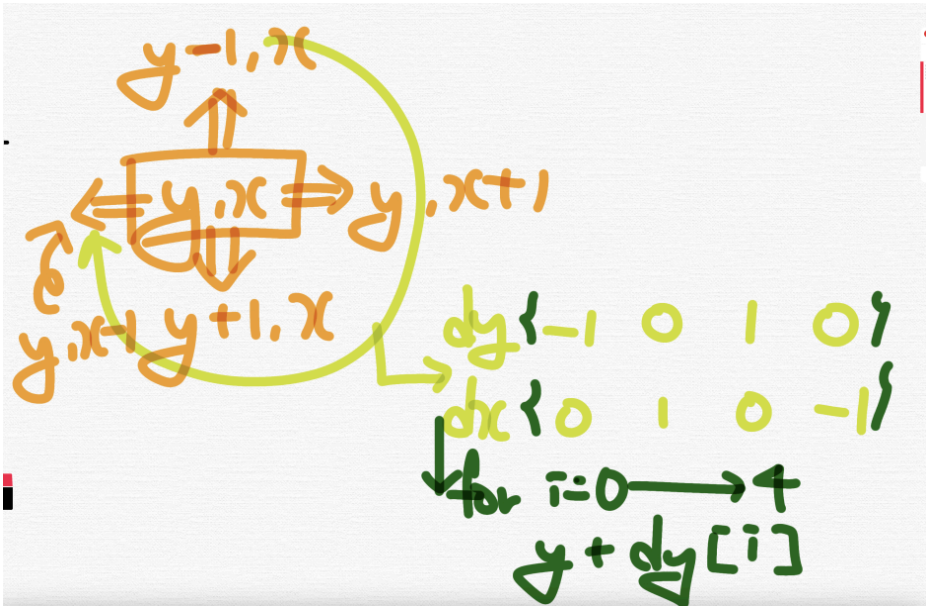
이런 경우에 문제에서 주어진 맵을 받아서 그 맵을 기준으로 탐색을 이어가야 합니다. 또한, 맵은 하나의 그래프입니다. 예를 들어 3 * 3 맵이며 1은 갈 수 있는 지역, 0은 갈 수 없는 지역이며 4방향으로 탐색이 가능하다고 하면 다음과 같은 그래프가 되는 셈입니다.



**주의사항 : 이 맵은 인접행렬이 절대 아닙니다!!**

## 4방향탐색과 방향벡터

보통 이 맵을 기준으로 4가지 방향으로 탐색해나아가라고 합니다. 4가지 방향은 보통 위, 아래, 오른쪽, 왼쪽으로 주어집니다.(문제마다 다를 수는 있지만 보통은 이렇게 주어집니다.)예를 들어 어떠한 y, x가 주어졌을 때 y, x를 중심으로 상하좌우 4가지 방향으로 탐색은 어떻게 할까요?




다음과 같이 위쪽부터 시계방향으로 탐색한다고 가정하면 -1, 0 / 0, 1 / 1, 0 / 0, -1 을 더해가며 탐색하게 됩니다.

이를 배열로 나타내면 이런 방향벡터를 정의할 수 있고 이를 기반으로 탐색한다면 이러한 코드가 작성되겠죠? 보통 방향벡터는 dy, dx라는 이름으로 정의합니다. direction_y, direction_x라는 의미입니다.

```
const int dy[] = {-1, 0, 1, 0};
const int dx[] = {0, 1, 0, -1};
for(int i = 0; i < 4; i++){
    ny = y + dy[i];
    nx = x + dx[i];
}
```

자 그러면 문제를 풀면서 이해해보겠습니다.

## Q1. {0, 0}좌표에서 dy, dx를 만들어 4방향(위, 오른쪽, 아래, 왼쪽)을 탐색하며 좌표를 출력하시오.

```
#include <bits/stdc++.h>
using namespace std;
const int dy[] = {-1, 0, 1, 0};
const int dx[] = {0, 1, 0, -1};
int main(){
    int y = 0, x = 0;
    for(int i = 0; i < 4; i++){
        int ny = y + dy[i];
        int nx = x + dx[i];
        cout << ny << " : " << nx << '\n';
    }
    return 0;
}
/*
-1 : 0
0 : 1
1 : 0
0 : -1
*/
```

## Q2. {0, 0}좌표에서 dy, dx를 만들어 8방향(위, 오른쪽, 아래, 왼쪽 및 대각선방향포함)을 탐색하며 좌표를 출력하시오.

만약에 8가지 방향이라면 어떻게 해야할까요? 예를 들어 위쪽, 위쪽오른쪽 .. 등으로 대각선까지 포함해서 문제가 주어진다면 어떻게 해야할까요?

```cpp
#include <bits/stdc++.h>
using namespace std;
const int dy[] = {-1, -1, 0, 1, 1, 1, 0, -1};
const int dx[] = {0, 1, 1, 1, 0, -1, -1, -1};
int main(){
    int y = 0, x = 0;
    for(int i = 0; i < 8; i++){
        int ny = y + dy[i];
        int nx = x + dx[i];
        cout << ny << " : " << nx << '\n';
    }
    return 0;
}
/*
-1 : 0
-1 : 1
0 : 1
1 : 1
1 : 0
1 : -1
0 : -1
-1 : -1
*/
```

반드시 4방향이 아닐 수 있습니다. 문제에 따라 방향은 달라질 수 있습니다. 이를 주의해서 코드를 구축해야 합니다.

## Q3. 3 * 3 맵을 입력받아야 함. 이 맵은 1과 0으로 이루어져있고 {0, 0}은 무조건 1임을 보장한다. {0, 0}부터 4방향을 기준으로 한칸씩 탐색해나가며 방문한 정점은 다시 방문하지 않으며 방문하는 좌표를 출력하는 코드. 0은 갈 수 없는 지역. 1은 갈 수 있는 지역을 구현하시오.

```cpp
#include<bits/stdc++.h>
using namespace std;
const int n = 3;
int a[n][n], visited[n][n];
const int dy[] = {-1, 0, 1, 0};
const int dx[] = {0, 1, 0, -1};

void go(int y, int x){
    visited[y][x] = 1;
    cout << y << " : " << x << "\n";
    for(int i = 0; i < 4; i++){
        int ny = y + dy[i];
```

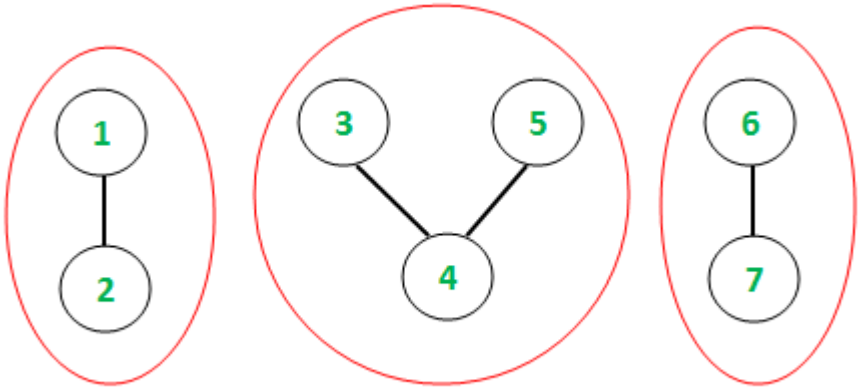
```
        int nx = x + dx[i];
        if(ny < 0 || ny >= n || nx < 0 || nx >= n) continue;
        if(a[ny][nx] == 0) continue;
        if(visited[ny][nx]) continue;
        go(ny, nx);
    }
    return;
}
int main(){
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            cin >> a[i][j];
        }
    }
    go(0, 0);
}
/*
1 0 1
1 0 1
0 1 1
*/
```

우리가 맵을 탐색할 때 다음코드가 무조건 들어가야 합니다. 맵의 주어진 범위를 벗어나지 않게 하는 코드입니다. 이 문제에서 맵의 범위는 0 ~ 3인데 그 밖을 벗어나면 배열 인덱싱에러가 나타나겠죠? 언더플로우와 오버플로우 에러를 이렇게 방지해줍시다. 맵을 탐색할 때는 이렇게 항상 맵을 벗어나는지를 염두해야 합니다.

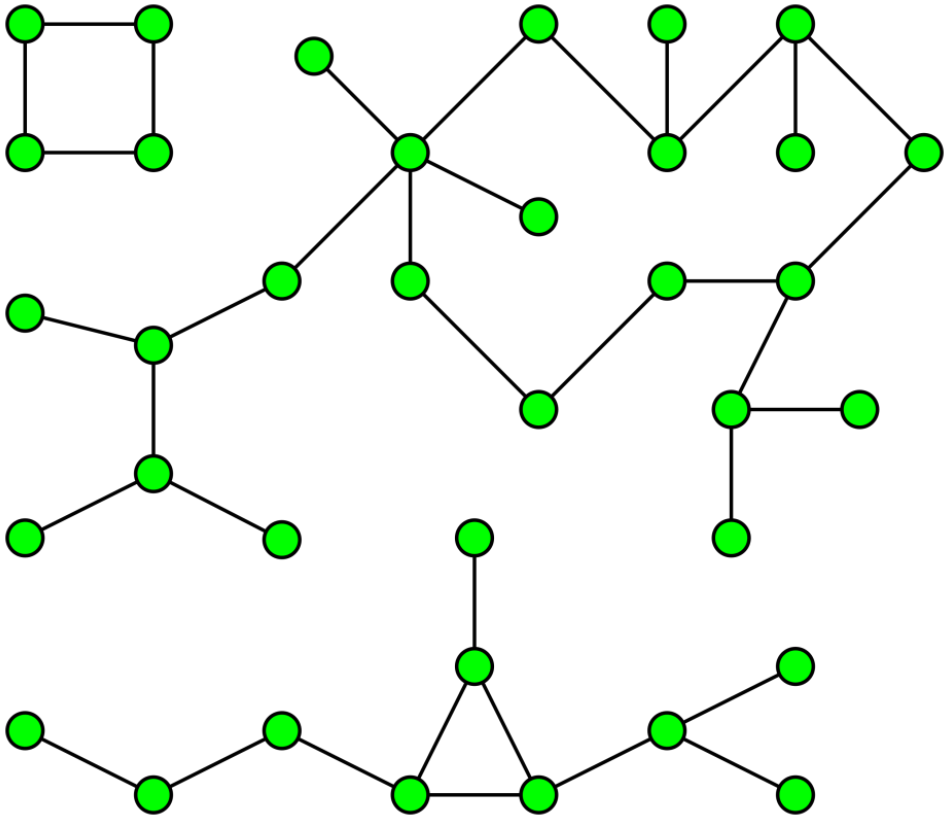
```
if(ny < 0 || ny >= n || nx < 0 || nx >= n) continue;
```

## 연결된 컴포넌트(connected component)

연결된 컴포넌트(connected component)는 연결된 하위그래프를 말하며 연결된 하나의 덩어리라고 보시면 됩니다. 이 덩어리는 연결된 컴포넌트에 속한 "모든 정점을 연결하는 경로가 있다."라는 특징을 가집니다.




앞의 그림에서 연결된 컴포넌트의 수는 총 3개이고 각각의 컴포넌트는 2개, 3개, 2개라는 정점을 가집니다.



앞의 그림에서 connected component는 3개가 됩니다. 즉, 연결되어있는지 아니면 연결되어있지 않는지를 토대로 연결된 컴포넌트로 나눕니다. 이러한 컴포넌트들을 번호를 붙여가며 색칠하는 알고리즘은 풀르드필(floodfill)이라고 합니다.

# 깊이우선탐색

깊이우선탐색, DFS(Depth First Search)는 그래프를 탐색할 때 쓰는 알고리즘이며 어떤 노드부터 시작해 인접한 노드들을 재귀적으로 방문하며 방문한 정점은 다시 방문하지 않으며 각 분기마다 가능한 가장 멀리 있는 노드까지 탐색하는 알고리즘입니다.

## DFS의 수도코드

DFS의 수도코드는 다음과 같습니다. 이 수도코드만 이해한다면. DFS 전체를 이해한거나 다름없습니다.

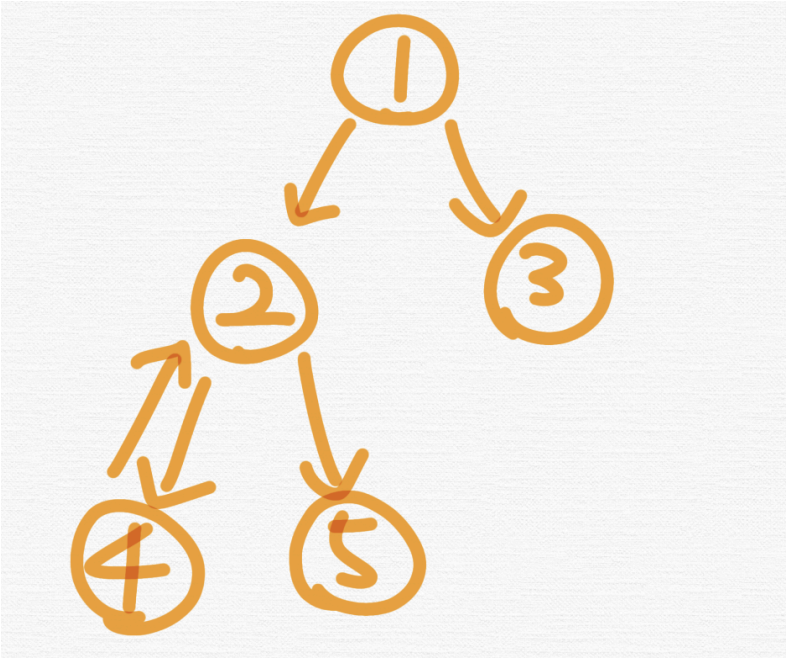
수도코드(pseudocode)는 프로그램의 로직을 표현하기 위해 쓰이는 코드입니다. 주로 어떤 알고리즘이 어떠한 로직을 갖고 있는지 나타내기 위해 씁니다.

```
DFS(u, adj)
    u.visited = true
    for each v ∈ adj[u]
        if v.visited == false
            DFS(v, adj)
```

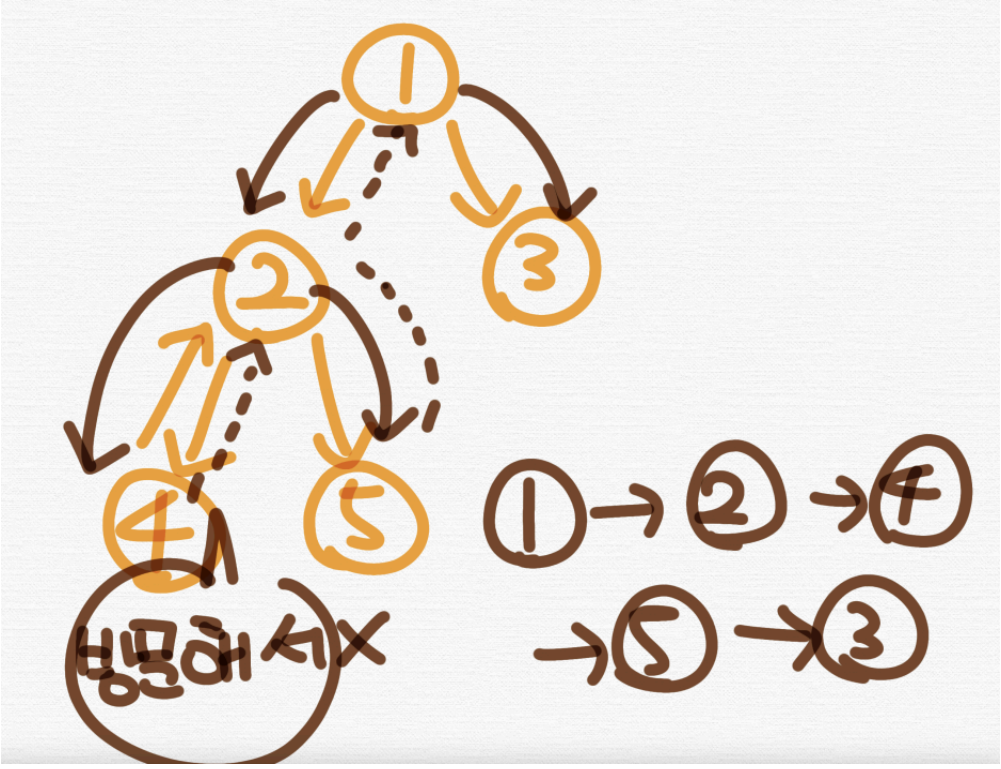
앞의 코드를 해석하면 다음과 같습니다.

1. 어떠한 정점 u의 visited를 참으로 바꾸고 u로부터 연결되어있는 v지점을 탐색합니다.
2. 이 때 방문되어있지 않은 노드에 대해 재귀적으로 DFS를 호출합니다.

자 다음 그림의 그래프를 DFS로 탐색해본다면 어떻게 될까요?




이런 꼴이 될 겁니다. 일단 계속해서 재귀적으로 1, 2, 4번으로 가는 것을 볼 수 있습니다. 또한, 4번에서 다시 2번으로 가려고 했지만 이미 방문한 노드이기 때문에 다시 방문하지 못하고 5번으로 가는 것을 볼 수 있습니다.



이를 구현한 코드입니다.

```
#include<bits/stdc++.h>
using namespace std;
const int n = 6;
vector<int> adj[n];
int visited[n];
void DFS(int u){
    visited[u] = 1;
    cout << u << "\n";
    for(int v : adj[u]){
        if(visited[v] == 0){
            DFS(v);
        }
    }
    cout << u << "로부터 시작된 함수가 종료되었습니다.\n";
    return;
}
int main(){
    adj[1].push_back(2);
    adj[1].push_back(3);
    adj[2].push_back(4);
    adj[4].push_back(2);
    adj[2].push_back(5);
    DFS(1);
}
/*
1
2
4
4로부터 시작된 함수가 종료되었습니다.
5
5로부터 시작된 함수가 종료되었습니다.
2로부터 시작된 함수가 종료되었습니다.
3
3로부터 시작된 함수가 종료되었습니다.
1로부터 시작된 함수가 종료되었습니다.
*/
```

이러한 DFS를 구현하는 방법은 크게 2가지가 있습니다.

## 구현방법1 : 돌다리를 두들겨 보다.

```
void DFS(int here){
    visited[here] = 1;
    for(int there : adj[here]){
        if(visited[there]) continue;
        DFS(there);
    }
}
```

만약 방문이 되어있다면(visited[there])이라면 방문을 하지 않고 방문이 안되어있다면 방문을 하는 코드입니다.

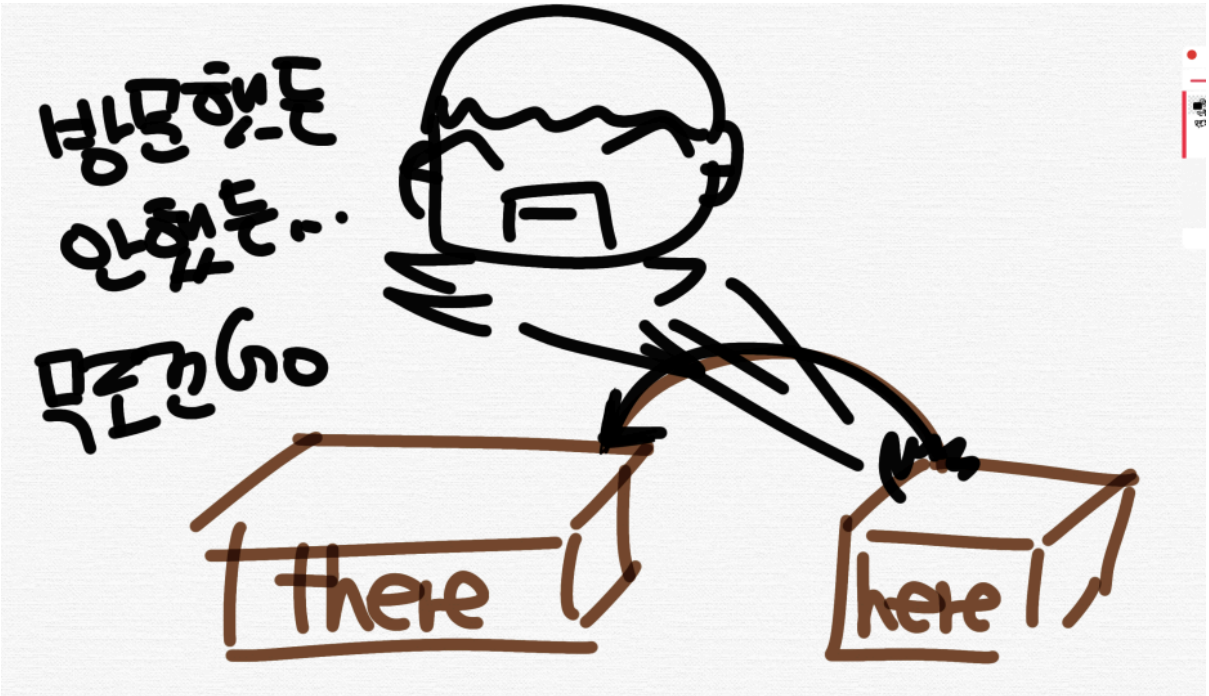
앞의 코드는 다음과 같이 구현해도 됩니다.

```
void DFS(int here){
    for(int there : adj[here]){
        if(visited[there]) continue;
        visited[there] = 1;
        DFS(there);
    }
}
```

다만 이럴 경우 시작지점에 대해 방문처리를 해주어야 합니다. 예를 들어 1, 2, 3, 4 지점을 탐색한다고 하고 1지점부터 탐색을 이어나간다고 했을 때 다음과 같이 코드를 구축해야 합니다.

```
visited[1] = 1;
DFS(1);
```

## 구현방법2 : 못 먹어도 GO




일단 방문되어있던 안되어있던 무조건 DFS를 호출하고 해당 함수에서 만약에 방문되어있다면 return 해 함수를 종료시키는 방법입니다.

```
void DFS(int here){
    if(visited[here]) return;
    visited[here] = 1;
    for(int there : adj[here]){
        DFS(there);
    }
}
```

문제에 따라 1번방법이 편할 때도 2번방법이 편할 때도 있습니다. 두 개의 코드 모두 자유롭게 구현할 줄 알아야 합니다.

## Q. 종화는 방구쟁이야!

종화는 21세기 유명한 방구쟁이다. 종화는 방구를 낄 때 "이러다... 다 죽어!!" 를 외치며 방구를 뀌는데 이렇게 방귀를 뀌었을 때 방귀는 상하좌우 네방향으로 뻗어나가며 종화와 연결된 "육지"는 모두 다 오염된다. "바다"로는 방구가 갈 수 없다. 맵이 주어졌을 때 종화가 "이러다... 다 죽어!!"를 "최소한" 몇 번외쳐야 모든 육지를 오염시킬 수 있는지 말해보자. 1은 육지며 0은 바다를 가리킨다.

**입력**

맵의 세로길이 N과 가로길이 M 이 주어지고 이어서 N * M의 맵이 주어진다.

**출력**

"이러다... 다 죽어!!"를 몇 번 외쳐야하는지 출력하라.

**범위**

1 <= N <= 100

1 <= M <= 100

**예제입력**

5 5

1 0 1 0 1

1 1 0 0 1

0 0 0 1 1

0 0 0 1 1

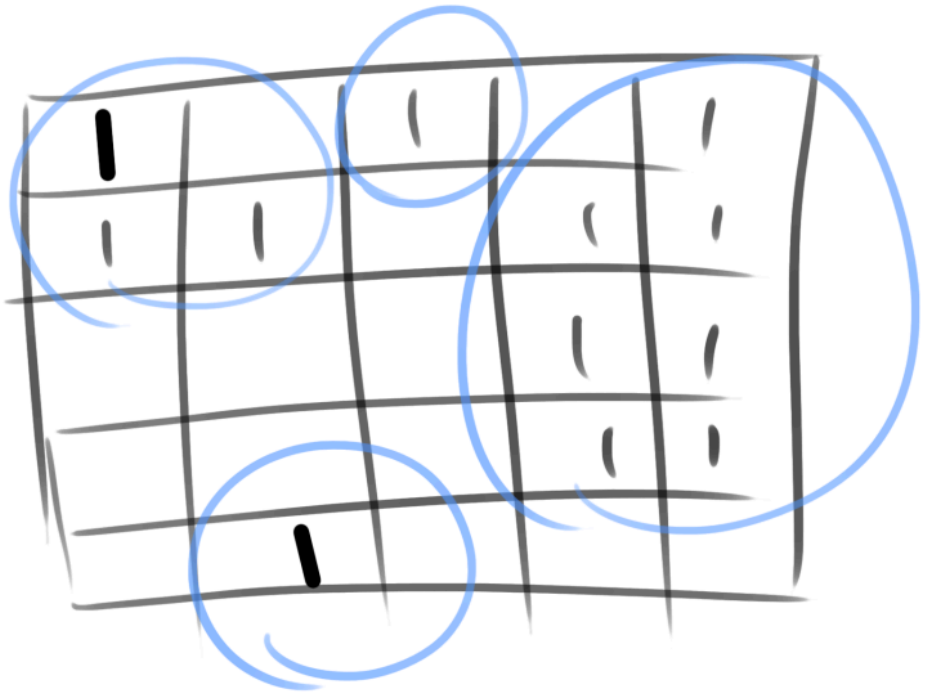
0 1 0 0 0

**예제출력**

4

**문제해설**

이 문제는 어떤 문제일까요? 바로 앞서 설명한 연결된 컴포넌트(connected component)를 찾는 문제입니다.



예제입력을 시각화했을 때 저렇게 묶은 부분이 바로 연결된 컴포넌트이며 이러한 덩어리가 몇개인지를 찾는 문제입니다. 그렇다면 어떻게 찾을 수 있을까요? 간단하게 DFS를 구축하면 됩니다.

**정답코드**

```
#include<bits/stdc++.h>
using namespace std;
int dy[4] = {-1, 0, 1, 0};
int dx[4] = {0, 1, 0, -1};
int m, n, k, y, x, ret, ny, nx, t;
int a[104][104];
bool visited[104][104];
void DFS(int y, int x){
    visited[y][x] = 1;
    for(int i = 0; i < 4; i++){
        ny = y + dy[i];
        nx = x + dx[i];
        if(ny < 0 || nx < 0 || ny >=n || nx >= m) continue;
        if(a[ny][nx] == 1 && !visited[ny][nx]){
            DFS(ny, nx);
        }
    }
    return;
}

int main(){
    cin.tie(NULL);
    cout.tie(NULL);
    cin >> n >> m;
    for(int i = 0; i < n; i++){
      for(int j = 0; j < m; j++){
            cin >> a[i][j];
            }
      }
      for(int i = 0; i < n; i++){
      for(int j = 0; j < m; j++){
            if(a[i][j] == 1 && !visited[i][j]){
                      ret++; DFS(i, j);
                  }
            }
      }
      cout << ret << '\n';
    return 0;
}
```

**상세설명**

```
if(a[i][j] == 1 && !visited[i][j]){
    ret++; DFS(i, j);
}
```

방문을 하지 않았고 육지라면 ret++를 하고 DFS를 호출합니다.

```
void DFS(int y, int x){
    visited[y][x] = 1;
    for(int i = 0; i < 4; i++){
        ny = y + dy[i];
        nx = x + dx[i];
        if(ny < 0 || nx < 0 || ny >=n || nx >= m) continue;
        if(a[ny][nx] == 1 && !visited[ny][nx]){
            DFS(ny, nx);
        }
    }
    return;
}
```

메인이 되는 DFS입니다. 방문했다고 표기하고 4방향을 탐색하면서 DFS를 탐색해나갑니다.

## 디버깅 팁

이 문제를 풀 때 DFS가 퍼져나가는 것이 상상이 안된다면 이렇게 해보시는 것은 어떨까요?

```
#include<bits/stdc++.h>
using namespace std;
int dy[4] = {-1, 0, 1, 0};
int dx[4] = {0, 1, 0, -1};
int m, n, k, y, x, ret, ny, nx, t;
int a[104][104];
bool visited[104][104];
void DFS(int y, int x){
    cout << y << " : " << x << '\n';
    visited[y][x] = 1;
    for(int i = 0; i < 4; i++){
        ny = y + dy[i];
        nx = x + dx[i];
        if(ny < 0 || nx < 0 || ny >=n || nx >= m) continue;
        if(a[ny][nx] == 1 && !visited[ny][nx]){
```

```
            DFS(ny, nx);
        }
    }
    return;
}

int main(){
    cin.tie(NULL);
    cout.tie(NULL);
    cin >> n >> m;
    for(int i = 0; i < n; i++){
      for(int j = 0; j < m; j++){
            cin >> a[i][j];
            }
      }
      for(int i = 0; i < n; i++){
      for(int j = 0; j < m; j++){
            if(a[i][j] == 1 && !visited[i][j]){
                  ret++; DFS(i, j);
                  }
            }
      }
      cout << ret << '\n';
    return 0;
}
```

이 코드는 DFS 함수 내에 아래와 같은 코드를 집어넣었습니다.

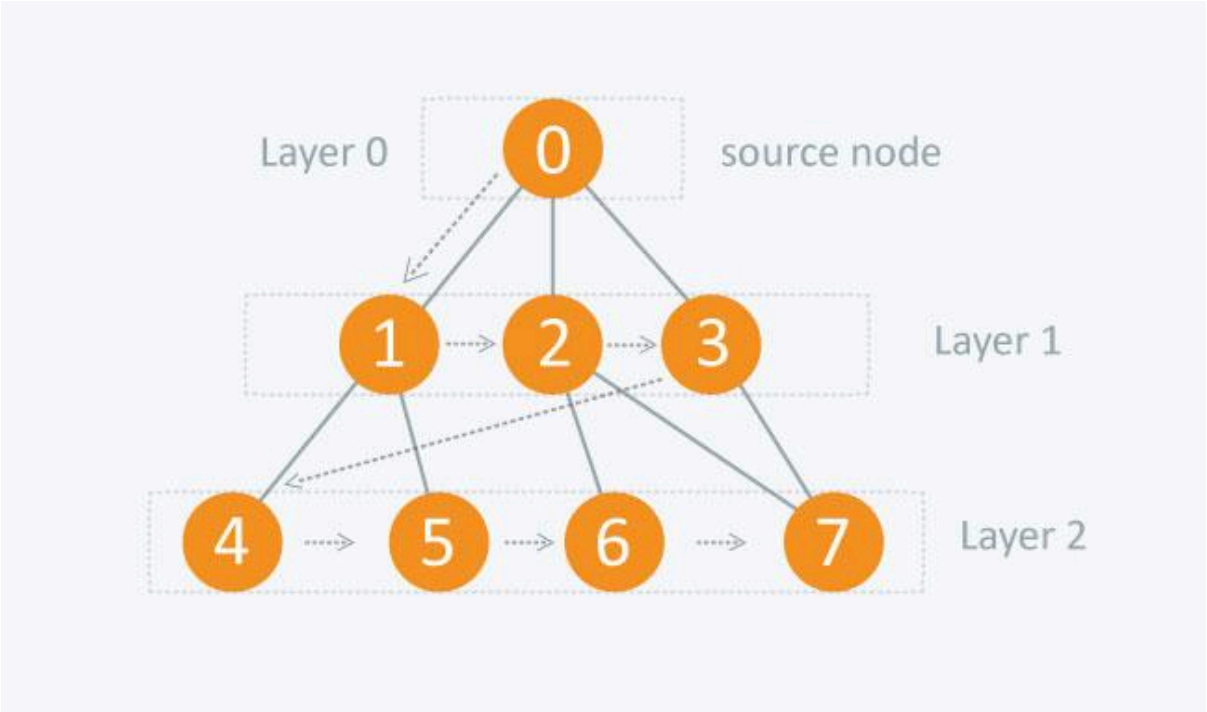
```
cout << y << " : " << x << '\n';
```

이렇게 y, x가 어떻게 뻗어나가는지를 보면서 이해를 하시는 것이 좋습니다.

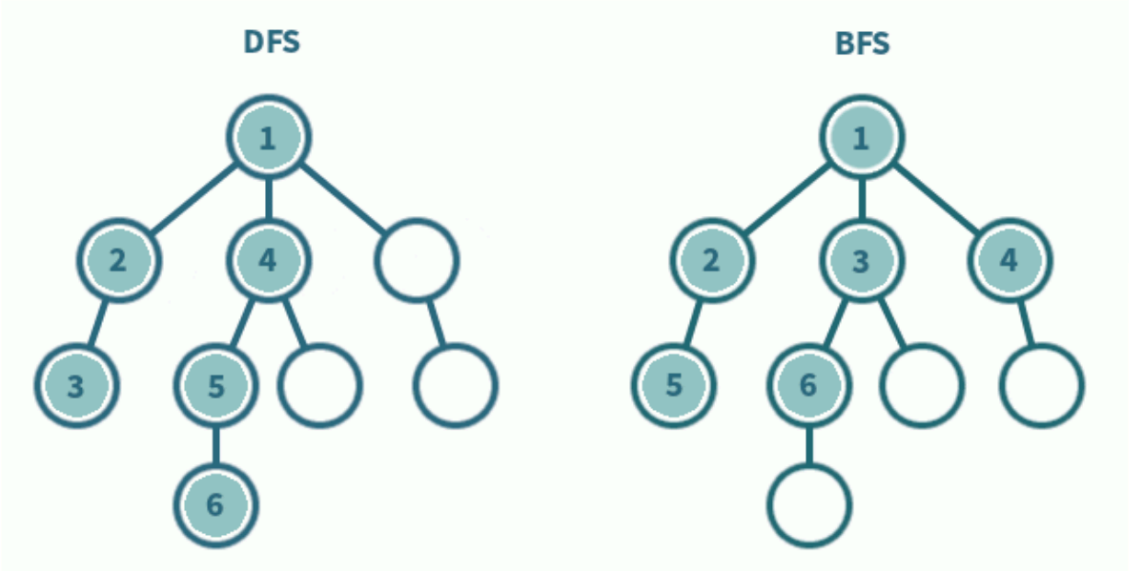
## 너비우선탐색(BFS, Breadth First Search)

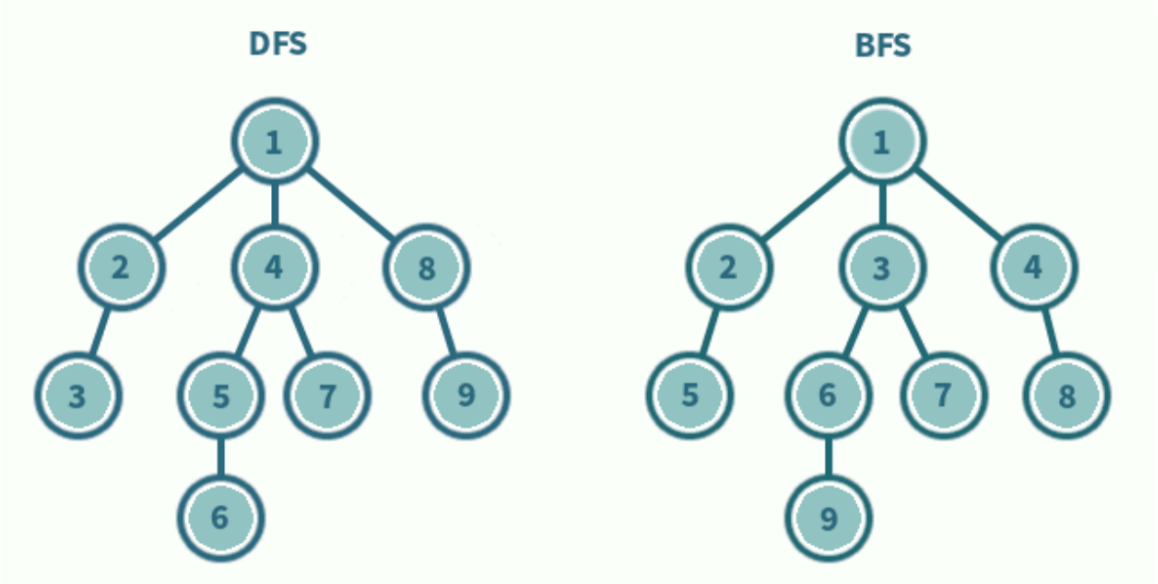
BFS는 그래프를 탐색하는 알고리즘이며 어떤 정점에서 시작해 다음 깊이의 정점으로 이동하기전 현재 깊이의 모든 정점을 탐색하며 방문한 정점은 다시 방문하지 않는 알고리즘입니다. 같은 가중치를 가진 그래프에서 최단거리알고리즘으로 쓰입니다.

BFS로 탐색을 한다는 것은 다음그림처럼 레이어별, 레벨별로 탐색한다는 뜻입니다.




DFS와 BFS의 탐색차이는 다음과 같습니다.






## BFS의 수도코드1

먼저 시작지점인 u를 "방문처리"를 하고 "큐(queue)에 푸시"를 합니다. 그리고 q.size() 만큼 while 반복문을 돌면서 큐 앞단의 있는 u를 다시 끄집어내서 그 u를 중심으로 인접한 노드들을 탐색을 합니다. 방문한 정점은 다시 방문하지 않고 방문처리를 하면서 큐(queue)에 푸시를 하며 방문처리를 합니다.

```
BFS(G, u)
    u.visited = true
    q.push(u);
    while(q.size())
        u = q.front()
        q.pop()
        for each v ∈ G.Adj[u]
            if v.visited == false
                v.visited = true
                q.push(v)
```

## BFS의 수도코드2

수도코드1은 방문처리만을 하는 코드입니다. 하지만 보통 문제에서 BFS를 쓸 때는 가중치가 같은 그래프 내에서 최단거리 알고리즘으로 쓰입니다. 그러기 위해 "최단거리 배열"을 방문하면서 만들어주어야 하는데 다음과 같은 수도코드를 사용합니다.

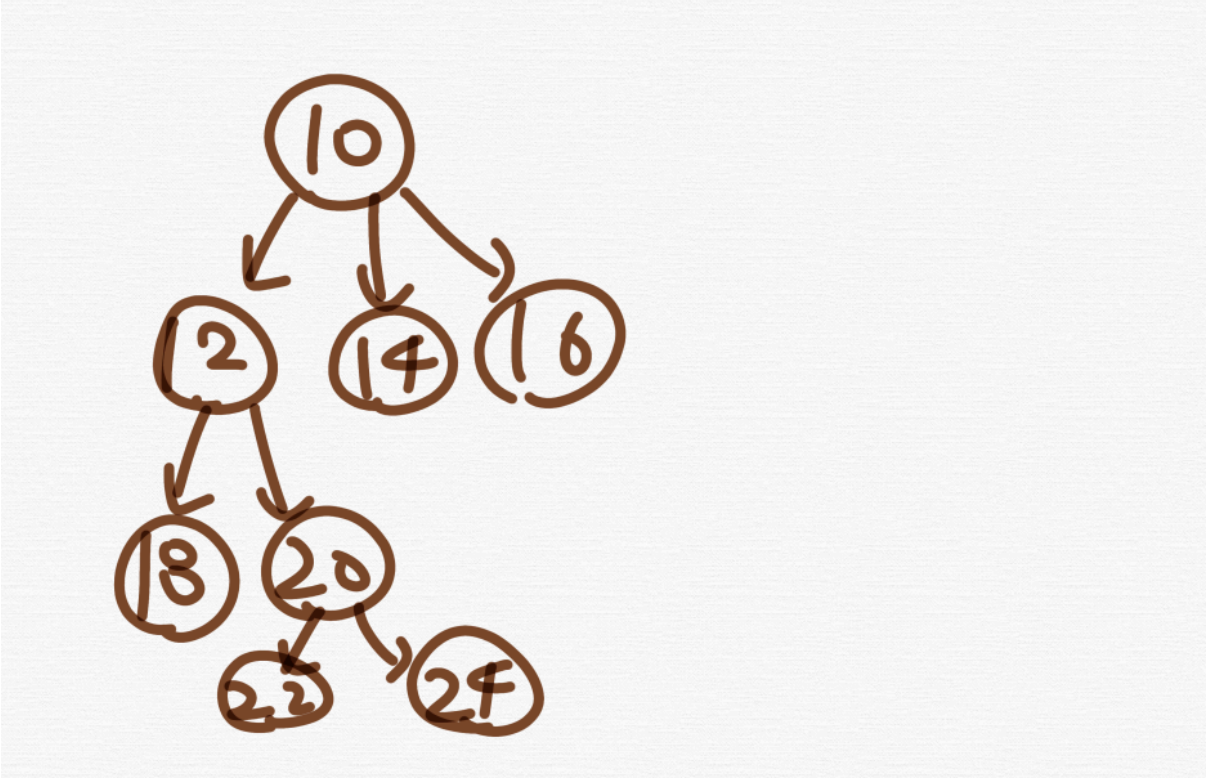
```
BFS(G, u)
    u.visited = 1
```

```
    q.push(u);
    while(q.size())
        u = q.front()
        q.pop()
        for each v ∈ G.Adj[u]
            if v.visited == false
                v.visited = u.visited + 1
                q.push(v)
```

차이점은 이 코드 한 줄입니다.

```
v.visited = u.visited + 1
```

자, 이 그래프에서 화살표는 한칸이라는 의미를 가집니다. 10번정점으로부터 24번 정점까지의 최단거리는 얼마일까요?




답은 3칸입니다. 이를 BFS로 구현해보면서 구해보겠습니다.

앞의 그래프를 인접리스트로 구현하고 10번정점 부터 탐색하는 BFS 코드입니다.

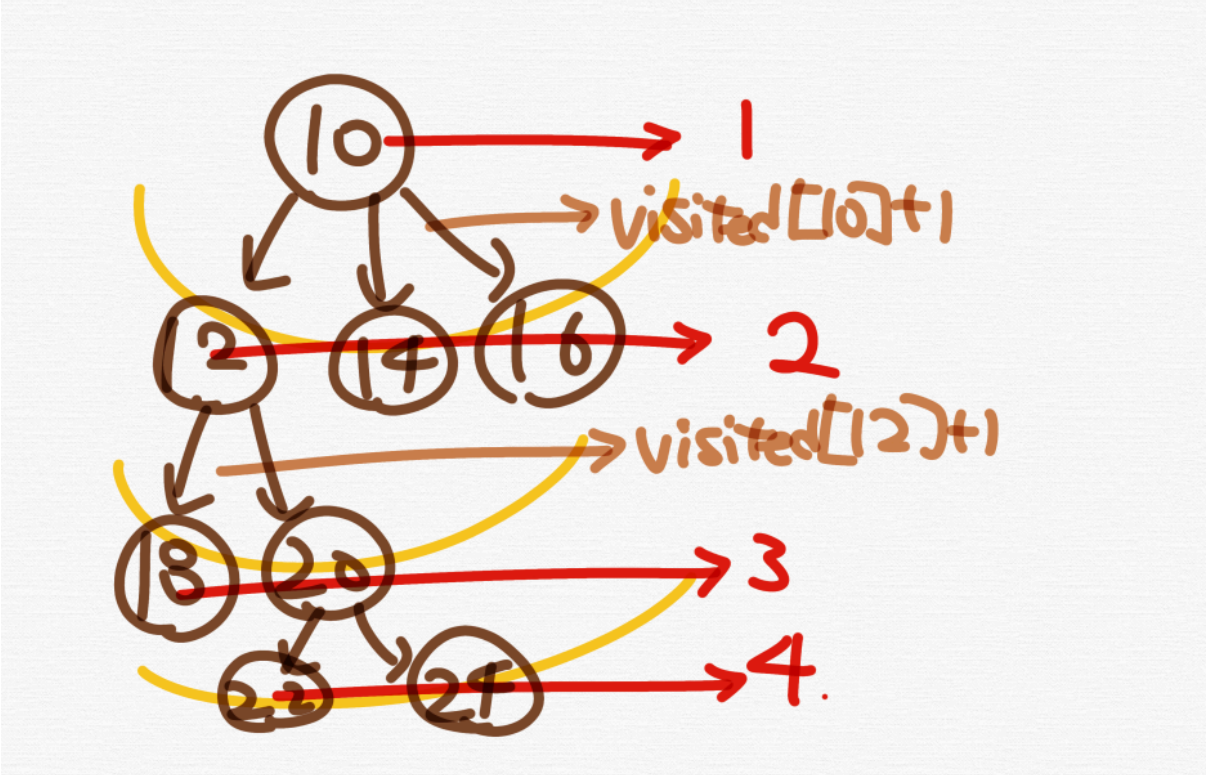
그림과 대조하면서 확인해주세요.

```cpp
#include<bits/stdc++.h>
using namespace std;
vector<int> adj[100];
int visited[100];
int nodeList[] = {10, 12, 14, 16, 18, 20, 22, 24};
void BFS(int here){
    queue<int> q;
    visited[here] = 1;
    q.push(here);
    while(q.size()){
        int here = q.front(); q.pop();
        for(int there : adj[here]){
            if(visited[there]) continue;
            visited[there] = visited[here] + 1;
            q.push(there);
        }
    }
}
int main(){
    adj[10].push_back(12);
    adj[10].push_back(14);
    adj[10].push_back(16);

    adj[12].push_back(18);
    adj[12].push_back(20);


    adj[20].push_back(22);
    adj[20].push_back(24);
    BFS(10);
    for(int i : nodeList){
        cout << i << " : " << visited[i] << '\n';
    }
    cout << "10번으로부터 24번까지 최단거리는 : " << visited[24] - 1 <<
'\n';
    return 0;
}
/*
10 : 1
12 : 2
14 : 2
16 : 2
18 : 3
20 : 3
22 : 4
```

```
24 : 4
10번으로부터 24번까지 최단거리는 : 3
*/
```

탐색원리는 다음과 같습니다.

이런식으로 탐색하게 되며 자동적으로 최단거리가 구해지는 것이죠.

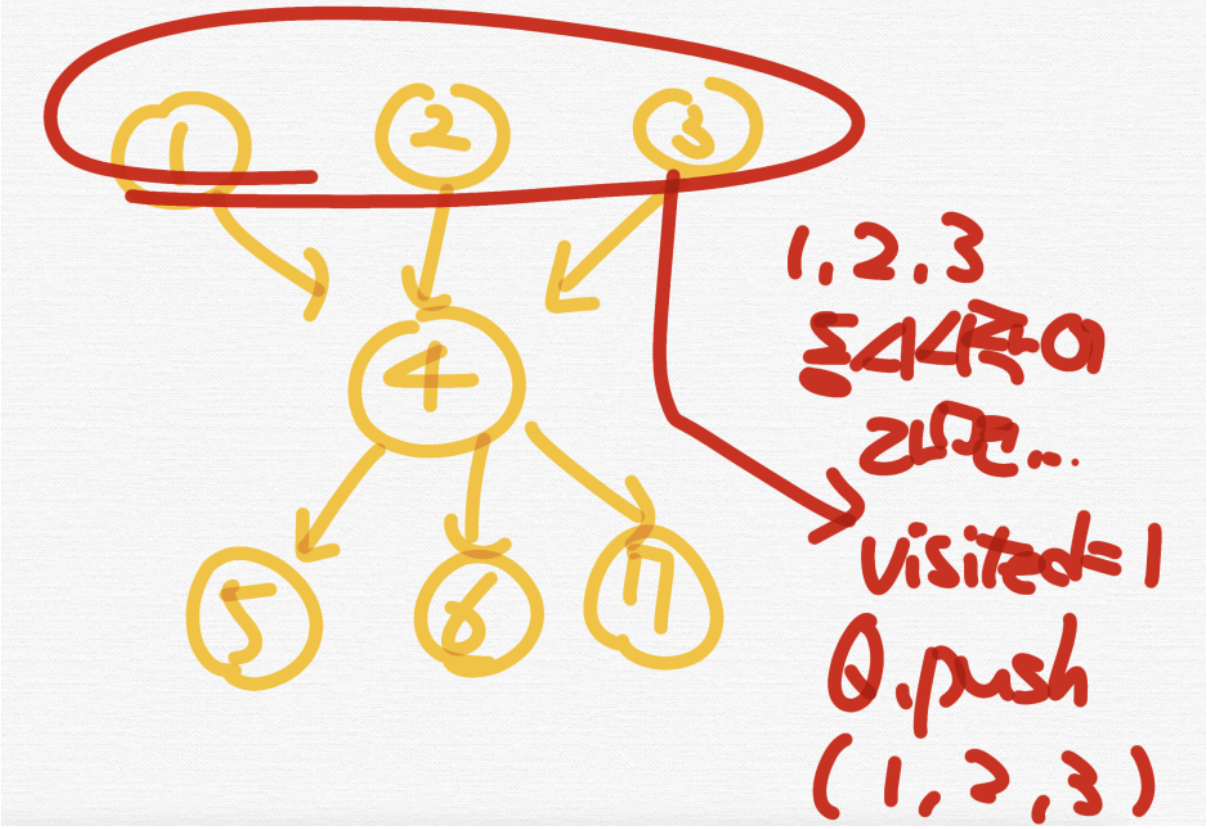



## Q. 만약 앞의 그래프의 가중치가 한칸이 아니고 2칸이라면 어떻게 될까요?

3 * 2를 하면 됩니다. 답은 6칸이 되겠죠?

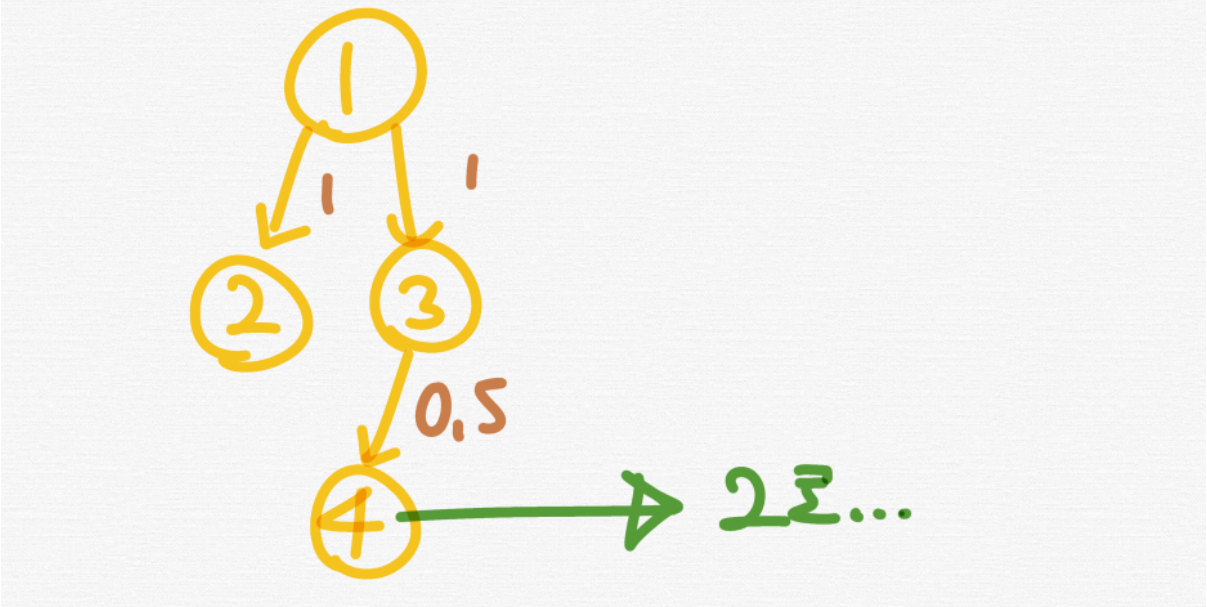
## Q. 시작지점이 다수라면?

"시작지점이 다수" 라면 처음 큐에 푸시하는 지점도 다수가 되어야 하며 해당 지점들의 visited를 모두 1로 만들면서 시작해야 합니다.



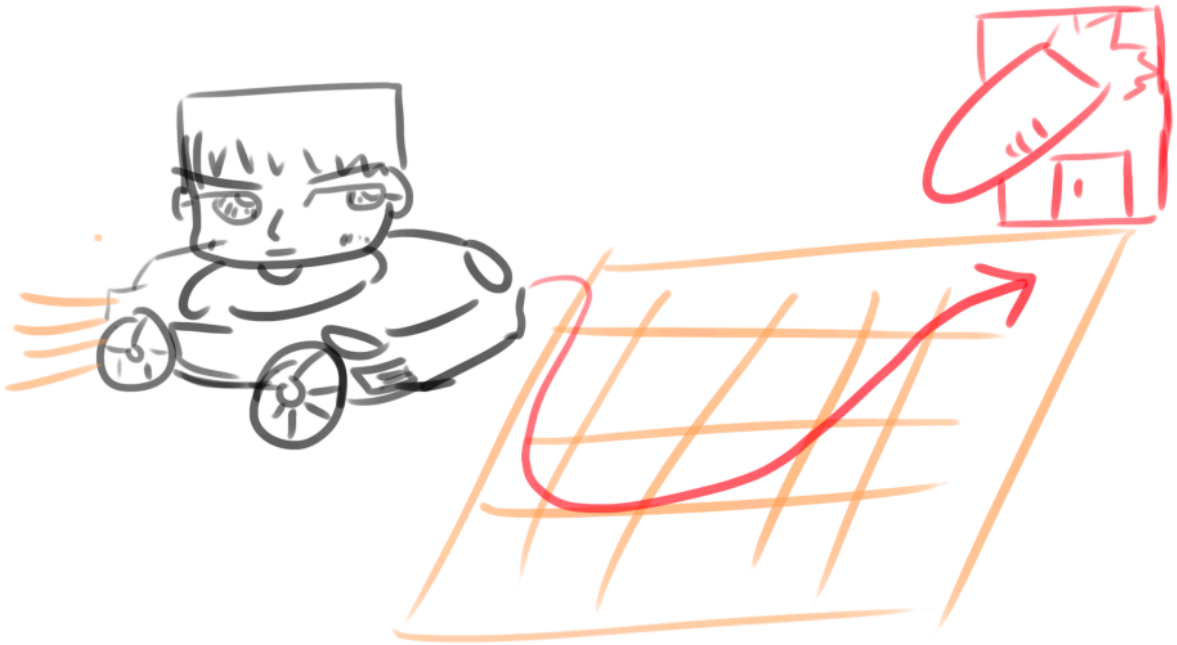

## Q. BFS는 왜 가중치가 같은 그래프내에서만 최단거리 알고리즘으로 쓰이나요? 가중치가 다른 그래프에서는 안되나요?

다음과 같은 그래프가 있다고 칩시다. 3번에서 4번으로 가는 가중치가 0.5이며 그래프내의 가중치가 서로 다르게 되죠? 이 때 1번부터 4번까지 가는 최단거리를 구한다고 했을 때 우리가 지금까지 배운 BFS로는 2밖에 출력이 안되게 됩니다. 그래서 못쓰는 것이죠.



[참고] 가중치가 다른 그래프 내에서 최단거리 알고리즘은 다익스트라, 벨만포드 등을 써야 합니다.

## Q. 당근마킷 엔지니어 승원이



승원이는 당근을 좋아해서 당근마킷에 엔지니어로 취업했다. 당근을 매우좋아하기 때문에 차도 당근차를 샀다. 이 당근차는 한칸 움직일 때마다 당근을 내뿜으면서 간다. 즉, "한칸" 움직일 때 "당근한개"가 소모된다는 것이다. 승원이는 오늘도 아침 9시에 일어나 당근마킷으로 출근하고자 한다. 승원이는 최단거리로 당근마킷으로 향한다고 할 때 당근몇개를 소모해야 당근마킷에 갈 수 있는지 알아보자. 이 때 승원이는 육지로만 갈 수 있으며 바다로는 못간다. 맵의 1은 육지며 0은 바다를 가리킨다. 승원이는 상하좌우로만 갈 수 있다.

**입력**

맵의 세로길이 N과 가로길이 M 이 주어지고 이어서 N * M의 맵이 주어진다. 그 다음 줄에 승원이의 위치(y, x)와 당근마킷의 위치(y, x)가 주어진다. 이 때 승원이의 시작위치(y, x)에서 "당근한개"가 이미 소모된 상태로 본다.

**출력**

당근을 몇개 소모해야 하는지 출력하라.

**범위**

1 <= N <= 100

1 <= M <= 100

**예제입력**

5 5

0 0

4 4

1 0 1 0 1

1 1 1 0 1

0 0 1 1 1

0 0 1 1 1

0 0 1 1 1

**예제출력**

9

**해설**

이 문제는 어떤 문제일까요? 바로 "가중치가 같은 그래프 내의 최단거리알고리즘" 문제입니다. 한칸씩 이동할 때마다 당근 한개를 쓰기 때문에 가중치가 같은 그래프이며 여기서 최단거리를 구하는 알고리즘이기 때문이죠. 그렇다면 BFS를 통해 최단거리 배열을 만들어서 해야 합니다.

**정답코드**

```
#include<bits/stdc++.h>
using namespace std;
const int max_n = 104;
int dy[4] = {-1, 0, 1, 0};
int dx[4] = {0, 1, 0, -1};
int n, m, a[max_n][max_n], visited[max_n][max_n], y, x, sy, sx, ey, ex;
int main(){
    scanf("%d %d", &n, &m);
    cin >> sy >> sx;
```

```
    cin >> ey >> ex;
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            cin >> a[i][j];
        }
    }
    queue<pair<int, int>> q;
    visited[sy][sx] = 1;
    q.push({sy, sx});
    while(q.size()){
        tie(y, x) = q.front(); q.pop();
        for(int i = 0; i < 4; i++){
            int ny = y + dy[i];
            int nx = x + dx[i];
            if(ny < 0 || ny >= n || nx < 0 || nx >= m || a[ny][nx] == 0)
continue;
            if(visited[ny][nx]) continue;
            visited[ny][nx] = visited[y][x] + 1;
            q.push({ny, nx});
        }
    }
    printf("%d\n", visited[ey][ex]);
    // 최단거리 디버깅
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            cout << visited[i][j] << ' ';
        }
        cout << '\n';
    }
    return 0;
}
```

상세설명

```
   queue<pair<int, int>> q;
    visited[sy][sx] = 1;
    q.push({sy, sx});
    while(q.size()){
        tie(y, x) = q.front(); q.pop();
        for(int i = 0; i < 4; i++){
            int ny = y + dy[i];
            int nx = x + dx[i];
            if(ny < 0 || ny >= n || nx < 0 || nx >= m || a[ny][nx] == 0)
continue;
            if(visited[ny][nx]) continue;
            visited[ny][nx] = visited[y][x] + 1;
```

```
            q.push({ny, nx});
        }
    }
```

2차원배열이기 때문에 y, x를 담기 위해 pair를 정의하고 visited 배열의 값을 1로 설정한 뒤 시작지점을 큐에 푸시를 합니다. 그 이후 큐의 크기만큼 while 반복문을 돌며 탐색을 진행합니다. 큐의 가장 앞부분에 있는 좌표를 끄집어내어 탐색을 다시 이어가며 4방향을 탐색하며 다시 visited배열을 갱신하며 큐에 넣으며 진행합니다.

여기서 핵심은 최단거리배열을 만드는 것입니다.

```
    visited[ny][nx] = visited[y][x] + 1;
```

이렇게 내가 가야할 곳인 ny, nx에 지금 있는 곳 y, x 로부터 + 1를 하면서 최단거리배열을 쌓다보면 어떻게 될까요? 바로 이렇게 쌓이게 됩니다.

```
9
1 0 5 0 9
2 3 4 0 8
0 0 5 6 7
0 0 6 7 8
0 0 7 8 9
```

이렇게 최단거리배열을 만들었을 뿐인데 -> "해당 목표좌표를 출력"하니 그 부분에 해당하는 최단거리값이 출력되는 것을 볼 수 있죠? 이렇게 BFS는 최단거리배열만 잘 만들면 됩니다.

## DFS와 BFS비교

둘 다 시간복잡도는 인접리스트로 이루어졌다면 O(V + E)이고 인접행렬의 경우 O(V^2)가 되는 것은 동일하며 다음과 같은 차이가 있습니다.

| 이름 | 설명 |
| --- | --- |
| DFS | 메모리를 덜 씀. 절단점 등 구할 수 있음. 코드가 좀 더 짧으며 완전탐색의 경우에 많이 씀. |
| BFS | 메모리를 더 씀. 가중치가 같은 그래프내에서 최단거리를 구할 수 있음. 코드가 더 김 |

문제에서 "퍼져나간다", "탐색한다" 이 2글자가 있으면 반드시 DFS, BFS가 생각이 나야 합니다. 한쪽은 깊이우선탐색이고 한쪽은 너비우선탐색이라 서로의 길이 다르지만 뭐 어떻습니까. 모두다 퍼져나거나 탐색할 때 쓰이는 알고리즘인 걸요.

# 트리순회

트리 순회(Tree traversal)는 트리 구조에서 각각의 노드를 정확히 한 번만, 체계적인 방법으로 방문하는 과정을 말합니다. 이는 노드를 방문하는 순서에 따라 후위순회, 전위순회, 중위순회, 레벨순회가 있습니다. 보통 설명할 때는 이진트리를 기반으로 설명하지만 다른 모든 트리에서 일반화를 시킬 수 있습니다.

## 후위순회

후위순회(postorder traversal)는 자식들 노드를 방문하고 자신의 노드를 방문하는 것을 말합니다.

[참고] post : ~이후의를 뜻하는 접두사

**수도코드**

```
postorder( node )
    if (node.visited == false)
        postorder( node->left )
        postorder( node->right )
        node.visited = true
```

방문한다라는 것은 visited 배열을 색칠하는 것을 생각해주세요.

## 전위순회

전위순회(preorder traversal)는 먼저 자신의 노드를 방문하고 그 다음 노드들을 방문하는 것을 말합니다. (DFS를 생각하면 됩니다.)

[참고] pre : ~이전의를 뜻하는 접두사

**수도코드**

```
preorder( node )
    if (node.visited == false)
        node.visited = true
        preorder( node->left )
        preorder( node->right )
```

## 중위순회

중위순회(inorder traversal)는 왼쪽 노드를 먼저 방문 그다음의 자신의 노드를 방문하고 그 다음 오른쪽 노드를 방문하는 것을 말합니다. 안에 끼워넣다. 를 생각해야 합니다.

**수도코드**

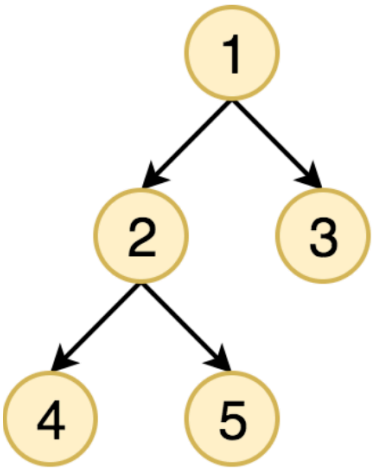
```
inorder( node )
    if (node.visited == false)
        inorder( node->left )
        node.visited = true
        inorder( node->right )
```

## 레벨순회

레벨순회(level traversal)입니다. 앞서 설명한 BFS를 생각하시면 됩니다.

### Q. 아래의 그래프가 주어졌을 때 preorder, inorder, postorder를 구현하라.

수도코드를 보고 코드를 구축하는 연습을 하셔야 합니다. 아래와 같이 그래프가 주어졌을 때 비어진 함수를 채워서 preorder, inorder, postorder를 구현하고 이 때 방문했을 때 해당 노드를 출력하는 것을 구현해보세요.




```
#include <bits/stdc++.h>
```

```
using namespace std;
vector<int> adj[1004];
int visited[1004];

void postOrder(int here){
}
void preOrder(int here){
}
void inOrder(int here){

}
int main(){
      adj[1].push_back(2);
      adj[1].push_back(3);
      adj[2].push_back(4);
      adj[2].push_back(5);
      int root = 1;
    cout << "\n 트리순회 : postOrder \n";
    postOrder(root); memset(visited, 0, sizeof(visited));
    cout << "\n 트리순회 : preOrder \n";
    preOrder(root); memset(visited, 0, sizeof(visited));
    cout << "\n 트리순회 : inOrder \n";
    inOrder(root);
    return 0;
}
```

**정답코드**

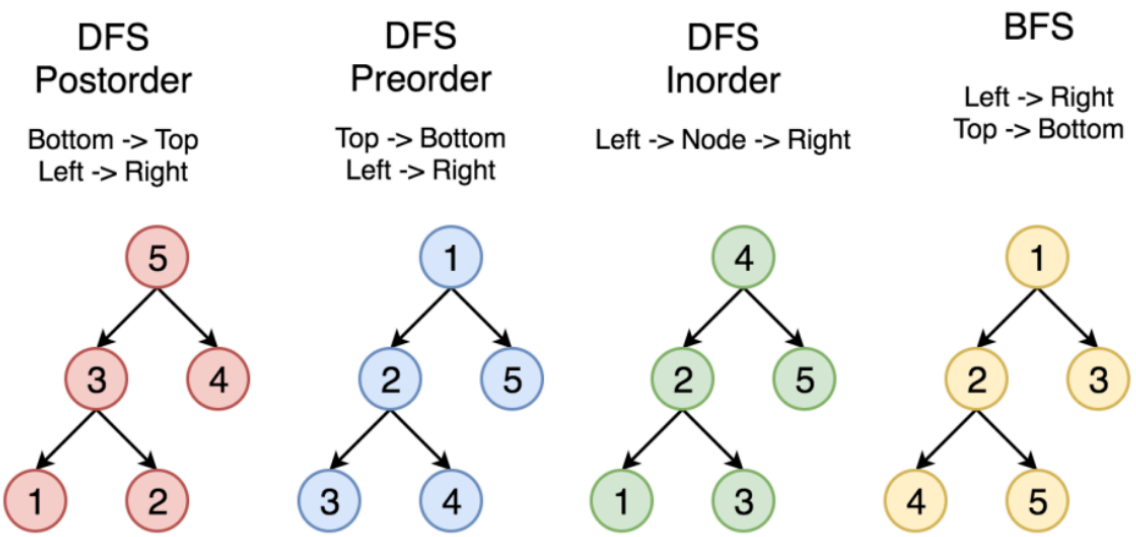
```
#include <bits/stdc++.h>
using namespace std;
vector<int> adj[1004];
int visited[1004];

void postOrder(int here){
      if(visited[here] == 0){
            if(adj[here].size() == 1)postOrder(adj[here][0]);
            if(adj[here].size() == 2){
                  postOrder(adj[here][0]);
                  postOrder(adj[here][1]);
            }
            visited[here] = 1;
            cout << here << ' ';
      }
}
void preOrder(int here){
```

```
	if(visited[here] == 0){
		visited[here] = 1;
		cout << here << ' ';
		if(adj[here].size() == 1)preOrder(adj[here][0]);
		if(adj[here].size() == 2){
			preOrder(adj[here][0]);
			preOrder(adj[here][1]);
		}
	}
}
void inOrder(int here){
	if(visited[here] == 0){
		if(adj[here].size() == 1){
			inOrder(adj[here][0]);
			visited[here] = 1;
			cout << here << ' ';
		}else if(adj[here].size() == 2){
			inOrder(adj[here][0]);

			visited[here] = 1;
			cout << here << ' ';

			inOrder(adj[here][1]);
		}else{
			visited[here] = 1;
			cout << here << ' ';
		}
	}

}
int main(){
	adj[1].push_back(2);
	adj[1].push_back(3);
	adj[2].push_back(4);
	adj[2].push_back(5);
	int root = 1;
    cout << "\n 트리순회 : postOrder \n";
    postOrder(root); memset(visited, 0, sizeof(visited));
    cout << "\n 트리순회 : preOrder \n";
    preOrder(root); memset(visited, 0, sizeof(visited));
    cout << "\n 트리순회 : inOrder \n";
    inOrder(root);
    return 0;
}
/*
```

```
 트리순회 : postOrder
4 5 2 3 1
 트리순회 : preOrder
1 2 4 5 3
 트리순회 : inOrder
4 2 5 1 3
*/
```

## 정리

수도코드처럼 어떤 부분을 "방문처리를 할까?"만을 생각하면 됩니다. 사실 이 그래프는 단방향간선이기 때문에 visited배열은 쓰지 않아도 됩니다. 하지만 수도코드에 있기 때문에 넣어서 구현해봤습니다.




앞의 그림처럼 postorder의 경우 4, 5, 2, 3, 1 이런식으로 방문되는 것을 볼 수있죠? 앞의 그림과 비교하면서 구현하는 것을 연습해 나가시면 됩니다.

# 3주차 개념

이번주차에는 삼성전자 기출로도 잘 나오며 코딩테스트에서 가장 많이 나오는 유형인 완전탐색과 백트래킹에 대해서 알아보겠습니다.

# 완전탐색

완전탐색은 brute force, exhaustive key search라고도 불리며 모든 경우의 수를 탐색하는 알고리즘입니다.(흔히 노가다라고도 합니다.)

그렇다면 경우의 수란 뭘까요? 바로 순열 또는 조합을 말합니다. 보통 완전탐색은 (조합 또는 순열) + 로직(DFS 또는 BFS)으로 이루어져 있습니다.  그러면 문제를 보고 어떨 때 이 완전탐색을 쓰는가를 판단할 수 있을까요? 보통은 시간복잡도가 1억 이하일 때 완전탐색을 써도 됩니다.

시간복잡도가 1억이 떠도 완전탐색, 즉 모든 경우의 수를 계산해서 해결할 수도 있습니다. 물론 거대한 숫자지만, 우리의 컴퓨터는 해낼 수 있습니다. 믿고 코드를 짜봅시다.

## 반복문을 활용한 완전탐색

완전탐색은 구현방법이 2가지입니다. 먼저  for 반목문 또는 while반목문을 이용한 완전탐색입니다.

다음 코드처럼 단순히 선형적으로 숫자 찾는 것도 완전탐색입니다. 그 수가 A야? 하는 것을 일일히 확인해보며 모든 경우의 수를 확인하니까요.

```
#include<bits/stdc++.h>
using namespace std;
int main() {
	vector<int> v = {1, 3, 2, 5, 6, 7, 8};
	for(int i = 0; i < v.size(); i++){
		if(v[i] == 5){
```

```
                cout << i << "\n";
                break;
        }
    }
    int i = 0;
    while(i < v.size()){
        if(v[i] == 5){
            cout << i << "\n";
            break;
        }
        i++;
    }
    return 0;
}
```

## Q. 2400!! 2400!! 2400!!

트위치BJ 랄로는 2400이란 숫자를 좋아한다. 파카는 랄로의 수를 만들고자 하는데 랄로의 수란 2400이 들어간 수를 말한다. 첫번째 랄로의 수는 2400이고 두번째 랄로의 수는 수는 12400, 세번째 랄로의 수는 22400이다.

N이 입력으로 주어졌을 때 N번째 랄로의 수를 구하라. N은 300이하로 주어진다.

정답코드:

```
#include<bits/stdc++.h>
using namespace std;
int cnt, n;
int main() {
    cin >> n;
    int i = 2400;
    while(true){
        string a = to_string(i);
        if(a.find("2400") != string::npos){
            cnt++;
            if(n == cnt){
                cout << a << '\n';
                break;
            }
        }
        i++;
```

```
        }
        return 0;
}
```

## 재귀함수를 활용한 완전탐색

이번에는 재귀함수를 활용한 완전탐색을 알아보겠습니다.

하지만 중요한 점이 있습니다. 만약 문제를 봤을 때 반복문을 활용한 완전탐색으로 되면 무조건 반복문으로 해야합니다. 보통의 경우 재귀함수를 활용한 완전탐색은 시간복잡도가 반복문보다 더 큽니다.  함수호출을 여러번 하는 것은 코스트가 너무나도 크기 때문입니다.

반복문으로 구현하기가 힘들다? 또는 어떠한 행위는 반복하는데 매개변수만 수정해서 넘기면 될 거 같다? 그러면 재귀함수로 하는게 좋습니다.

예를 들어 nC1, nC2, nC3 .. 등 이런 경우의 수를 다 생각해야 한다면 어떻게 해야할까요? 이런 것은 반복문으로 하기에는 어렵습니다. 이 때 재귀함수로 해야하는 것이죠.

재귀함수로 하는게 나음. (물론 비트마스킹도 있지만...)

## Q.승철이의 문단속

승철이는 도쿄 위의 빨간 구름위에 올라가있다. 이 구름은 그대로 내버려두면 땅으로 떨어져 100만명의 사상자가 발생한다. 구름을 멈추는 방법은 구름의 특정 위치에 요석을 꽂으면 된다. 해당 위치에는 숫자가 표기가 되어있고 몇 개를 골라 숫자의 합이 “소수 " 가 될 때 구름은 멈춘다. 총 몇 개의 경우의 수가 있는지 말하라.

N개의 요석 후보의 숫자와 다음 줄에 해당 숫자들이 나온다. N <= 100

[3D입체음향] 스즈메의 문단속OST - 🗼 도쿄상공(東京上空) | すずめの戸締まり [고음질]

[참고링크] : https://youtu.be/qmNBT3_wpko

**입력:**

10

24 35 38 40 49 59 60 67 83 98

**출력:**

176

```
#include<bits/stdc++.h>
using namespace std;
int n, temp;
vector<int> v;
bool check(int n) {
    if(n <= 1) return 0;
    if(n == 2) return 1;
    if(n % 2 == 0) return 0;
    for (int i = 3; i * i <= n; i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}
int go(int idx, int sum){
      if(idx == n){
            //cout << "SUM " << sum << "\n";
            return check(sum);
```

```
	}
	return go(idx + 1, sum + v[idx]) + go(idx + 1, sum);
}
int main() {
	cin >> n;
	for(int i = 0; i < n; i++){
		cin >> temp;
		v.push_back(temp);
	}
	cout << go(0, 0) << "\n";
	return 0;
}
```

# 백트래킹

백트래킹(back tracking)은 완전탐색 + 가지치기라고 보시면 됩니다.

완전탐색에 인간미를 섞었다라고 보면 되는데 “어? 이런 경우의 수는 탐색을 하지 않아도 될텐데? “라고 가늠하면서 최대한 불필요한 탐색을 피하는 알고리즘입니다.

백트래킹(Backtracking)은 완전탐색(Brute Force)과 가지치기(Pruning)를 결합한 방식입니다. 이 방법은 모든 가능한 해를 찾는 과정에서 불필요한 탐색을 줄여줍니다.

## 완전탐색과의 차이

완전탐색은 가능한 모든 경우의 수를 탐색하여 해를 찾는 방법입니다. 하지만 이 과정에서 많은 비효율적인 탐색이 발생할 수 있습니다. 백트래킹은 이러한 비효율적인 탐색을 줄이기 위해 "이런 경우의 수는 탐색을 하지 않아도 된다"는 판단을 내리며 불필요한 경로를 배제합니다. 이를 통해 더 적은 연산으로 목표 해를 찾을 수 있게 합니다.

## 가지치기

가지치기(Pruning)는 백트래킹의 핵심 기법 중 하나로, 현재 탐색하고 있는 경로가 목표 해에 도달할 가능성이 없다고 판단되면 더 이상 그 경로를 탐색하지 않고 되돌아가는(Backtracking) 방법입니다. 이는 마치 "여기서 더 진행해봤자 의미가 없다"라고 가정하고, 다른 경로를 탐색하는 방식입니다. 이로 인해 알고리즘의 효율성이 크게 향상됩니다.

백트래킹은 주로 재귀(Recursion)를 통해 구현됩니다. 각 단계에서 가능한 선택지를 시도하고, 그 결과가 유망하지 않으면(가지치기) 이전 단계로 돌아가 다른 선택지를 시도합니다.

이 과정이 반복되면서 최종적으로 목표 해를 찾거나 모든 가능한 해를 탐색합니다.

문제를 풀어보면서 개념을 알아보도록 하겠습니다.

## Q. N과 N개의 자연수가 주어진다. 여기서 몇개의 숫자를 골라 합을 mod 11을 했을 때 나오는 가장 큰수를 구하라.

**입력:**

10

24 35 38 40 49 59 60 67 83 98

**출력:**

10

이를 완전탐색으로 하게 된다면?

```
#include<bits/stdc++.h>
using namespace std;
int n, temp, ret;
vector<int> v;
const int mod = 11;

void go(int idx, int sum){
	if(idx == n){
		ret = max(ret, sum % mod);
		return;
	}
	go(idx + 1, sum + v[idx]);
	go(idx + 1, sum);
}
int main() {
	cin >> n;
	for(int i = 0; i < n; i++){
		cin >> temp;
		v.push_back(temp);
	}
	go(0, 0);
	cout << ret << "\n";
	return 0;
}
```

이렇게 됩니다. 하지만 너무나도 불필요한 경우의 수를 탐색을 많이 하는 코드입니다. 디버깅 코드를 추가해볼게요.

```
#include<bits/stdc++.h>
using namespace std;
int n, temp, ret, cnt;
vector<int> v;
const int mod = 11;

void go(int idx, int sum){
	if(idx == n){
		ret = max(ret, sum % mod);
		cnt++;
		return;
	}
	go(idx + 1, sum + v[idx]);
	go(idx + 1, sum);
}
int main() {
	cin >> n;
	for(int i = 0; i < n; i++){
		cin >> temp;
		v.push_back(temp);
	}
	go(0, 0);
	cout << ret << "\n";
	cout << cnt << "\n";
	return 0;
}
```

앞의 코드처럼 cnt를 찍어봤더니 1024가 나오는 것을 알 수 있습니다. 무려 1024의 모든 경우의 수를 모조리 탐색하는 것을 볼 수 있습니다.

근데 이 문제를 자세히 보면 **mod N를 하게 된다면 해당 숫자의 범위는 0 ~ n - 1의 범위를 가지는 것**이라는 사실을 알 수 있습니다.

따라서, 11 mod는 10이 최대의 숫자가 된다는 것은 자명합니다. 즉, 10을 찾으면 다른 경우의 수는 가지치기를 해도 되는 것이죠. 이렇게 문제의 핵심을 파악하고 해당 경우의 수를 탐색하지 않는 것이 바로 백트래킹입니다.

```
#include<bits/stdc++.h>
using namespace std;
```

```
int n, temp, ret;
vector<int> v;
const int mod = 11;
int cnt = 0;
void go(int idx, int sum){
	if(ret == 10) return;
	if(idx == n){
		ret = max(ret, sum % mod);
		cnt++;
		return;
	}
	go(idx + 1, sum + v[idx]);
	go(idx + 1, sum);
}
int main() {
	cin >> n;
	for(int i = 0; i < n; i++){
		cin >> temp;
		v.push_back(temp);
	}
	go(0, 0);
	cout << ret << "\n";
	cout << cnt << "\n";
	return 0;
}
```

앞의 코드에서 다음 코드부분이 백트래킹의 핵심입니다.

```
if(ret == 10) return;
```

간단하죠?

이렇게 필요없는 부분에 대한 경우의 수를 return; 등으로 빠르게 종료시켜 해당 부분에 대한 탐색을 이어나가지 않게 하는 것입니다. 그렇다면 디버깅 코드르 추가해 얼마나 가지치기가 되었는지 보겠습니다.

```
#include<bits/stdc++.h>
using namespace std;
int n, temp, ret, cnt;
vector<int> v;
const int mod = 11;

void go(int idx, int sum){
```

```
	if(ret == 10) return;
	if(idx == n){
		ret = max(ret, sum % mod);
		cnt++;
		return;
	}
	go(idx + 1, sum + v[idx]);
	go(idx + 1, sum);
}
int main() {
	cin >> n;
	for(int i = 0; i < n; i++){
		cin >> temp;
		v.push_back(temp);
	}
	go(0, 0);
	cout << ret << "\n";
	cout << cnt << "\n";
	return 0;
}
```

찍어보면 cnt는 10이 찍히게 됩니다. 1024의 경우의 수가 10개의 경우의 수로 줄어들었습니다.

이로써 필요없는 경우의 수는 모조리 가지치기를 한 것을 볼 수 있습니다.

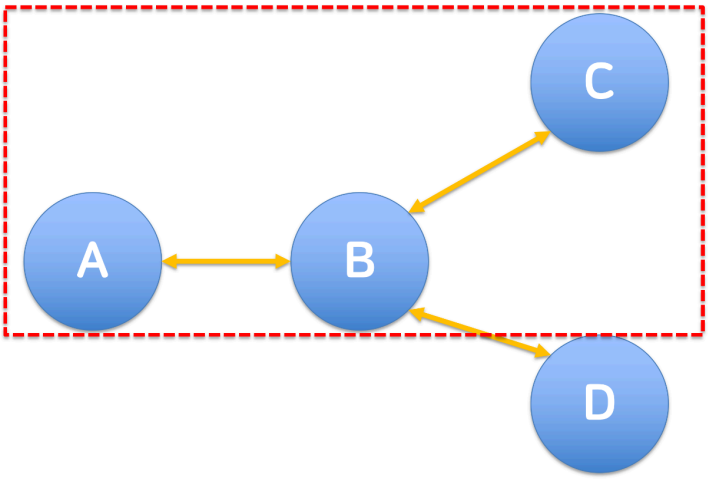
## 완전탐색 - 원복

완전탐색은 가능한 모든 경우의 수를 탐색하여 해를 찾는 방법입니다. 하지만 탐색 과정에서 특정 경로를 탐색한 후, 그 경로에 대한 상태값이 다른 경로의 탐색에 영향을 미쳐서는 안 됩니다. 이때 필요한 것이 바로 원복입니다.

원복은 탐색 과정에서 특정 경로를 탐색한 후, 그 경로에 대한 상태를 이전 상태로 복구하는 과정입니다. 이는 탐색 과정이 마치 "색칠하고, 다시 지우는" 것과 비슷합니다. 즉, 현재의 경로를 탐색하고 나서, 그 탐색에 대한 흔적을 제거하여 다른 경로 탐색에 영향을 주지 않도록 합니다.

구현하는 방법은 방문배열인 visited 를 통해 "색칠하고" "다시 지운다(원복)"를 통해 구현합니다.

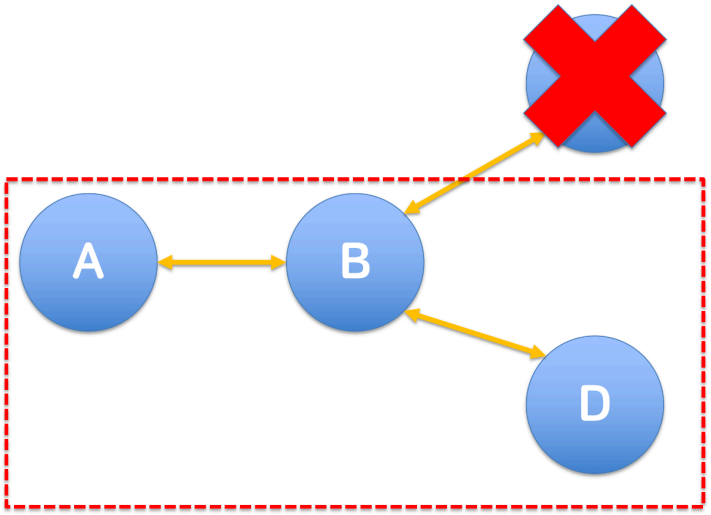
예를 들어 다음의 그래프에서 A -> B -> C 와 A -> B -> D라는 모든 경로의 수를 출력하는 문제가 있다고 해보겠습니다.




보통은 방문 배열을 통해 A, B, C를 색칠하게 됩니다. 색칠한다는 것을 코드로 나타내면 visited[A] = 1 따위가 됩니다. 자 그다음에 A -> B -> D라는 경우의 수를 뽑아낼 때 입니다.

어떻게 해야할까요? C라는 상태가 색칠되어있지만, 해당 상태를 지워야 합니다.

A - > B - >D라는 경우의 수를 뽑아낼 때 C라는 정점은 필요없는 경우의 수고 C는 D로 갈 때 영향을 미치면 안되는 경우의 수기 때문입니다.



**정답코드:**

```
#include <bits/stdc++.h>
using namespace std;
int visited[4];
vector<int> adj[4];
vector<int> v;
void print(){
      for(int i : v) cout << char(i + 'A') << " ";
      cout << '\n';
}

void go(int idx){
    if(v.size() == 3){
        print(); return;
    }
      for(int there : adj[idx]){
            if(visited[there]) continue;
            visited[there] = 1;
            v.push_back(there);
            go(there);
            visited[there] = 0;
            v.pop_back();
      }
}
int main() {
      adj[0].push_back(1);
      adj[1].push_back(2);
      adj[1].push_back(3);
      adj[1].push_back(0);
      adj[2].push_back(1);
      adj[3].push_back(1);

      visited[0] = 1;
      v.push_back(0);
      go(0);
    return 0;
}
```

**출력:**

A B C

A B D

앞의 코드에서 중요한 점은 바로 이부분입니다.

```
            visited[there] = 1;
            v.push_back(there);
            go(there);
            visited[there] = 0;
            v.pop_back();
```

풀어 해석해보면 다음과 같습니다.

해당 정점을 방문한다. 는 다음의 코드로 나타낼 수 있습니다.

```
visited[there] = 1
```

그리고 해당 정점을 나중에 출력할 수 있도록 vector v에 push한다. (이부분은 문제에서 모든 경우의 수를 출력하라는 말이 없다면 굳이 vector v에 넣지 않아도 됩니다.)

```
            visited[there] = 1;
            v.push_back(there);
```

자 이렇게 해서 해당 정점을 방문했다면 해당 방문한 정점의 상태값을 지워버립니다.

```
            visited[there] = 0;
            v.pop_back();
```

이를 통해 A - > B - > C와 A - > B -> D는 서로 독립적인 경우의 수라는 상태값으로 남아있을 수 있는 것이죠.

색칠한 경우의 수의 상태값이 다음 경우의수에 영향을 미치지 않도록 A -> B -> C가 아닌 A - > B 상태로 그 이전 상태로 원상복구하는 것을 원복이라고 합니다.

**주요코드 주석:**

```
void go(int idx) {
```

```
    if(v.size() == 3) {  // 경로의 길이가 3이 되었을 때 출력
        print();
        return;
    }
    for(int there : adj[idx]) {
        if(visited[there])
            continue;  // 이미 방문한 정점은 건너뜀
        visited[there] = 1;  // 해당 정점을 방문했다고 표시
        v.push_back(there);  // 경로에 추가
        go(there);  // 다음 정점으로 재귀적으로 탐색
        visited[there] = 0;  // 탐색 후 방문 상태 복구 (원복)
        v.pop_back();  // 경로에서도 해당 정점 제거 (원복)
    }
}
```

## Q. 긍정왕 홍철이의 구걸 여행

홍철이는 3 * 3 맵에서 {0, 0} 지점에서 길을 잃어버렸다. 긍정왕 홍철이는 길을 잃어버린 김에 구걸을 하면서 돈을 모으면서 여행을 가려고 한다. 목적지는 {2, 2}이며 방문한 정점은 다시 방문할 수 없고 해당 맵에 구걸로 얻을 수 있는 돈들이 있다. 홍철이는 4방향(상하좌우)로 움직일 수 있다. {2, 2}까지 간다고 했을 때 이 돈들을 모으는 모든 경우의 수를 출력하여라.

**맵 :**

{10, 20, 21},

{70, 90, 12},

{80, 110, 120}

```
#include <bits/stdc++.h>
using namespace std;
const int n = 3;
int a[3][3] = {
{10, 20, 21},
{70, 90, 12},
{80, 110, 120}
};
int visited[3][3];
const int dy[] = {-1, 0, 1, 0};
const int dx[] = {0, 1, 0, -1};
```

```
vector<int> v;
void print(){
	for(int i : v) cout << i << " ";
	cout << '\n';
}

void go(int y, int x){
	if(y == n - 1 && x == n - 1){
		print();
		return;
	}
	for(int i = 0; i < 4; i++){
		int ny = y + dy[i];
		int nx = x + dx[i];
		if(ny < 0 || nx < 0 || ny >= n || nx >= n) continue;
		if(visited[ny][nx]) continue;
		visited[ny][nx] = 1;
		v.push_back(a[ny][nx]);

		go(ny, nx);

		visited[ny][nx] = 0;
		v.pop_back();
	}
}
int main() {
	visited[0][0] = 1;
	v.push_back(a[0][0]);
	go(0, 0);
    return 0;
}
```

**출력:**

10 20 21 12 120

10 20 21 12 90 110 120

10 20 21 12 90 70 80 110 120

10 20 90 12 120

10 20 90 110 120

10 20 90 70 80 110 120

10 70 90 20 21 12 120

10 70 90 12 120

10 70 90 110 120

10 70 80 110 90 20 21 12 120

10 70 80 110 90 12 120

10 70 80 110 120

# 4주차 개념

# 이진수

비트마스킹을 이해하기 위해서는 이진수를 이해해야 합니다.우리가 평소에 표현하는 수는 0 ~ 9라는 수를 기반으로 수를 표현하는 "십진법"인데요..

먼저 십진법을 예로 들어보겠습니다. 123은 어떻게 이루어진 수일까요?오른쪽 끝에서부터 보면 다음과 같습니다.

3 * 10 ^ 0

을 더하고

2 * 10 ^ 1

을 더하고

1 * 10 ^ 2

를 더한 것을 알 수 있습니다.

그리고 이 각각의 자리는 0 ~ 9로 10개의 숫자로 "표현"됩니다.

자 이제 그러면 이진수를 볼까요? 0 과 1, 두개의 숫자로 표현하는 "이진법"으로 표현하는 수가 바로 이진수라고 합니다. 일반적으로 이진법의 수를 십진법의 수와 구별하기 위해 다음과 같은 방법을 씁니다.

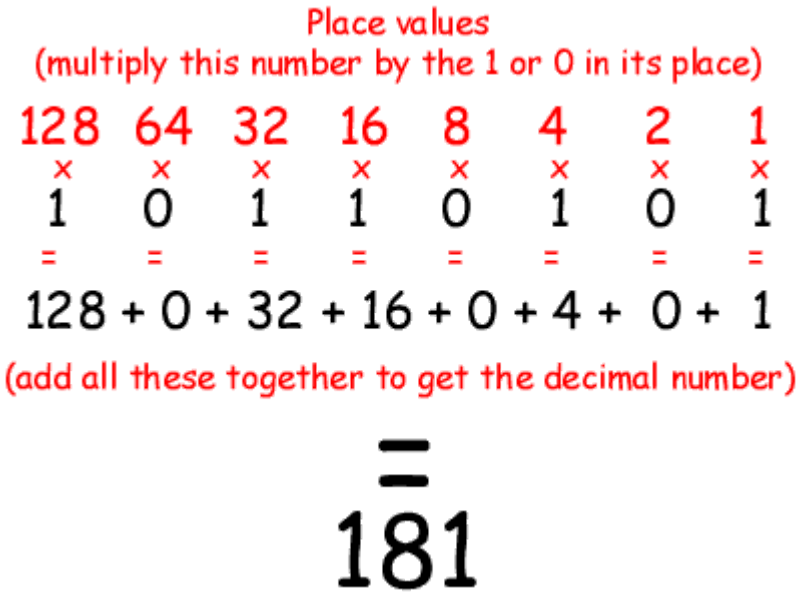
- 100101b (b를 덧붙임(b는 binary의 약자))(binary = 이진)

- $100101_{(2)}$ ((2)를 덧붙임, 주로 수학에서 쓰임)

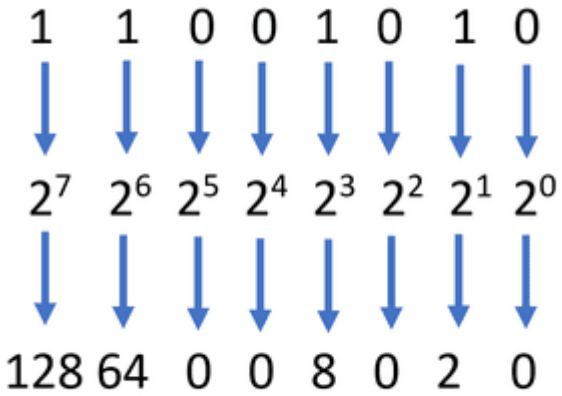
- 0b100101 (앞에 0b를 덧붙임)

자 그러면 예제를 통해 10진수를 2진수로 표현해보며 이진수를 알아보겠습니다. 오른쪽 끝에서부터 각각의 자리는 1부터 2가 곱해지며 1, 2, 4, 8, 16, 32 ... 이런식으로 2배씩 증가하게 되며 수를 표현합니다. 각각의 자리는 "비트"라고 할 수 있으며 0인지 1인지를 통해 해당 수 1 또는 2, 4 등을 더하지 않거나 더하는 걸 기반으로 수를 표현합니다.

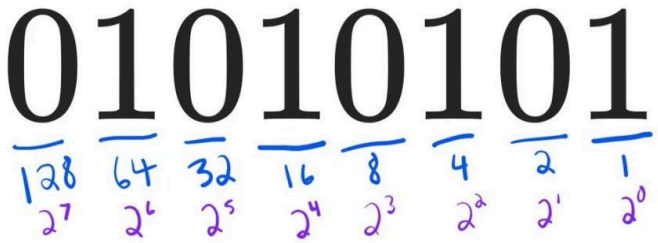
다음 그림은 181을 표현하는 이진수인 10110101 입니다.




다음그림은 어떤 수를 뜻하는걸까요? 이진수로 11001010이며 십진법으로 만들고자 했을 때 각 자리수를 더해서 128 + 64 + 8 + 2가 되어 202라는 수가 나오는 것을 알 수 있습니다.




이렇게 각 비트가 나타내는 수는 오른쪽 끝부터 2의 0승, 2의 1승 2의 2승... 이 됩니다. 다음 그림은 어떤 수를 의미할까요? 64 + 16 + 4 + 1 = 85가 되겠죠?



자, 그러면 이 표가 이해가 가시나요?

[표2] 10진수의 2진수 표현

| 10진수 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| 2진수 | 1 | 10 | 11 | 100 | 101 | 110 | 111 | 1000 | 1001 |

하나씩 써보면서 해보시면 이해가 가실 겁니다. 이렇게 이진수에 대해서 알아봤습니다.

## 이진수를 이용한 불리언(boolean) 배열

1은 {0, 0, 1}으로 표현될 수 있고, 3은 {0, 1, 1}으로 표현될 수 있습니다.

다시 설명하자면 다음과 같습니다.

1. 0번째 비트가 켜져있다 = 0번째 포함되어있다. = 001 = 1

2. 0, 1번째 비트가 켜져있다. = 0, 1번째 포함되어있다. = 011 = 3

즉, 이진수를 이용해서 “하나의 숫자”를 기반으로 불리언배열을 표현할 수 있습니다.

## 비트연산자 기초 +, -

비트연산자의 기초인 +와 -에 대해서 알아보겠습니다.

참고로 다음이 비트연산자입니다.

| 연산 | 설명 |
|---|---|
| & | 비트단위로 AND 연산을 한다. |
| \| | 비트단위로 OR 연산을 한다. |

| | |
|---|---|
| ^ | 비트단위로 XOR 연산을 한다. |
| ~ | 피연산자의 모든 비트를 반전시킨다. |
| << | 피연산자의 비트 열을 왼쪽으로 이동시킨다. |
| >> | 피연산자의 비트 열을 오른쪽으로 이동시킨다. |

사실 +, -는 비트연산자로 볼 수는 없습니다. 다만 비트연산자를 활용할 때 -가 나오기 때문에 한번 배워봅시다.

## +연산의 이해

이는 십진법의 +와 -의 방식과 동일합니다.

5 + 1을 이진수로 표현해볼까요?

101 + 1이죠?

1이 더해지면서 윗자리로 1로써 옮겨지면서

110, 6이 됩니다.

이는 우리가 9 + 1 = 10이 되는 것과 동일합니다.

십진법은 10을 기수로 한 숫자표현법이며 더해서 10이 되면 왼쪽으로 옮겨지면서 +1이 되는 표현법입니다.

이진법은 2를 기수로 한 숫자표현법이기 때문에 더해서 2가 되면 왼쪽으로 옮겨지면서  표현법입니다.

## -연산의 이해

5 - 1을 이진수로 표현해볼까요?

101 - 1이죠?

그렇다면 100 = 4가 됩니다.

조금 더 질문해볼게요.

**Q.  10101 + 11(21 + 3)은 어떻게 될까요?**

**답 : 11000(24)**

**Q. 10101 - 11(21 - 3)은 어떻게 될까요?**

**답 : 10010(18)**

또한 아직 십진법 ~~를 이진법 ~~로 하는게 익숙하지 않다면 다음 사이트를 통해서 익히시면 됩니다. 이 사이트는 큰수를 넣지 않으면 16자리까지 표현되고 큰수를 넣으면 32자리까지 표현되는 것을 참고해주세요.

참고 : https://www.rapidtables.com/convert/number/decimal-to-binary.html

## 비트연산자 기초 &, |

자, 이번에는 &(AND)와 |(OR)입니다. &는 true & true = true (1 & 1 = 1)고 나머지는 모두 false를 반환하는 것입니다. (1은 true고, false는 0입니다. )

| 연산 | 결과 |
|---|---|
| 0 & 0 | 0 |
| 0 & 1 | 0 |
| 1 & 0 | 0 |
| 1 & 1 | 1 |

**Q. 1001 & 1000을 하면 어떻게 될까요?**

**답 : 1000**

왜냐하면 모두 1이 아닌 부분에 대해서는 0이 되기 때문입니다

|를 볼까요? 이는 둘 중 하나라도 true면 모두 true가 됩니다.

| 연산 | 결과 |
|---|---|
| 0 \| 0 | 0 |
| 0 \| 1 | 1 |
| 1 \| 0 | 1 |
| 1 \| 1 | 1 |

**Q. 1001 | 1000을 하면 어떻게 될까요?**

**답 : 1001**

왜냐하면 1| 0은 1이 되니까요.

**Q. 1001 | 0110을 하면 어떻게 될까요?**

**답 : 1111**

## 비트연산자 기초 <<, >>

왼쪽 시프트 연산자 << , 오른쪽 시프트 연산자 >> 에 대해서 알아보겠습니다.

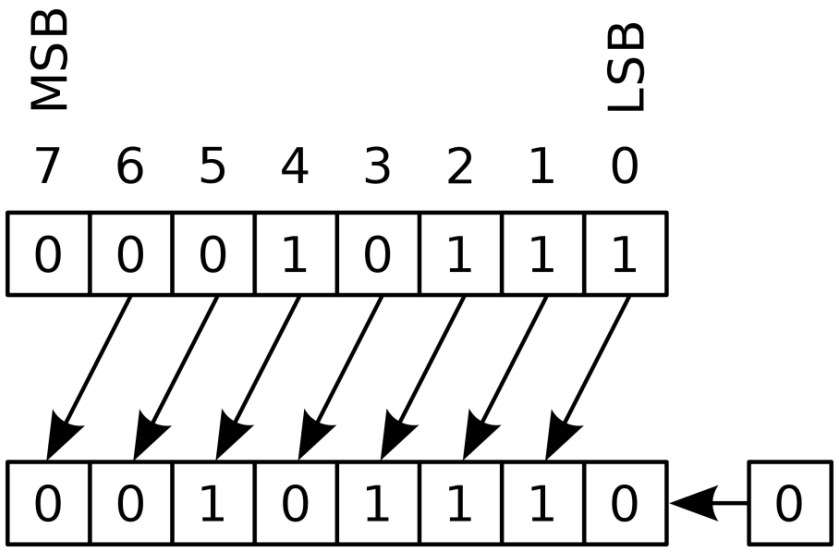
좀 더 자세히 설명하면, 사실 시프트 연산자는 logical shift, circular shift 등이 있지만 C++ 에서는 <<라고 했을 때 logical shift로 되니 logical shift 만 배우면 됩니다.

따라서 정확한 워딩은 left logical shift, right logical shift이며 앞으로는 그냥 왼쪽 시프트 연산자, 오른쪽 시프트 연산자라고 하겠습니다.

## 왼쪽 시프트 연산자 <<

a << b에서 a는 시프트할 값이고, b는 시프트할 비트의 수입니다. 이 연산이 수행되면 a의 이진수 표현에서 모든 비트가 왼쪽으로 b 비트만큼 이동하게 됩니다. 왼쪽으로 시프트된 후, 오른쪽에서 남은 빈 자리는 0으로 채워집니다.

예를 들어, a << 1은 a의 비트를 왼쪽으로 1칸 이동시키는 것입니다.




**Q. 111(7) << 1 한다면?**

**답 : 1110(14)**

**Q. 111(7) << 2를 한다면?**

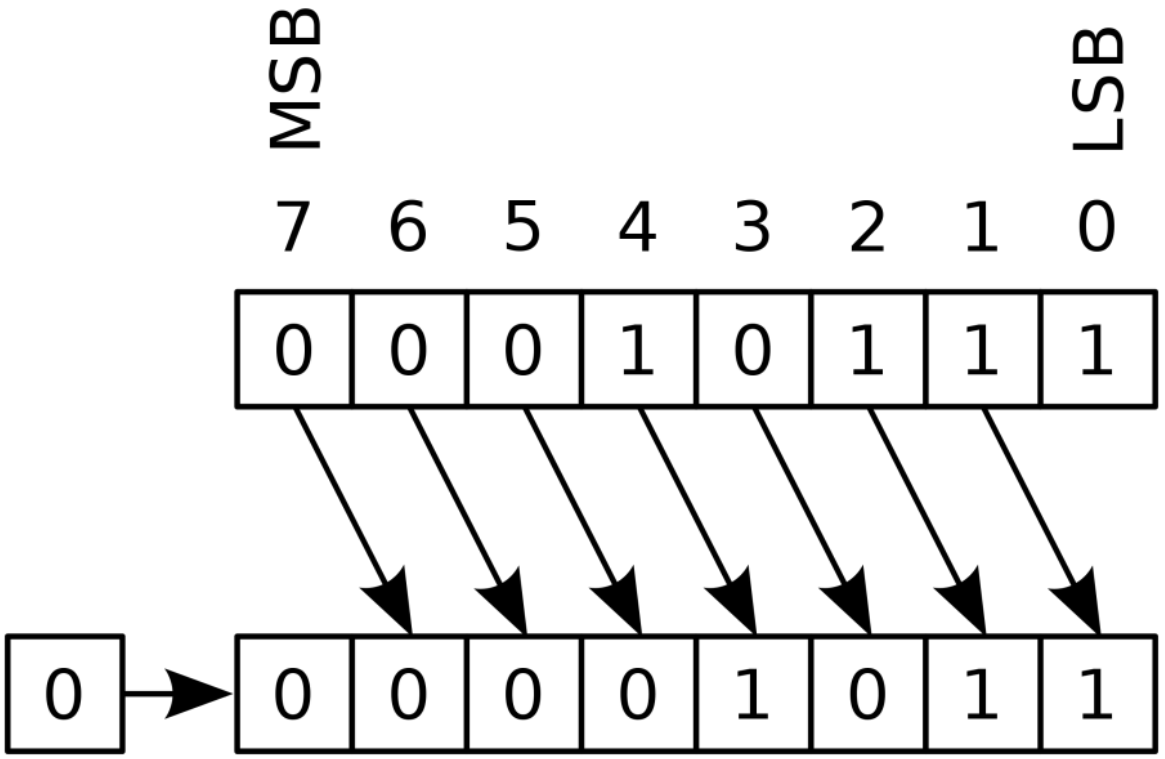
**답 : 11100(28)**

즉, a << b 는 a * (2 ^ b)와도 같은 의미를 가집니다. 비트마스킹할 때는 (1 << value)꼴로 많이 쓰이는데요. 예를 들어 1 << 3을 하면 1000(8)이 되고 1 << 4를 하면 10000(16)이 됩니다.

요약하면, `<<` 연산자는 값을 2의 n제곱만큼 곱하는 효과를 가지며, 비트를 왼쪽으로 이동시키고 오른쪽을 0으로 채우는 역할을 합니다.

## 오른쪽 시프트 연산자 >>

오른쪽 시프트 연산자 `>>`는 비트 연산에서 사용되며, 지정된 비트 수만큼 이진수의 각 비트를 오른쪽으로 이동시키는 역할을 합니다. a >> b에서 a는 시프트할 값이고, b는 시프트할 비트의 수입니다. 이 연산이 수행되면 a의 이진수 표현에서 모든 비트가 오른쪽으로 b 비트만큼 이동하게 됩니다.




**Q. 101(5) >> 1을 한다면?**

**답 : 010(2)**

**Q. 1111(15) >> 2을 한다면?**

**답 : 0011(3)**

a >> b 는 (int) a * (1 / 2) ^ b 와 같은 의미를 가집니다. 하지만 a >> b 는 (int) a * (1 / 2) ^ b라고 외운다면 힘드니 **비트를 오른쪽으로 한칸 뺀다!** 라고 생각하며 외우는 것을 추천드립니다.

## 비트연산자 기초 ^

XOR연산자, ^입니다. true ^ true = false, false ^ false = false 이며 나머지는 다 true를 반환하는 녀석입니다. (1은 true고, false는 0입니다. ) “난!! 같은 걸 싫어해!!” 하면서 false(0)을 반환한다고 생각하면 외우기 쉽습니다.

XOR(Exclusive OR) 연산자는 두 비트가 서로 다른 경우에만 1(또는 true)을 반환하고, 두 비트가 같으면 0(또는 false)을 반환하는 연산자입니다.

만약에 같으면 “난!! 같은 걸 싫어해!!” 하면서 false(0)을 반환하는 말괄량이 비트연산자라고 생각하면 됩니다.

| 연산 | 값 |
| --- | --- |
| 0 ^ 0 | 0 |
| 0 ^ 1 | 1 |

| | |
|---|---|
| 1 ^ 0 | 1 |
| 1 ^ 1 | 0 |

**Q. 1001(9)^ 1000(8) 을 한다면?**

**답 : 0001**

**Q. 1010(10) ^ 1000(8) 을 한다면?**

**답 : 0010**

## 비트연산자 기초 ~

~ 입니다. 1의 보수연산자이며 이는 해당 수의 모든 비트를 반전하는 연산자입니다.

~value = -(value + 1)이라는 특징을 갖습니다.

| 연산 | 값 |
|---|---|
| ~ 0 | 1 |
| ~ 1 | 0 |

~011 하면 어떻게 될까요?

사실 011은

00000000,00000000,00000000,00000011이죠?

여기서 비트를 뒤집으면 어떻게 될까요?

11111111 , 11111111 , 11111111 , 11111100이 됩니다.

이게 비트연산자를 통해서 음수가 나오니 음수에 대해 조금 설명해보겠습니다.

우리가 음수를 표현할 때는 2의 보수법을 이용합니다.

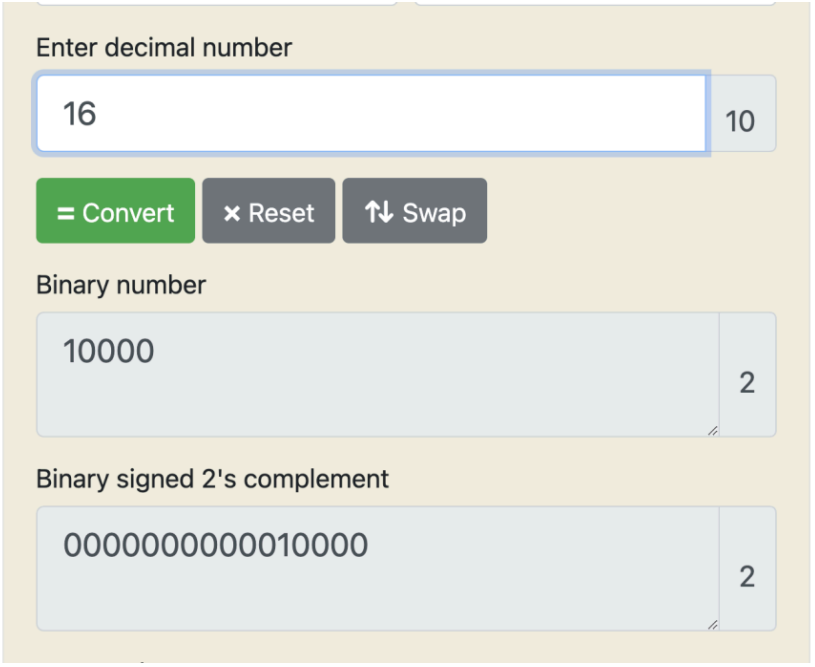
해당 양수의 모든 비트를 반전한 수에 1을 더하여 음수를 표현하는 방법인데요.

이 사이트는 큰수를 넣지 않으면 16자리까지 표현되고 큰수를 넣으면 32자리까지 표현되는 것을 참고해주세요.

[참고링크] : https://www.rapidtables.com/convert/number/decimal-to-binary.html

이 사이트와 함께 설명해보겠습니다.

16은 뭐죠? 00000000,00000000,00000000,00010000 이죠?




여기서 ~를 통해 비트를 뒤집으면 무엇이 될까요?

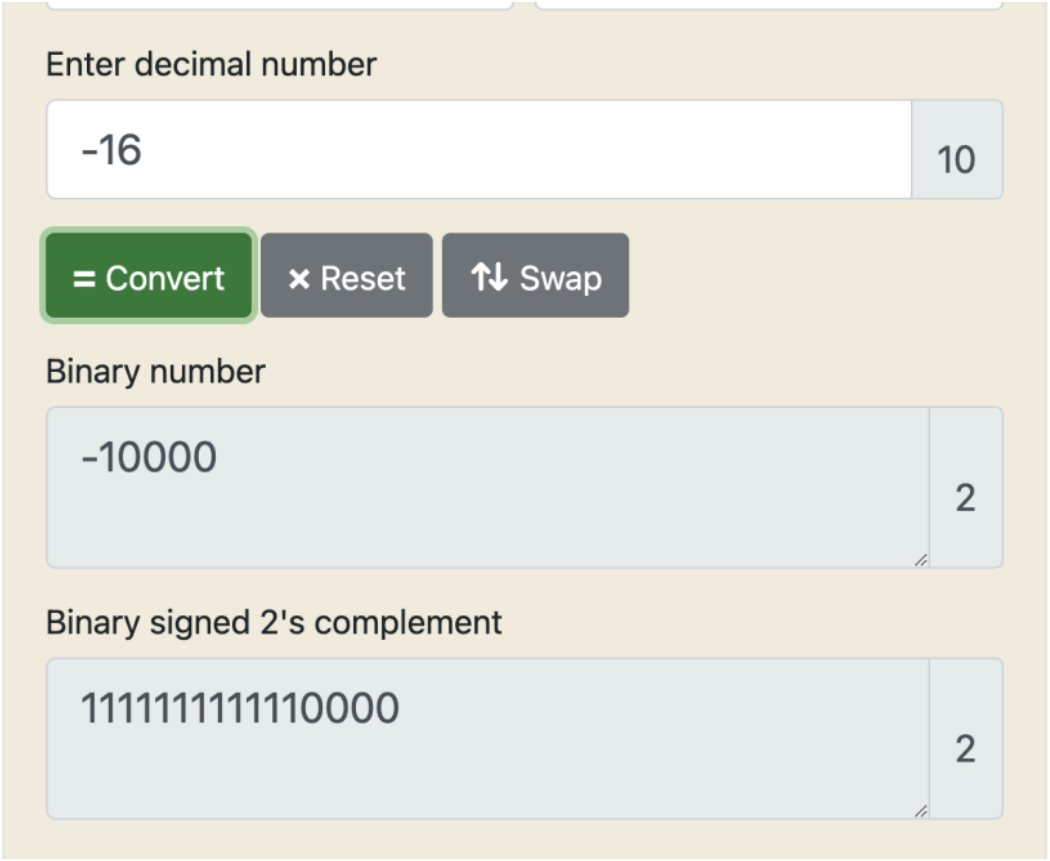
~00000000,00000000,00000000,00010000

=

11111111 , 11111111 , 11111111 , 11101111

이 됩니다.

자 여기서 +1을 해서 음수를 표현하는 것이 지금의 음수 표현법입니다.




즉, -value= ~value + 1이기 때문에

~ value = -(value + 1)이 됩니다.

## 비트연산자 활용법

사실 우리가 이제는 비트마스킹을 할 때 쓰이는 비트연산자 활용법에 대해서 알아보도록 할건데요.

아래의 이것들을 힘들지만 다 외워주셔야 합니다.

| 로직 | 코드 |
| --- | --- |
| idx번째 비트끄기 | S &= ~(1 << idx) |

| | |
|---|---|
| idx번째 비트 XOR 연산 | S ^= (1 << idx) |
| 최하위 켜져있는 비트 찾기 | idx = (S & -S) |
| 크기가 n인 집합의 모든 비트를 켜기 | (1 << n) - 1 |
| idx번째 비트를 켜기 | S \|= (1 << idx) |
| idx번째 비트가 켜져 있는지 확인하기 | if(S & (1 << idx)) |

## 1. idx번째 비트끄기 : S &= ~(1 << idx)

S = 10010(18)에서 저는 10010 이부분의 비트를 끄고 싶어요.

즉 1번째 비트죠?

참고로 1은 비트가 켜졌다. 0은 비트가 꺼진 겁니다.

어떻게 해야할까요?

먼저 000010을 만듭니다.

1 << 1을 하면 만들 수 있겠죠?

그 다음 비트를 뒤집어 줍니다.

111101 이런게 완성이 됩니다. (왼쪽에 수많은 1이 있는 것은 생략합니다.)

여기서

S = 10010 과 & 연산자를 하면 어떻게 될까요?

10010

&

111101을 하면?

10000이 되죠? 이렇게 해당 비트를 끌 수 있습니다.

**정답 코드:**

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int S = 18;
    int idx = 1;
    S &= ~(1 << idx);
    cout << S << '\n'; // 16
    return 0;
}
```

## 2. idx번째 비트 XOR 연산 : S ^= (1 << idx)

자, 이번에는 해당 idx번째의 비트에 XOR연산을 할겁니다.

그렇게 해서 무엇을 우리는 이룰것이냐??!!

해당 비트가 0이라면 1, 1이라면 0으로 만드는 걸 해볼껀데요.

S = 10010(18)에서

1번째 비트를 0으로 만들어볼게요.

(1 << 1)을 통해 00010을 만듭니다.

그리고

10010과 00010을 XOR연산자를 하면?

자 왼쪽에서부터

1 ^ 0 = 1

0 ^ 0 = 0

0 ^ 0 = 0

1 ^ 1 = 0

0 ^ 0 = 0

이렇게 되서

해당 부분만 바뀌어 10000이 되는 걸 볼 수 있죠?

자 그러면 이번에는 맨오른쪽 0을 1로 바꿔볼게요.

10010에서

0번째 비트는 0이죠?

(1 << 0)을 통해 00001을 만들어줍니다.

그리고 이를 ^ 연산자를 해줍니다.

1 ^ 0 = 1

0 ^ 0 = 0

0 ^ 0 = 0

1 ^ 0 = 1

0 ^ 1 = 1

짠, 10011이 되는 것을 볼 수 있죠?

**정답코드:**

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int S = 18;
    int idx = 1;
    S ^= (1 << idx);
    cout << S << '\n'; // 16
    return 0;
```

```
}
```

## 3. 최하위 켜져있는 비트 찾기 idx = (S & -S)

자, 일단 최하위 켜져있는 비트가 뭘까요?

10010에서 최하위 켜져있는 비트는 1번째의 비트죠?

10000에서 최하위 켜져있는 비트는 4번째 비트죠?

오른쪽에서부터 탐색을 해서 1을 찾아서 해당 인덱스를 찾으면 됩니다.

이 인덱스 idx는

idx = (S & -S)를 통해 찾을 수 있습니다.

S = 10010이라고 할게요.

-S는 뭐죠?

~S = -(S + 1)이기 때문에

01101 + 1= -S

01110 = -S죠?

자 그럼 여기서

01110

&

10010

을 하면?

00010이 반환이 되죠?

**정답코드:**

```
#include <bits/stdc++.h>
```

```cpp
using namespace std;
int main() {
    int S = 18;
    int idx = (S & -S);
    cout << idx << '\n'; // 2
    return 0;
}
```

## 4. 크기가 n인 집합의 모든 비트를 켜기 (1 << n) - 1

자, 우리가 이진수부터 시작해서 비트연산자, 비트마스킹을 배우는 이유는 이러한 비트들을 하나의 배열처럼 쓰기 위함입니다.

예를 들어

1111은

0번째, 1번째, 2번째, 3번째 요소가 "포함"되어있다. 로 놓을 수 있습니다.

배열 크기가 4인 집합 [1, 1, 1, 1] 이 아니라 숫자 15 딱 하나만으로 이를 우리는 "표현"할 수 있죠.

자 그러면 n이 4, 즉 크기가 4인 배열의 모든 요소를 포함한다를 어떻게 나타내죠?

[1, 1, 1, 1]로 나타낼 수도 있겠죠?

만약 여기서 1번째는 포함하지 않는다고 표현하면 [1, 1, 0, 1]이 될 거구요.

자 이를 비트연산자를 활용해서 표현해볼게요.

크기가 4라면

16 (1 << 4)가 되구요.

(1 << 4) - 1을 하게 되면

15가 나옵니다.

15는 앞서 설명한 1111이죠? 이렇게 크기가 4인 집합을 하나의 숫자로 나타낸다면 (1 << n) - 1을 하면 됩니다.

이는 이진수에서 10000에서 1을 빼면 01111이 되는 원리를 이용한 것입니다.

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n = 4;
    cout << (1 << n) - 1 << '\n'; // 15
    return 0;
}
```

## 5. idx번째 비트를 켜기 S |= (1 << idx)

10010(18)이 있습니다. 여기서 0번째 비트를 키면 어떻게 될까요?

10011(19)가 되죠?

이를

S |= (1 << idx)로 나타낼 수 있습니다.

1 << 0은

00001이며

이를

10010

|

00001을 하게 되면

왼쪽에서부터

1 | 0 = 1

0 | 0 = 0

0 | 0 = 0

1 | 0 = 1

0 | 1 = 1

이 되어서 10011이 됩니다.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int S = 18;
    int idx = 0;
    S |= (1 << idx);
    cout << S << '\n'; // 19
    return 0;
}
```

## 6. idx번째 비트가 켜져있는지 확인하기 if(S & (1 << idx))

자, 10010에서 3번째 비트는 켜져있을까요?

아니죠.

이를

10010

&

00100을 만들어서 확인할 수 있습니다.

왼쪽에서부터

1 & 0 = 0

0 & 0 = 0

0 & 1 = 0

1 & 0 = 0

0 & 0 = 0

즉, 결과값은 00000이 되기 때문에 0, 즉 false를 반환하게 됩니다.

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    int S = 18;
    int idx = 0;
    if(S & (1 << idx)){
        cout << "해당 idx : " << idx << "가 켜져있습니다.\n";
    }else{
        cout << "해당 idx : " << idx << "가 꺼져있습니다.\n";
    }
    return 0;
}
// 해당 idx : 0가 꺼져있습니다.
```

## 비트마스킹 : 경우의수, 매개변수

위의 것들을 외우고 이해했다면 이제 비트마스킹을 해야 할 시간입니다.

여러분. 어떠한 특정원소를 찾을 때 어느정도의 시간복잡도는 어떻게 될까요? 바로 자료구조 등등마다 다르겠죠?

- 배열의 어떤 요소를 찾을 때 선형적인 시간 : O(N)이 걸립니다.

- sorted array에서 이분탐색으로 찾을 때는 O(logN)이 걸립니다.

- 해싱테이블에서는 O(1)이 걸립니다.

자 그렇다면 불리언 배열에서 무언가를 찾는 등의 연산은 어떻게 될까요? 보통 불리언배열은 vector<bool>, bitset, set<int> 로 표현해서 처리하곤 합니다. 여기서 find메서드를 쓰는 등을 통해 연산을 하겠죠. 하지만 이러한 것들보다 비트연산을 쓰면 좀 더 가볍고 빠르게 연산할 수 있습니다.

불리언배열의 역할을 하는 "하나의 숫자"를 만들어서 "비트 연산자"를 통해 탐색, 수정 등의 작업을 하는 것을 비트마스킹이라고 합니다.

예를 들어 101은 1과 4의 합집합이며 이는 {0, 2}로 표현이 가능하죠?(0과 2번째 비트가 켜져있다!)

111은? {0, 1, 2}로 표현이 가능하죠?

즉, 불리언배열을 만들어서 {0, 1, 0, 1} 이렇게 만들지 않고 0101 이라는 하나의 수, 5 등을 이용해 하나의 숫자로 불리언 배열같은 역할을 할 수 있습니다. 또한 앞서 배운 비트연산자를 통해 해당 요소가 포함되어있는지 안되어있는지 등을 쉽게 알 수 있죠.

[참고] 왜 비트마스킹이라고 하나요?

컴퓨터의 저장 공간인 메모리에는 0이나 1이 저장됩니다. 이러한 메모리의 용량을 표현하는 단위는 여러가지가 있는데 그 중 최소 단위를 비트(Bit)라고 하며 이 비트는 0 또는 1이라는 값을 가집니다. 이를 기반으로 해당 비트를 켜거나(1로 만들거나) 끄거나(0으로 만들거나) 하는 "마스킹"하기 때문에 비트마스킹이라고 합니다. 비트는 이진수의 약어입니다. 비트나 이진수나 똑같은 개념이라고 보시면 됩니다. (BIT = Binary Digit)

## 비스마스킹을 이용한 경우의 수

비스마스킹은 경우의 수를 표현하는데도 잘 쓰입니다.

예를 들어 {사과, 딸기, 포도, 배}의 모든 경우의 수는 어떻게 될까요?

{사과}

{딸기, 사과}

..

이렇게 총 16가지수가 되겠죠? 사과를 포함하거나 포함하지 않거나 해서 각각의 요소는 2개의 상태값을 가지기 때문에 2^4 = 16이 될 겁니다.

```
#include <bits/stdc++.h>
using namespace std;
const int n = 4;
int main() {
      string a[n] = {"사과", "딸기", "포도", "배"};
      for(int i = 0; i < (1 << n); i++){
            string ret = "";
            for(int j = 0; j < n; j++){
                  if(i & (1 << j)){
                        ret += (a[j] + " ");
                  }
            }
            cout << ret << '\n';
      }
    return 0;
}
```

```
/*

사과
딸기
사과 딸기
포도
사과 포도
딸기 포도
사과 딸기 포도
배
사과 배
딸기 배
사과 딸기 배
포도 배
사과 포도 배
딸기 포도 배
사과 딸기 포도 배
*/
```

이렇게 모든 집합을 표현한 것을 볼 수 있습니다.

i는 0000, 0001, 0010을 상징합니다.

j는 0, 1, 2, 3을 기반으로 (1 << 0), (1 << 1) 등으로 해당 번째의 비트가 켜져있냐 켜져있지 않냐를 통해 집합을 확인합니다.

즉 이러한 것을 통해 4C1, 4C2 ... 이러한 조합들의 모든 경우의 수를 한번의 for문 만으로 표현이 가능한 셈이죠.

combi라는 함수를 통해 조합을 할 수 있다고 설명했는데요. 문제에서 4C1, 4C2 ... 여러가지 조합 등 모든 경우의 수를 기반으로 로직을 짜야 하는 경우 comi 함수를 여러번 써야하고 구현도 복잡해지는데 이 때 유용하게 됩니다.

## 비트마스킹을 이용한 매개변수 전달

사과라는 매개변수가 포함이 되어있고 이어서 사과 + 포도, 사과 + 배 이런식의 매개변수를 더하는 것을 구현하고 싶다면 이렇게 하면 됩니다.

```
#include <bits/stdc++.h>
using namespace std;
const int n = 4;
```

```
string a[4] = {"사과", "딸기", "포도", "배"};
void go(int num){
	string ret = "";
	for(int i = 0; i < 4; i++){
		if(num & (1 << i)) ret += a[i] + " ";
	}
	cout << ret << '\n';
	return;
}
int main() {
	for(int i = 1; i < n; i++){
		go(1 | (1 << i));
	}
    return 0;
}
/*
사과 딸기
사과 포도
사과 배
*/
```

## 비트마스킹의 한계

이처럼 << 또는 & 등 비트연산자 등으로 불리언배열의 역할을 하는 비트마스킹을 알아보았습니다.

하지만 비트마스킹은 한계가 있습니다. 바로 31까지 가능합니다. int형 숫자의 범위까지인 셈입니다. long long은 어떠냐 하는 의견도 있는데 보통 2^ 30승정도만 해도 시간복잡도가 10억이 되기 때문에 그 이후의 경우의 수를 센다는 것 자체가 이미 시간복잡도를 많이 초과하기 때문에 보통은 31 이하까지의 경우의 수만을 비트마스킹으로 해결할 수 있다고 보면 됩니다.

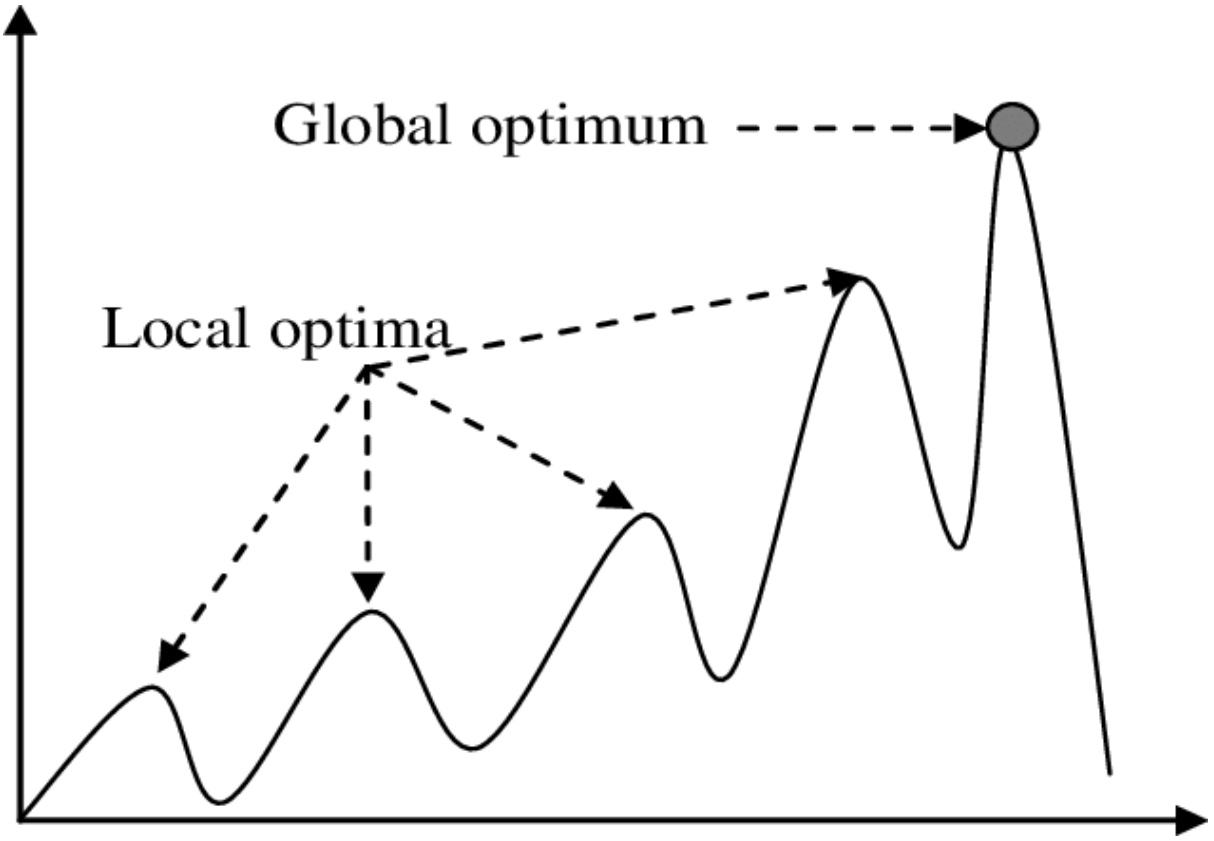
# 5주차 개념

이번 주차에는 그리디, 라인스위핑, 투포인터를 알아보도록 하겠습니다.

# 그리디

그리디 알고리즘(Greedy Algorithm)은 문제를 해결할 때 각 단계에서 그 순간에 최선이라고 생각되는 선택을 하는 방식으로, 이 과정에서 얻은 선택들이 모여 전체 문제에 대한 최적의 해를 구하는 알고리즘입니다.

즉, 그리디 알고리즘은 지역적 최적해(Local Optimal Solution)를 찾는 과정에서, 이를 모아 전역적 최적해(Global Optimal Solution)를 구하는 방식입니다. 다만, 모든 경우에 전역 최적해를 보장하지는 않습니다.




## 그리디를 구현하는 방법

그리디는 다음과 같이 구편합니다.

0. 무식하게 풀기 -> DP -> 그리디 순으로 가장 마지막 단계에서 그리디를 생각합니다.

1.“이 문제는 ~~기반으로 ~~ 를 하면 답을 찾을 수 있을것이다.”라고 어떠한 명제를 생각합니다.

2. 해당 명제를 기반으로 TC가 되는지 판단, 그 후 코드를 구축하고 TC 검증을 합니다.

3. 만약 반례가 있다면 다시 명제를 구축하고 2번 과정을 반복합니다.

## Q. 가장 지폐를 적게 내는 경우의 수

예를 들어 12100원짜리 물건을 구매할 때 내가 가지고 있는 돈이 다음과 같다고 해봅시다.

10000원 5개

5000원 5개

1000원 5개

100원 5개

라고 했을 때 가장 지폐를 적게 내는 (동전도 지폐라고 취급) 경우의 수는 무엇일까요?

바로 10000원 1개. 1000원 2개. 100원 1개.를 내는 것입니다. 이렇게 우리는 가장 큰 것부터 내면 최적해라는 것을 경험상 압니다.

이걸 코드로 나타내면 돈을 내림차순 정렬하고 순차적으로 뽑아내면 되는 것이죠.

```
#include <bits/stdc++.h>
using namespace std;
int ret, totalAmount = 12100;
int main() {
    // 각 화폐 단위와 개수를 저장한 벡터
    vector<pair<int, int>> currency = {{10000, 5}, {5000, 5}, {1000, 5},
{100, 5}};

    // 내림차순으로 정렬
    sort(currency.rbegin(), currency.rend());
    for (auto &c : currency) {
        while (totalAmount >= c.first) {
            totalAmount -= c.first;
            c.second--;
            ret++;
        }
    }
    if(totalAmount == 0) cout << ret << '\n';
    else cout << "불가능합니다. \n";
    return 0;
```

```
}
```

이렇듯 어떠한 명제를 우리의 경험 또는 지식으로부터 만들고 그걸 기반으로 답을 만들어내는 것이 그리디입니다.

## 그리디의 조건

그리디 알고리즘을 실행하려면 다음과 같은 조건이 충족되어야 합니다.

### 1. 최적부분구조

최적 부분 구조(Optimal Substructure)는 문제를 작은 부분 문제로 나눌 수 있으며, 그 부분 문제들의 최적해가 모여서 전체 문제의 최적 해를 이룰 수 있는 구조를 말합니다.

아까 예시로 든 문제를 단계별로 나눠보면 다음과 같습니다.

1. 가장 큰 화폐 단위부터 선택하여 부분 문제로 나눕니다.
2. 선택된 화폐를 사용하고 남은 금액을 다음 단계의 문제로 넘깁니다.
3. 이를 남은 금액이 0원이 될 때까지 위 과정을 반복하여 전체문제에 대한 최적해를 찾습니다.

단계별 과정은 다음과 같습니다.

총 금액: 12100원

사용할 수 있는 화폐: 10000원 5개, 5000원 5개, 1000원 5개, 100원 5개

**[단계 1]**

| 현재 금액: 12100원 |

| 선택 화폐: 10000원 |

| 남은 금액: 2100원 |

**[단계 2]**

| 현재 금액: 2100원 |

| 선택 화폐: 1000원 |

| 남은 금액: 1100원 |

**[단계 3]**

| 현재 금액: 1100원 |

| 선택 화폐: 1000원 |

| 남은 금액: 100원 |

**[단계 4]**

| 현재 금액: 100원 |

| 선택 화폐: 100원 |

| 남은 금액: 0원 |

이렇게 4단계를 거쳐서 최종적으로  10000원 1개, 1000원 2개, 100원 1개 라는 화폐 라는 답을 얻게 된 것이죠.

이렇게 작은 단계들로 나누고 그 단계에서의 최적해가 모여 전체문제의 최적해를 만들어낼 수 있는 구조를 말합니다.

## 2. 탐욕 선택속성

탐욕선택속성이란 각 단계에서의 최적 선택이 전체 문제에 대해서도 최적의 해를 보장한다는 것을 의미합니다.

즉, 그 순간 가장 최적인 선택을 하는 것이 전체 문제를 푸는 데 있어서도 최적의 결과를 낳는다는 것을 말합니다. 앞서 설명한 12100원 문제에서 만약 10000원짜리가 아니라 8000원짜리 5개가 있다고 가정한다면 “각 정점에서 가장 큰 지폐를 쓰는 것"은 올바른 명제가 아니게 됩니다. 8000원짜리 1개, 1000원짜리 4개, 100원짜리 1개. 총 6개를 쓰게 되는데요. 이경우 5000원짜리 2개, 1000원짜리 2개, 100원짜리 1개를 썼을 때의 5개보다 더 많은 지폐를 쓰게 되는 반례가 있기 때문입니다.

즉, 이런 탐욕선택속성이 충족되는 명제를 만들어야 합니다. 그렇다면 이 탐욕선택속성이 충족되는지 어떻게 알 수 있을까요?

정확하게 하는 방법은 바로 증명하는 것입니다. 보통은 귀류법과 같은 간접 증명 방법을 통해 증명됩니다.

## 직접증명과 간접증명

직접 증명은 증명하려는 명제가 참이라는 것을 전제로 논리적인 순서에 따라 증명하는 방법입니다.

예를 들어, **짝수의 합이 짝수임**을 직접 증명하는 과정은 다음과 같습니다:

- 가정: 두 짝수 a=2k와 b= 2m이 있다고 가정하자. k와 m은 정수 이다.
- 두 수의 합을 계산하면, a+b=2k+2m = 2(k + m)이며 이는 2로 나누어지므로 짝수입니다. 따라서 짝수의 합은 짝수임이 증명됩니다.

하지만 이렇게 증명하려는 명제가 참이라고 가정했을 때 증명하기 어려운 방법은 간접증명을 씁니다.

간접 증명방법 중 하나인 귀류법은 어떤 명제의 참을 직접 증명하는 대신, 그 명제가 거짓이라고 가정하고 그 가정이 모순을 일으킴을 보여줍니다. 이로써 그 명제가 거짓일 수 없으므로, 그 명제가 참임을 증명하는 방식입니다.

예를 들어, 어떤 명제 P가 참임을 증명하기 위해 P가 거짓이라고 가정했을 때 논리적 모순이 발생하면 P가 참일 수 밖에 없다고 결론짓는 것을 말합니다.

## √2가 무리수임을 증명해보기

예를 들어 √2가 무리수임을 증명해보겠습니다.

**√2는무리수입니다.**

무리수는 순환하지 않는 무한소수입니다. 이를 어떻게 알죠? 우리가 1억번을 1.41... 이런식으로 세어봐야 할까요? 힘들죠? 이럴 때 쓰는 것이 바로 간접증명입니다.

명제를 역으로 바꿔보는 것입니다.

**√2는유리수이다.**

라고 말이죠.

유리수는 뭐죠? 유리수는 분수로 표현되는 숫자를 말합니다. 예를 들어 n = a / b 꼴로 표현할 수 있는 것이 유리수인데 이 때 a와 b는 서로소입니다. 즉 약분이 되지 않는 기약분수를 뜻하죠.

자 그렇다면

**√2 = a/b**

가 됩니다.

여기서 제곱을 해봅시다.

**2b^2 = a^2**

따라서 a^2 는 짝수이며 **a = 2k**로 표현할 수 있습니다.

**2b^2 = 4k^2**

따라서 b도 짝수가 됩니다.

이렇게 되면 a와 b 둘 다 짝수이기 때문에 초기에 가정했던 a와 b는 서로소라는 명제가 깨지기 때문에 **√2**는 무리수라는 명제가 증명이 됩니다.

## 현실적인 방법

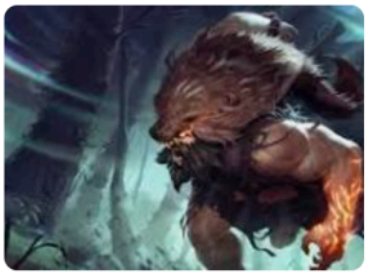
원래 그리디 알고리즘을 정석적으로 하기에는"~~를 위해서 탐욕적으로 ~~를 해야 한다." 라는 명제를 앞의 설명처럼 증명을 하고 해당 명제가 맞는 것임을 확인하고 그 명제로 문제를 풀어야 합니다.

**하지만 이러한 것을 코딩테스트에서 하기에는 불가능합니다.**

어떤 명제를 생각한다음 -> 어떠한 상태값(idx, here 등..)에서 가장 최적의 로직은 무엇일까 생각하고 -> 코드로 구현 -> 해당 명제가 틀렸다면 -> 빠르게 변경해서 다시 푸는 것입니다.

저는 이것을 우디르급 태세전환이 필요하다고 말합니다.

**우디**르의 스킬과 플레이 스타일을 보면 쉴 틈 없이 항상 **태세**를 바꿔야 한다는 특성을 가지고 있다. 이렇게 이리저리 급변하는 **우디**르의 특성에 자신의 태도를 바로 바꾸는 사람을 가리켜 각종 온라인 커뮤니티에서는 **'우디르급 태세전환'**라고 한다. Dec 2, 2019



[참고링크] : https://youtu.be/P44wC6LftMs?si=bfQVUdC4uMnct1nE / 1:30부터

마치 우디르처럼 자신의 명제나 코드가 틀리더라도 빠르게 다시 다른 방법으로 시도해보는 것이죠.

그리고 앞에서 설명했듯이 그리디는 틀릴 가능성이 높은 알고리즘입니다. 그래서 안전하게 다음과 같은 플로우로 그리디를 시도해야 합니다.

무식하게 시도하기 -> DP -> 그리디

*DP는 7주차 때 배웁니다.

## 팁

그리디 알고리즘을 쉽게 구축하는 방법 중 하나는 자주 나오는 풀이를 외우고 그걸 기반으로 푸는 것입니다. 보통 그리디는 정렬, 우선순위 큐 를 사용하는 2가지의 방법이 주로 나옵니다. 이 방법에 숙달을 먼저 하고 이 방법을 안될 경우에는 각 상태에서 최선이 되는 선택이 되는 로직을 생각해내면서 풀면 됩니다.

## Q. 큰돌은 욕심많은 도서관 사서야!!!

큰돌은 도서관의 사서입니다. 그의 도서관에는 학생들이 자유롭게 사용할 수 있는 1개의 좌석이 있습니다. 학생들은 좌석에 앉아서 공부를 하기 위해 특정한 시간에 도착하여 특정한 시간에 떠납니다. 큰돌은 학생들의 스케줄을 조정해 가능한 한 많은 학생들이 좌석을 사용할 수 있도록 배정할 때의 최대 학생 수를 구하세요.

**입력**

첫 줄에 학생의 수 N이 주어집니다. (1 ≤ N ≤ 100,000)

다음 N개의 줄에는 각 학생의 도착 시간과 떠나는 시간이 주어집니다. 도착 시간과 떠나는 시간은 정수로 주어지며, 도착 시간은 떠나는 시간보다 항상 작거나 같습니다.

**출력**

최대 몇 명의 학생들이 도서관 좌석을 사용할 수 있는지 출력하세요.

**예제입력 1**

5

1 4

2 5

3 6

5 7

8 9

**예제 출력 1**

3

**예제 입력 2**

7

1 4

2 3

3 5

4 6

5 7

7 8

8 9

**예제 출력 2**

5

**해설**

이 문제를 어떻게 풀어야 할까요?

먼저 그림을 그려야 합니다.(시작과 끝을 기반으로 도식화를 해주세요.)

그렇게 해서 문제분석을 하는 것이죠.

그리고 제가 처음에는 무식하게 풀라고 말했죠? 완전탐색(Brute Force)으로 문제를 해결하자면, 시간 복잡도는 O(2^N)이 됩니다.

학생을 포함하거나 포함하지 않는 경우가 각각 존재하기 때문에, 가능한 모든 조합을 생각해 볼 수 있습니다. 각 학생을 선택하거나 선택하지 않는 두 가지 경우가 존재하고, 이런 경우의 수는 2^N이 됩니다. 따라서, 모든 가능한 조합을 조사하여 겹치지 않는 조합 중 가장 많은 학생을 포함하는 조합을 찾는 방법입니다.

이런 알고리즘은 N이 작을 때는 사용 가능하지만, N이 클 경우에는 실행 시간이 매우 길어질 수 있습니다. 이 문제의 경우 최대 N의 크기가 10만입니다.

그렇다면 무식하게 풀기 -> DP -> 그리디

에서 DP라는 단계까지 온 것을 볼 수 있습니다.

사실, 이 문제는 DP로도 풀 수 있습니다.

하지만 아직 DP는 배우지 않았기 때문에 DP는 안된다고 가정해보겠습니다.

[참고] : 정답 DP 코드

```
#include <bits/stdc++.h>
```

```
using namespace std;


struct Student {
    int start, end;
};


int findPreviousStudent(const vector<Student>& students, int index) {
    int low = 0, high = index - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (students[mid].end <= students[index].start) {
            if (mid + 1 <= high && students[mid + 1].end <=
students[index].start)
                low = mid + 1;
            else
                return mid;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int maxstudentsDP(vector<Student>& students) {

    sort(students.begin(), students.end(), [](const Student& a, const
Student& b) {
        return (a.end == b.end) ? a.start < b.start : a.end < b.end;
    });

    int n = students.size();
    vector<int> dp(n, 0);


    dp[0] = 1;

    for (int i = 1; i < n; ++i) {
        dp[i] = dp[i - 1];

        int j = findPreviousStudent(students, i);

        if (j != -1) {
            dp[i] = max(dp[i], dp[j] + 1);
```

```
        } else {
            dp[i] = max(dp[i], 1);
        }
    }

    return dp[n - 1];
}

int main() {
    int N;
    cin >> N;
    vector<Student> students(N);

    for (int i = 0; i < N; ++i) {
        cin >> students[i].start >> students[i].end;
    }

    cout << maxstudentsDP(students);

    return 0;
}
```

그리디는 보통 정렬과 우선순위 큐를 쓴다고 했으니 그 부분을 먼저 생각해봅시다. 먼저 정렬을 생각한다고 했을 때 정렬 방식에 대한 다양한 경우의 수를 고려해봅시다.

- 시작 시간을 기준으로 오름차순 정렬
- 끝나는 시간을 기준으로 오름차순 정렬
- 끝나는 시간 - 시작 시간을 기반으로 오름차순 정렬

물론 이 외에도 내림차순 정렬을 고려할 수 있지만, 익숙한 것부터 시도를 하는게 좋습니다. 오름차순 정렬을 하고 왼쪽에서 오른쪽으로 탐색하는게 익숙하기 때문에 이부분부터 시도하는 것입니다.

나중에 오름차순이 안되면 -> 그 때 내림차순을 생각하면 되는 것이죠.

만약 시작 시간을 기준으로 오름차순 정렬하는 경우 예제입력2에서 올바른 답이 나오지 않습니다.

답은 5인데 4가 나오게 되는 것이죠. -> 그렇다면 -> 끝나는 시각을 기준으로 정렬해보는게 어떨까? 하는 것이죠.

즉, 우디르처럼 명제를 바꿔보는 것이죠.

정답코드:

```
#include <bits/stdc++.h>
using namespace std;
int from, to, n, idx = 0, ret = 1;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cin >> n;
    vector<pair<int, int>> v;
    for(int i = 0; i < n; i++){
        cin >> from >> to;
        v.push_back({to, from});
    }
    sort(v.begin(), v.end());
    from = v[0].second;
    to = v[0].first;
    for(int i = 1; i < n; i++){
        if(v[i].second < to) continue;
        from = v[i].second; to = v[i].first;ret++;
    }
    cout << ret << '\n';
    return 0;
}
```

핵심코드 설명

```
sort(v.begin(), v.end());  // 떠나는 시간을 기준으로 정렬
```

학생들을 떠나는 시간(to)을 기준으로 정렬합니다. 이렇게 하면 가장 빨리 자리를 비우는 학생을 우선적으로 선택할 수 있습니다. 즉, 먼저 끝나는 활동을 선택해 나가는 방식입니다.

```
for(int i = 1; i < n; i++){
// 현재 학생의 도착 시간이 이전 학생의 떠나는 시간보다 작으면 좌석 사용 불가
```

```
    if(v[i].second < to) continue;
    from = v[i].second;
    to = v[i].first;
    ret++;
}
```

이전 학생이 떠나는 시간(to)보다 늦게 도착하는 학생을 선택하여 좌석을 사용하도록 합니다. 이렇게 매번 좌석을 사용할 수 있는 학생을 선택할 때마다 카운터 ret를 증가시켜 최종적으로 가능한 학생 수를 구합니다.

## Q. 골동품 수집가 큰돌은 욕심쟁이야!!!

골동품 수집가인 큰돌은 세계 여러 곳을 여행하며 희귀한 골동품을 수집해 왔습니다. 이번에 큰돌은 거대한 골동품 박람회에 참석하기로 했습니다. 박람회에서는 다양한 골동품들이 전시되어 있으며, 각각의 골동품은 무게M[i]와 가치V[i]를 가지고 있습니다. 큰돌은 자신이 수집한 골동품들을 보관할 수 있는 고급 가방 K개를 가지고 있습니다. 각 가방은 최대 C[i] 무게까지 골동품을 담을 수 있으며, 가방 하나에는 최대 한 개의 골동품만 넣을 수 있습니다.

큰돌은 가능한 한 최대한 가치 있는 골동품들을 가방에 넣어 가져가고 싶습니다. 큰돌이 수집할 수 있는 골동품의 가치의 합의 최대값을 구하는 프로그램을 작성하세요.

**입력**

첫째 줄에 골동품의 수 N과 가방의 수 K가 주어집니다.

(1 ≤  N,K ≤ 300,000)

다음 N개의 줄에는 각 골동품의 정보(M[i], V[i])가 주어집니다.

(0 ≤M[i], V[i]≤1,000,000)

다음 K개의 줄에는 가방에 담을 수 있는 최대 무게 C[i]가 주어집니다.

(1 ≤ C[i] ≤100,000,000)

**출력**

큰돌이 수집할 수 있는 골동품 가치의 합의 최대값을 출력합니다.

**예제 입력 1**

4 2

1 100

2 200

11 400

10 500

10

11

**예제 출력 1**

900

**예제 입력 2**

7 4

3 30

4 35

5 40

6 60

10 100

7 90

8 85

9

5

4

15

**예제 출력2**

265

어떤 것부터 담아야 할까요?

1.가방의 크기를 내림차순으로 정렬하고 골동품의 가치가 높은 것부터 내림차순으로 담을까요?

2.아니면 가방의 크기를 오름차순으로 정렬하고 골동품의 가치가 작은 것부터 담을까요?

사실 어떤 명제가 정답인지는 모릅니다. 그저 하나하나 명제를 시도 -> 틀릴 경우 빠르게 다른 명제를 시도해봐야 합니다. 그리디는 그렇게 풀어야 합니다.

1번 명제의 경우 예제1의 반례가 있습니다.

2번 명제의 경우 예제1, 2의 반례가 있습니다.

자 그러면 어떻게 해야할까요? 먼저 경험상 효율적 = 가성비가 높다라는 것을 알 수 있습니다. 이 문제의 경우 가성비가 높은게 무엇일까요? **무게를 적게 담을 수 있는 가방 부터 보석을 넣기 시작하고 넣을 수 있는 보석 중 가치가 최대인 보석만 하나만 넣는게 아닐까요?**

### 정답코드

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
ll n, k, ret, temp1, temp;
int main(){
	ios_base::sync_with_stdio(false);cin.tie(NULL); cout.tie(NULL);

	cin >> n >> k;
	vector<pair<ll,ll>> v(n);
	vector<ll> vv(k);
	for(int i = 0; i < n; i++){
		cin >> v[i].first >> v[i].second;
	}
	for(int i = 0; i < k; i++) cin >> vv[i];
	sort(v.begin(), v.end());
	sort(vv.begin(), vv.end());
	priority_queue<ll> pq;

	int j = 0;
	for(int i = 0; i < k; i++){
		while(j < n && v[j].first <= vv[i]) pq.push(v[j++].second);
		if(pq.size()){
			ret += pq.top(); pq.pop();
		}
	}
	cout << ret << "\n";
	return 0;
}
```

여기서 미리 오름차순 정렬 기반으로 풀면 좋은게 그 이전가방에서 담을 수 있는 골동품을 지금 이 가방에도 담을 수 있다라는 것은 자명하게 됩니다. 이 때문에 정렬하고 오른쪽으로 탐색했을 때 지금의 가방이 그 이전 가방보다 크거나 같은 것은 자명하기 때문에 매번 보석과 가방의 크기를 비교하는 로직자체가 사라지게 됩니다.

### 핵심코드설명

```
for(int i = 0; i < k; i++){
    while(j < n && v[j].first <= vv[i]) pq.push(v[j++].second);
```

```
        if(pq.size()){
        ret += pq.top(); pq.pop();
    }
}
```

1. for(int i = 0; i < k; i++): 각 가방에 대해서 반복문을 실행합니다. 가방의 크기 순서대로 처리합니다.
2. while(j < n && v[j].first <= vv[i]): 현재 가방 vv[i]의 무게 제한보다 작거나 같은 무게의 골동품을 모두 우선순위 큐에 넣습니다. 즉, 이 가방에 넣을 수 있는 모든 골동품을 큐에 추가하는 과정입니다.
3. pq.push(v[j++].second): 현재 선택된 골동품의 가치를 우선순위 큐에 넣습니다.
4. if(pq.size()): 만약 우선순위 큐에 선택 가능한 골동품이 있으면, 그 중에서 가장 가치가 높은 골동품을 선택하여 ret에 더합니다. 그 후 우선순위 큐에서 그 골동품을 제거합니다.

## Q. 큰돌 교수님의 과제는 너무 어려워!!!

큰돌 교수님은 데이터 분석 과목을 담당하고 있으며, 학생들의 성적 분석을 과제로 내주었습니다. 교수님은 과제에서 최하위 성적을 받은 5명의 학생에게 추가적인 지도를 해주기로 결정했습니다. 이들 중 성적이 좋지 않은 5명을 선발하여 따로 지도를 해주려 합니다.

교수님을 돕기 위해 학생들의 최종 성적이 주어질 때, 성적이 좋지 않은 5명의 학생을 선택하여 성적이 낮은 순서대로 출력하는 프로그램을 작성하세요.

**입력**

첫째 줄에 학생의 수 N이 주어집니다.(6 ≤ N ≤ 50,000,000)

둘째 줄부터 N개의 줄에는 학생들의 성적이 무작위로 주어집니다. 성적은 최소 0점부터 최대 100점까지 0.01점 단위로 부여됩니다.

**출력**

성적이 좋지 않은 5명의 학생의 성적을 낮은 순서대로 각 줄마다 출력합니다. 동점자가 있을 경우에도 5명만 출력합니다.

**예제 입력 1**

8

95.6

74.3

88.2

53.1

92.7

67.9

88.2

45.5

**예제 출력 1**

45.5

53.1

67.9

74.3

88.2

**예제 입력 2**

10

34.5

56.7

78.9

34.5

12.3

90.1

23.4

67.8

12.3

89.9

**예제 출력 2**

12.3

12.3

23.4

34.5

34.5

이 문제는 그냥 정렬하면 되는 문제이지만, N의 크기가 너무크기 때문에 이걸 그냥 배열에 담고 정렬하기에는 공간복잡도도 너무 큰 것을 알 수 있습니다.  5000만은 너무 큽니다. 1주차 때 제가 보통은 3000만까지는 가능하다 라고 말씀을 드렸죠?

이 때는 우선순위큐를 써야 합니다.

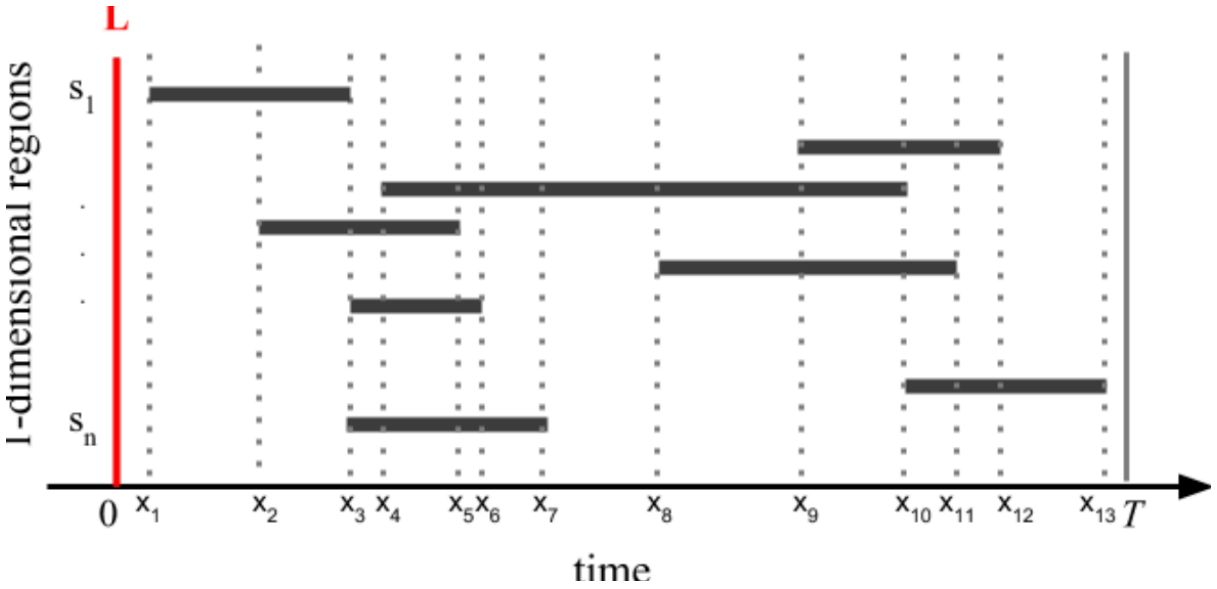
```
#include <bits/stdc++.h>
```

```
using namespace std;
int n;
double temp;
priority_queue<double> pq;
vector<double> v;
int main () {
    scanf("%d", &n);
    for(int i = 0; i < n; i++){
        scanf("%lf", &temp);
        if(pq.size() == 5){
            pq.push(temp); pq.pop();
        }else pq.push(temp);
    }
    while(pq.size()){
        v.push_back(pq.top()); pq.pop();
    }
    reverse(v.begin(), v.end());
    for(double i : v) printf("%.2lf\n", i);
    return 0;
}
```

## 라인스위핑

라인 스위핑(Line Sweeping) 알고리즘은 가상의 수직선을 왼쪽에서 오른쪽으로 이동시키면서 각 지점을 처리하는 알고리즘입니다. 이 수직선을 기반으로 문제에서 주어진 정점들을 탐색해나가면서 계싼을 수행합니다.

하나의 라인을 한번에 빗자루 쓸 듯이 탐색한다. 라고 해서 라인스위핑이라고 불립니다.




사실 이는 굉장히 고급 알고리즘에 속합니다. 교차점, 점과 점의 집합 찾기 등 기하문제를 풀 때 사용되는데 코딩테스트에는 그정도 기하수준은 나오지 않기 때문에 이부분은 신경 쓸 필요가 없으며 그저 구간과 관련된 문제에서 최대의 구간, 횟수 등을 찾는데 많이 쓰이는 알고리즘이다. 라고 보면 됩니다.

## Q. 큰돌이는 못말리는 화가야!!!

큰돌은 새로운 예술 프로젝트를 위해 도화지 위에 여러 선을 긋기로 했습니다. 그는 도화지의 특정 위치에서 다른 위치까지 여러 번 선을 그릴 수 있습니다. 하지만 선을 그을 때마다 이미 그어진 선 위에 다시 선을 그을 수 있으며, 이렇게 여러 번 그어진 부분은 한 번만 계산됩니다.

큰돌이 도화지에 그린 모든 선들의 총 길이를 구하는 프로그램을 작성하세요. 여러 번 그어진 선이 있더라도, 겹쳐진 부분은 한 번만 계산해야 합니다.

첫째 줄에 선을 그은 횟수 N(1 ≤ N ≤ 1,000,000)이 주어지고 그 다음 N개의 줄에는 선을 그을 때 선택한 두 점의 위치 a, b(-1,000,000,000 ≤ a ≤ b ≤ 1,000,000,000)가 주어진다.

**입력1:**

5

0 2

1 5

3 7

8 10

6 9

**출력1:**

10

**입력2:**

3

1 5

2 6

8 10

**출력2:**

7

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
typedef pair<int, int> P;
P L[1000004];
int n, from, to, l, r, ret;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> from >> to;
        L[i] = P(from, to);
    }
    sort(L, L + n);
    l = L[0].first; r = L[0].second;
    for(int i = 1; i < n; i++){
        if(r < L[i].first){
            ret += (r - l);
            l = L[i].first;
            r = L[i].second;
        }else if(L[i].first <= r && L[i].second >= r){
            r = L[i].second;
        }
    }
    ret += r - l;
    cout << ret << '\n';
}
```

# 투포인터

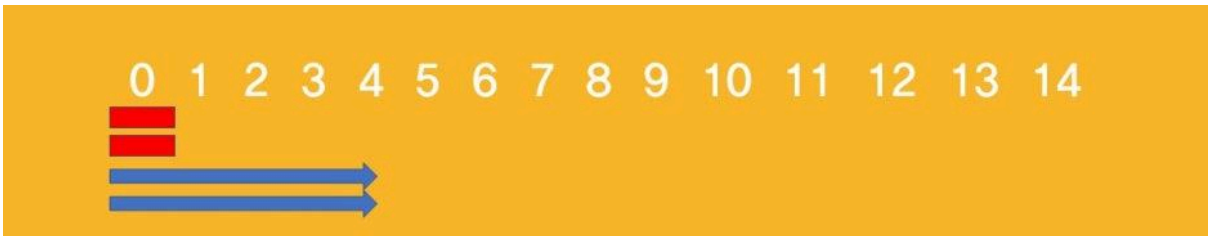
투포인터(Two Pointers) 알고리즘은 배열이나 리스트와 같은 선형 자료구조에서 두 개의 포인터를 사용하여 정렬된 배열에서 특정 조건을 만족하는 부분 배열 또는 원하는 값을 찾는 등의 문제를 풀 때 주로 사용하는 알고리즘입니다.

보통 **정렬된 배열에서 두 수의 합 찾기, 연속된 부분배열의 합 찾기**에 사용됩니다.

투포인터 알고리즘은 두 개의 포인터를 사용합니다

- 왼쪽 포인터 (Left Pointer): 배열의 시작 지점에서 시작하여 오른쪽으로 이동합니다.
- 오른쪽 포인터 (Right Pointer): 배열의 끝 지점에서 시작하여 왼쪽으로 이동합니다. 또는 왼쪽 포인터와 함께 배열의 시작 지점에서 시작하여 오른쪽으로 이동합니다.

앞의 그림처럼 시작지점과 끝점 이렇게 2개로 이루어진 투포인터를 활용해서 문제를 풀 수도 있고




앞의 그림처럼 시작점에서 같이 시작하는 투포인터를 이용해 문제를 풀 수도 있습니다.

## Q. 큰돌이는 숫자매니아야!!!

큰돌은 숫자와 관련된 문제를 푸는 것을 좋아하는 사람입니다. 이번에는 그가 만든 문제 중 하나를 풀어보려 합니다. 큰돌은 서로 다른 양의 정수로 이루어진 수열에서 특정 합을 이루는 숫자 쌍을 찾고자 합니다.큰돌이 만든 수열은 n개의 양의 정수로 이루어져 있으며 이 중에서 두 숫자의 합이 주어진 숫자 X가 되는 쌍의 개수를 구해야합니다.

**입력**

첫째 줄에 수열의 크기 n이 주어집니다. 그리고 그 다음 수열에 포함한 수가 주어집니다.(1 ≤ a[i] ≤ 100,000) 그리고 셋째줄에는 X가 주어집니다. (1 ≤ X ≤ 200,000)

**출력**

조건을 만족하는 쌍의 개수를 출력합니다.

**예제 입력 1**

8

1 9 2 8 4 7 3 6

10

**예제 출력 1**

4

**예제 입력 2**

5

4 1 3 2 5

6

**예제 출력 2**

2

```
#include<bits/stdc++.h>
using namespace std;
int n, ret, x;
int main(){
    cin >> n;
    vector<int> a(n);
    for(int i = 0; i < n; i++) cin >> a[i];
    cin >> x;
    sort(a.begin(), a.end());
    int l =  0, r = n - 1;
    while(l < r){
        if(a[l] + a[r] == x)r--, ret++;
        else if(a[l] + a[r] > x)r--;
        else if(a[l] + a[r] < x)l++;
    }
    cout << ret << "\n";
}
```

이는 시작점과 끝점에서 비교해보면서 푸는 것을 볼 수 있습니다.

만약 처음 X를 찾았을 때 어떻게 하는 것이 좋을까요?

당연히 L을 증가시키면 숫자가 커지겠죠? 따라서 R을 감소시켜야 더 작은 수를 만들고 그 다음의 경우의 수를 생각할 수 있습니다. 이러면서 가능한 모든 조합을 판단해나가는 것입니다.

# 6주차 개념

6주차에서는 이분탐색과 LIS, 최대증가부분수열에 대해 다룹니다.

# 이분탐색

이분탐색(Binary Search)은 정렬된 배열에서 탐색 범위를 절반씩 줄여나가는 방식으로 특정 값을 찾는 알고리즘입니다. 시간 복잡도는 O(log N)이며 이진탐색이라고도 불립니다.

문제에서 주어진 배열의 크기가 너무나도 큰 상태에서 어떤 값을 찾아야 하는 로직이 있을 때  해당 배열을 정렬하고 이분탐색을 쓰면 O(logN) 의 시간복잡도로 효율적으로 값을 찾을 수 있습니다.

## 이분탐색의 동작 원리

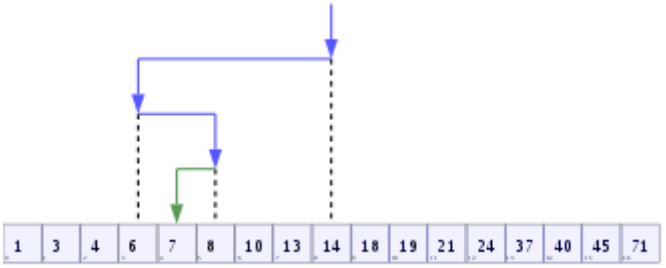
1.초기 설정: 먼저, 배열의 가장 왼쪽 인덱스 left와 가장 오른쪽 인덱스 right를 설정합니다.

2.중간값 계산: 현재 배열의 중간 인덱스 mid를 계산합니다. 중간 인덱스는 (left + right) / 2로 계산됩니다.

3.비교:

- 목표값과 중간값이 같을 때: 중간 인덱스 mid에 위치한 값이 목표값(target)과 일치하면, 탐색을 종료하고 mid를 반환합니다.
- 목표값이 중간값보다 작을 때: 목표값이 중간값보다 작다면, 오른쪽 절반은 탐색할 필요가 없으므로 right를 mid - 1로 설정하여 왼쪽 절반만 탐색합니다.
- 목표값이 중간값보다 클 때: 목표값이 중간값보다 크다면, 왼쪽 절반은 탐색할 필요가 없으므로 left를 mid + 1로 설정하여 오른쪽 절반만 탐색합니다.

4.반복: 이 과정을 목표값을 찾거나 left가 right보다 커질 때까지 반복합니다. 만약 목표값을 찾지 못하면 보통 -1을 반환합니다. -1 또는 배열의 범위에 해당하지 않는 숫자로 반환합니다.



앞의 그림처럼 왼쪽에 있는지 오른쪽에 있는지를 판단하며 탐색을 진행하는데 할 때마다 탐색범위가 절반으로 줄어들기 때문에 이 알고리즘의 시간복잡도는 O(logN)가 됩니다.

C++에서는 내장된 라이브러리 lower_bound 또는 upper_bound를 통해 구현할 수 있습니다.

**중요!! : 이분탐색은 정렬된 배열에서만 가능합니다.**

```cpp
#include <bits/stdc++.h>
using namespace std;

int binarySearch(const vector<int>& arr, int target) {
    int left = 0;
    int right = arr.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            return mid;  // target 값을 찾았을 때
        } else if (arr[mid] < target) {
            left = mid + 1;  // target이 중간 값보다 크므로 오른쪽 부분을 탐색
        } else {
            right = mid - 1;  // target이 중간 값보다 작으므로 왼쪽 부분을 탐색
        }
    }

    return -1;  // target 값을 찾지 못했을 때
}

int main() {
    vector<int> arr = {1, 3, 6, 9, 10, 21, 22, 30};
    int target = 6;
    int result = binarySearch(arr, target);

    if (result != -1) {
        cout << "Target found at index: " << result << "\n";
    } else {
        cout << "Target not found in the array\n";
    }

    return 0;
}
```

## 이분탐색의 시간복잡도 증명해보기

탐색 대상 배열의 크기가 N이라면, 첫 번째 단계에서 전체 배열을 절반으로 나누고, 그 절반 중에서 탐색을 이어갑니다. 매 단계마다 남아 있는 탐색 범위는 이전 단계의 절반이 되므로, N -> N/2 -> N/4 -> ... 이런 식으로 줄어들게 됩니다.

- 최악의 경우 남아 있는 탐색 범위가 1이 될 때까지 탐색을 반복하며 이를 수식으로 표현하면, k번째 단계에서 탐색 범위가 N/(2^k)가 됩니다.
- 이 값이 1보다 작거나 같아질 때 탐색이 종료되므로, N/(2^k) <= 1이 되는 k를 구합니다.
- 이를 수학적으로 풀어보면, N <= 2^k가 되고, 양변에 로그(log)를 취하면 log2(N) <= k가 됩니다.

즉, k는 log2(N)과 같습니다. 따라서 이분탐색에서 최악의 경우, 목표값을 찾는 데 걸리는 단계 수는 log2(N)입니다.

예를 들어 길이가 16인 배열에서 이분탐색을 진행한다고 가정하면 다음과 같습니다.

- 1단계: 전체 16개 중에서 중간 값을 비교 → 탐색 범위가 8개로 줄어듦.
- 2단계: 남은 8개 중에서 중간 값을 비교 → 탐색 범위가 4개로 줄어듦.
- 3단계: 남은 4개 중에서 중간 값을 비교 → 탐색 범위가 2개로 줄어듦.
- 4단계: 남은 2개 중에서 중간 값을 비교 → 탐색 범위가 1개로 줄어듦.

즉, 이 과정은 4단계(log2(16) = 4)에서 완료됩니다. 따라서 시간 복잡도는 O(log N)가 됩니다.

## Q. 큰돌의 비밀 수첩

큰돌은 놀라운 기억력을 자랑하는 인물로, 하루 동안 본 모든 정수들을 수첩에 기록하는 습관이 있다. 하지만 그의 친구 지민이는 큰돌의 기억력을 의심하며 시험해보기로 했다. 지민이는 큰돌이 하루 동안 본 정수들을 모두 기록한 '수첩1'을 보고, 그가 정말 모든 숫자를 기억하는지 확인하기 위해 몇 가지 질문을 던지기로 했다. 지민이는 "이 숫자를 기억해?"라는 질문을 몇 번 던졌고, 큰돌은 항상 "기억해!"라고 자신 있게 답했다. 이제, 지민이는 큰돌이 말한 숫자들이 정말 수첩1에 기록되어 있는지 확인하고자 한다.

여러분은 지민이를 도와 큰돌의 기억력을 검증하는 프로그램을 작성해보자.

**입력**

첫째 줄에 테스트케이스의 개수 T가 주어진다.

각 테스트케이스마다 다음과 같은 입력이 주어진다:

첫 번째 줄에 수첩1에 적힌 정수의 개수 N(1 ≤ N ≤ 1,000,000)이 주어진다.

두 번째 줄에 수첩1에 적힌 N개의 정수가 공백으로 구분되어 주어진다.

세 번째 줄에 수첩2에 적힌 정수의 개수 M(1 ≤ M ≤ 1,000,000)이 주어진다.

네 번째 줄에 수첩2에 적힌 M개의 정수가 공백으로 구분되어 주어진다.

모든 정수는 int 범위 내에 있다.

**출력**

각 테스트케이스마다 수첩2에 적힌 M개의 정수에 대해, 해당 숫자가 수첩1에 존재하면 큰돌이는 기억해!를, 존재하지 않으면 큰돌이는 까먹었어!를 각 숫자마다 한 줄씩 출력한다.

**예제 입력 1**

1

6

7 3 2 5 9 11

4

3 7 10 11

**예제 출력 1**

큰돌이는 기억해!

큰돌이는 기억해!

큰돌이는 까먹었어!

큰돌이는 기억해!

**정답코드:**

```
#include<bits/stdc++.h>
using namespace std;

int t, n, m, temp;
```

```
string check(int temp, vector<int> &v){
    int l = 0, r = v.size() - 1;
    int mid;
    while(l <= r){
        mid = (l + r) / 2;
        if(v[mid] > temp) r = mid - 1;
        else if(v[mid] == temp) return "큰돌이는 기억해!";
        else l = mid + 1;
    }
    return "큰돌이는 까먹었어!";
}

int main(){
    cin >> t;
    while(t--){
        vector<int> v;
        cin >> n;
        for(int i = 0; i < n; i++) {
            cin >> temp;
            v.push_back(temp);
        }
        sort(v.begin(), v.end());
        cin >> m;
        for(int i = 0; i < m; i++) {
            cin >> temp;
            cout << check(temp, v) << "\n";
        }
    }
    return 0;
}
```

## Q. 큰돌은 형광펜 질투쟁이야!!!

큰돌은 여러 색상의 형광펜을 모으는 걸 좋아합니다. 어느 날 큰돌은 N명의 친구들과 함께 형광펜을 나누기로 했습니다. 형광펜은 M가지 서로 다른 색상 중 하나를 가집니다. 각 친구는 한 가지 색상의 형광펜만 가질 수 있고, 여러 색상의 형광펜을 동시에 가지는 것은 불가능합니다.

큰돌은 질투심을 최소화하려고 합니다. 여기서 질투심은 가장 많은 형광펜을 가져간 친구가 가지고 있는 형광펜의 개수를 의미합니다. 질투심이 최소가 되도록 형광펜을 친구들에게 나누어 주는 프로그램을 작성하세요.

이 때 모든 형광펜을 다 나눠주어야 하며 형광펜을 받지 못하는 학생이 있어도 됩니다.

**입력**

첫 번째 줄에는 친구들의 수 N과 색상의 수 M이 주어집니다. (1 ≤ N ≤ 1,000,000,000, 1 ≤ M ≤ 300,000, M ≤ N)
다음 M개 줄에는 각 색상별 형광펜의 개수가 주어집니다. K번째 줄에 주어지는 숫자는 K번 색상의 형광펜의 갯수입니다. 이 값은 1 이상 10^9 이하입니다.

**출력**

질투심이 최소가 되도록 형광펜을 나누어 줄 때, 그 최소 질투심의 값을 출력하세요.

**예시**

**입력 1:**

5 2

8

5

**출력 1:**

3

**입력 2:**

7 5

100

23

40

50

21

**출력 2:**

50

**코드**

```
#include <bits/stdc++.h>
using namespace std;

int n, m, result = 987654321;
int arr[300000];

bool search(int estimate_val) {
	int ret = 0;
	for(int i=0; i<m; i++) {
		ret += arr[i] / estimate_val;
		if(arr[i] % estimate_val) ret++;
	}

	return ret <= n;
}

int main() {
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
	cin >> n >> m;

	int left = 1, right = 0, mid;

	for(int i=0; i<m; ++i) {
		cin >> arr[i];
		right = max(right, arr[i]);
	}

	while(left <= right) {
		mid = (left+right) / 2;

		if(search(mid)) {
```

```
                result = min(result, mid);
                right = mid - 1;
            }else {
                left = mid + 1;
            }
        }

        cout << result << '\n';

        return 0;
}
```

**코드설명**

이분탐색을 이용하면 **최적화 문제를 결정 문제**로 변환할 수 있습니다.

이 방법은 주로 어떤 값을 최대화하거나 최소화하는 최적화 문제를 해결할 때 사용됩니다. 여기서 최적화 문제와 결정문제는 다음과 같은 차이가 있습니다.

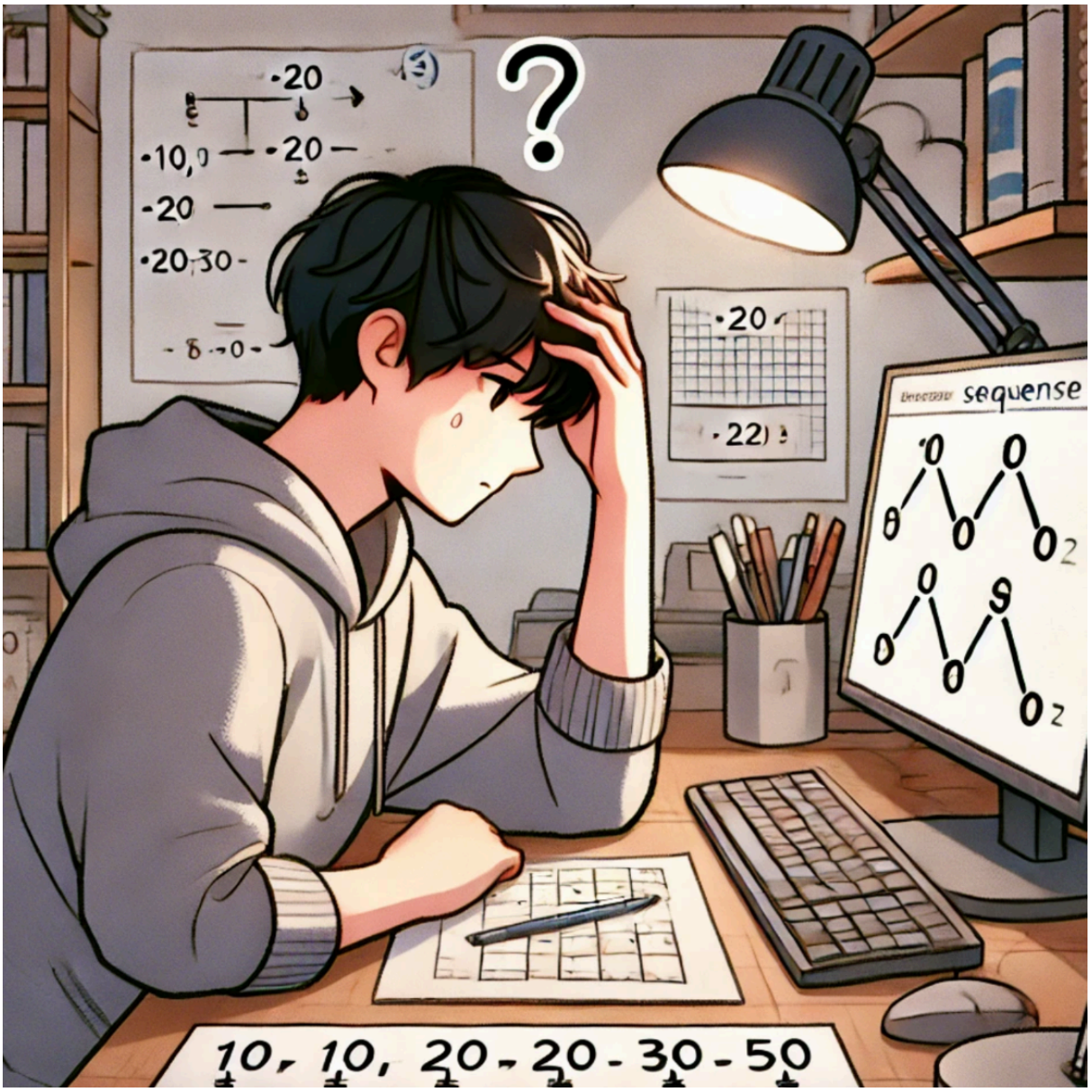
- **최적화 문제**: 어떤 값의 최대값이나 최소값을 찾는 문제입니다. 예를 들어, "가장 적은 질투심을 갖게 형광펜을 나누는 방법을 찾아라"는 최적화 문제입니다.
- **결정 문제**: 어떤 조건을 만족하는지 여부를 확인하는 문제입니다. (Yes or No) 예를 들어, "주어진 질투심 값으로 형광펜을 나눌 수 있는가?"라는 것은 결정 문제입니다.

이 문제에서 최적화문제인 "질투심을 최소화하려면 가장 많이 받은 친구의 형광펜 개수를 얼마나 줄일 수 있을까?"를 결정 문제인 "주어진 질투심으로 모든 학생에게 형광펜을 나눌 수 있는가?"로 변환해서 이분탐색으로 문제를 푸는 것이죠.

# 최대증가부분수열

최대증가부분수열(LIS, Longest Increasing Subsequence)은 문제는 주어진 배열에서 순서를 유지하면서 숫자가 오름차순으로 증가하는 가장 긴 부분 수열을 찾는 문제입니다. 예를 들어, 배열 [10, 20, 10, 30, 20, 50]에서 가장 긴 증가하는 부분 수열은 [10, 20, 30, 50]이며, 그 길이는 4입니다.

## Q. 큰돌의 LIS  O(N^2)




큰돌은 오늘도 머리를 부여잡고 수열 문제를 풀고 있습니다. 수열 A가 주어졌을 때, 가장 긴 증가하는 부분 수열을 구하는 문제를 해결하려고 합니다. 예를 들어, 수열 A = {10, 20, 10, 30, 20, 50}인 경우, 가장 긴 증가하는 부분 수열은 {10, 20, 30, 50}이고, 그 길이는 4입니다. 큰돌이를 도와 가장 긴 증가하는 부분 수열의 길이를 구하는 프로그램을 작성하세요.

**입력**

첫 번째 줄에 수열 A의 크기 N (1 ≤ N ≤ 1,000)이 주어집니다.

두 번째 줄에는 수열 A를 이루고 있는 N개의 A[i]가 주어집니다. (1 ≤ A[i] ≤ 1,000)

**출력**

첫 번째 줄에 수열 A의 가장 긴 증가하는 부분 수열의 길이를 출력하세요.

**예제 입력 1**

6

10 20 10 30 20 50

**예제 출력 1**

4

```
#include <cstdio>
#include <iostream>
#include <algorithm>
using namespace std;
int n, a[1001], cnt[1001], ret;
int main(){
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d", a + i);
    }
    for (int i = 0; i < n; i++) {
      int maxValue = 0;
      for (int j = 0; j < i; j++) {
        if (a[j] < a[i] && maxValue < cnt[j]) maxValue = cnt[j];
      }
      cnt[i] = maxValue +1;
      ret = max(ret,cnt[i]);
    }
    printf("%d\n", ret);
}
```

**코드설명**

cnt[]: 각 위치에서 끝나는 가장 긴 증가하는 부분 수열의 길이를 저장하는 배열. i번째 숫자까지 포함하는 증가하는 부분 수열의 최대길이를 저장합니다.

이중 for 문을 사용해 각 원소마다 가장 긴 증가하는 부분 수열을 찾습니다.

- a[j] < a[i]: 즉, 현재 수열에서 a[j]가 a[i]보다 작은 경우에만 증가하는 부분 수열이 될 수 있으므로 이 조건을 검사합니다.
- maxValue < cnt[j]: 현재까지 확인된 가장 긴 부분 수열의 길이(maxValue)를 갱신합니다.
- cnt[i] = maxValue + 1: a[i]를 포함하는 가장 긴 부분 수열의 길이를 cnt[i]에 저장합니다.
- ret = max(ret, cnt[i]): 현재까지 찾은 가장 긴 부분 수열의 길이 ret을 갱신합니다.

시간복잡도는 O(N^2) 이 됩니다.

## Q. 큰돌의 LIS trace  O(N^2)

여기서 만약 길이뿐만 아니라 해당 수열을 출력하라라는 지문이 있다면 어떻게 해야할까요?

```
#include<bits/stdc++.h>
using namespace std;
int n, a[1001], ret = 1, cnt[1001], idx;
int prev_list[1001];
vector<int> v;
void go(int idx){
    if(idx == -1) return;
    v.push_back(a[idx]);
    go(prev_list[idx]);
    return;
}
int main(){
    scanf("%d", &n);
    for(int i = 0; i < n; i++){
        scanf("%d", &a[i]);
    }
    fill(prev_list, prev_list + 1001, -1);
    fill(cnt, cnt + 1001, 1);
    for(int i = 0; i < n; i++){
        for(int j = 0; j < i; j++){
            if(a[j] < a[i] && cnt[i] < cnt[j] + 1){
                cnt[i] = cnt[j] + 1;
                prev_list[i] = j;
                if(ret < cnt[i]){
                    ret = cnt[i];
                    idx = i;
                }
```

```
            }
        }
    }
    printf("%d\n", ret);
    go(idx);
    for(int i = v.size() -1; i >= 0; i--){
        printf("%d ", v[i]);
    }
    return 0;
}
```

앞의 코드처럼 prev_list[i]에 담으면 됩니다.

## Q. 큰돌의 LIS O(NlogN)

그러면 O(N^2)의 시간복잡도가 아닌 좀 더 시간복잡도를 줄일 수 있는 알고리즘은 무엇이 있을까요?

```
#include <cstdio>
#include <algorithm>
using namespace std;

int n, lis[1001], len, num;
int main() {
      scanf("%d", &n);
      for (int i = 0; i < n; i++){
        scanf("%d", &num);
        auto lowerPos = lower_bound(lis, lis + len, num);
        if(*lowerPos == 0) len++;
        *lowerPos = num;
        for(int j = 0; j < n; j++){
            printf("%d ", lis[j]);
        }
        printf("\n");
      }
      printf("%d", len);

      return 0;
}
/*
6
10 20 10 30 25 50
*/
```

앞의 코드처럼 O(NlogN)으로 푸는 방법입니다. 이는 이분탐색으로 구현하거나 C++의 내장된 함수인 lower_bound()로 구현할 수 있습니다.

**코드설명:**

- lowerPos에 "현재의 증가수열길이"중에서 num보다 크거나 같은 값의 위치를 찾습니다.
- 만약에 len안에 없다면 *lowerPos는 0을 반환하게 됩니다. 왜냐면 초기화를 0으로 했기 때문입니다. 여기서 *lowerPos에 해당값을 넣어서 배열을 변화시키고 len을 증가시킵니다.
- 만약에 len안에 있다면 *lowerPos는 크거나 같은 값위치를 반환하고 그 값은 이제 좀 더 작거나 같은 값으로 변화 됩니다. 만약에 num이 2라면 {1, 3} 은 {1, 2}이 되게 됩니다. 2 이상의 위치는 3 -> 2로 재할당이 됩니다.

디버깅 코드를 추가할 경우 다음과 같이 lis 배열이 쌓이게 됩니다.

**입력:**

6

10 20 10 30 20 50

**lis배열:**

10 0 0 0 0 0

10 20 0 0 0 0

10 20 0 0 0 0

10 20 30 0 0 0

10 20 30 0 0 0

10 20 30 50 0 0

하지만 이 코드를 사용하게 된다면 trace를 하지 못합니다. 즉, 원본배열을 출력하지는 못합니다. 이는 원본배열이 수정되기 때문인데요. 예를 들어  10 20 30 12 라는 수열이 있을 때 이 코드의 경우 lis배열이 10 12 30 으로 설정이 되기 때문에 원래의 배열인 10 20 30 이 없어지기 때문에 그렇습니다.

# 7주차 개념

동적 계획법(Dynamic Programming, DP)은 문제를 해결하기 위해 문제를 작은 하위 문제로 나누고, 그 결과를 저장하여 동일한 하위 문제를 반복해서 풀지 않도록 하는 알고리즘 기법입니다.

먼저 DP의 조건부터 시작해 자세히 알아보겠습니다.

## DP의 조건

DP는 사실 다음과 같은 4가지의 조건이 충족되어야 쓸 수 있는 알고리즘입니다.

1. 참조투명성을 가져야 합니다.

입력을 제외한 외적요소에 결과값이 영향을 미치지 않고 동일한 입력에 대해 동일한 출력을 가지는 것을 의미합니다.

2. Optimal Substructure (최적 부분 구조)

문제를 해결할 때 하위 문제들을 해결한 결과를 이용해 전체 문제의 최적 해답을 구성할 수 있는 구조를 가지는 것을 말합니다. 예를 들어 피보나치 수열을 구할 때 $F(n)=F(n-1)+F(n-2)$와 같이 작은 하위 문제들을 이용해 문제를 해결할 수 있는 경우입니다.

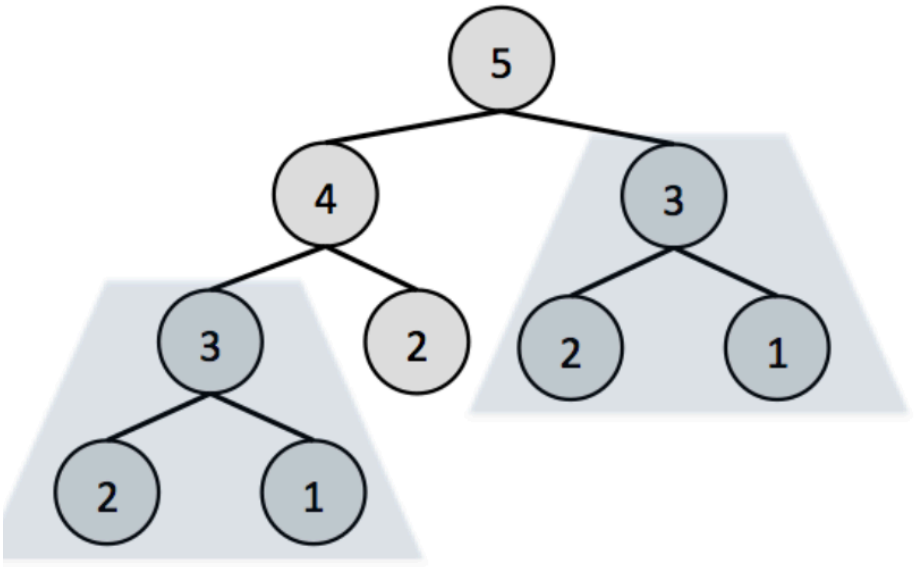
3. Overlapping Subproblems (겹치는 부분 문제)

동일한 하위 문제가 여러 번 반복해서 등장하는 경우, 이러한 문제의 해를 저장해 두고 재사용할 수 있는 구조를 가져야 합니다.

4. DAG(directed acyclic graph)구조

이는 방향성이 있고 사이클이 없는 그래프 구조를 가지는 것을 말합니다. 이는 난이도가 높은 DP문제일 때 많이 나오는데요. 예를 들어  사이클이 있는 그래프 같은 경우 전처리를 통해 DAG를 만들어 DP문제를 풀어야 합니다.

사실 1번은 보통의 코딩테스트 문제라면 알아서 다 지켜서 주고 4번은 코딩테스트 수준에서는 거의 신경쓸 필요가 없으므로 2, 3번만  주요하게 봐주시면 됩니다.




앞의 그림처럼 F(n)=F(n−1)+F(n−2)의 형태를 가졌고 3, 2, 1과 같은 값들이 여러 번 계산되는 것을 볼 수 있는데, 이러한 구조가 있을 때 DP를 적용할 수 있습니다. DP는 이러한 반복되는 계산을 배열, 맵, 셋과 같은 자료구조에 저장하여 불필요한 재계산을 피하고, 문제를 효율적으로 해결하는 방법입니다.

## DP를 시도할 때

그렇다면 이러한 조건이 충족되는지를 판단하고 DP를 시도해봐야할까요?

사실 코딩테스트를 풀 때 앞의 조건이 충족되는지 아닌지를 판단해서 시도하는것은 현실적으로 어렵습니다.

이렇게 생각하면서 시도를 하는게 좋습니다.

1. 완전탐색으로 하기에는 너무나도 큰 경우의 수다.
2. 메모이제이션이 가능할까? 가능하다면 DP를 시도해보자.
3. 만약 배열의 크기가 10억정도가 필요한다는 등 메모이제이션 하기에 너무나도 큰 공간이 피 배열에 넣기에도 너무나 크다면  그리디 등 다른 알고리즘을 쓰자.

*메모이제이션은 배열이든 맵이든 셋이든 어떠한 자료구조에 계산된 결과값을 담는 것을 의미합니다.

## DP = 완전탐색 + 메모이제이션

사실 DP를 만들 때 먼저 점화식을 만들고 거기에 맞춰 코드를 구축해야 합니다. 하지만 그렇게 하기에는 현실적으로 굉장히 어렵습니다.

이렇게 해보는게 어떨까요? 완전탐색을 풀듯이 모든 경우의 수를 생각하고 그 경우의 수를 메모이제이션하는 것입니다.

즉, 다음과 같은 순서로 DP를 구축하면 됩니다.

1. 어떠한 idx에서 모든 경우의 수를 생각하는 겁니다.
2. 그렇게 완전탐색을 하는 구조를 만듭니다. (사실상 이 구조가 점화식이 됩니다.)
3. 그리고 메모이제이션을 겁니다.

## 메모이제이션

그렇다면 메모이제이션이라는 것은 뭘까요? 바로 어떤 상태값을 자료구조에 저장하는 것입니다. **맵이든 셋이든 배열이든 계산된 값을 저장해 놓아서 다시 한번 계산하는 것을 방지하는 것이죠.**

이 상태값을 정의하고 잘 저장하는 것이 DP의 핵심입니다.

예를 들어 BFS를 하고 거기에 따른 최단거리를 담는 2차원 방문배열인 visited[y][x]를 생각해보겠습니다. y라는 상태값과 x라는 상태값을 기반으로 최단거리를 저장해놓죠? 이것과 비슷하게 문제에서 어떤 idx에서 어떤 turn의 상태값이 중요하다면  다음과 같은 2차원 배열을 정의해야 합니다.

* 크기는 임의로 정했습니다.

```
int dp[101][2004];
```

만약 여기서 어떤 idx에서 어떤 turn의 상태값 뿐만아니라 짝수인지 홀수인지도 중요하다면 다음과 같은 3차원 배열이 필요하겠죠?

```
int dp[101][2004][2];
```

이렇게 문제를 보고 어떤 상태값을 기반으로 몇개의 차원을 가진 배열을 만들어서 어떻게 메모이제이션을 할건지를 설정하는게 DP의 핵심 중 하나입니다.

## Q. 큰돌의 사과나무

큰돌은 사과를 매우 좋아한다. 그래서 집에 사과나무를 두 그루 심어두고, 그 나무에서 떨어지는 사과를 받아먹곤 한다. 하지만 큰돌은 키가 작아서 사과를 직접 따먹지는 못하고, 사과가 떨어질 때까지 기다린다. 큰돌은 사과가 떨어지는 순간에만 이를 잡을 수 있으며, 사과가 땅에 떨어지면 깨져서 먹을 수 없게 된다.

매 초마다, 두 개의 나무 중 하나에서 사과가 떨어진다. 큰돌이 사과를 잡으려면 사과가 떨어지는 나무 아래에 서 있어야 하며, 큰돌은 빠르게(1초보다 짧은 시간 안에) 다른 나무로 이동할 수 있다. 그러나 큰돌의 체력은 그리 좋지 않아 많이 움직일 수 없다.

큰돌은 S(1≤S≤100)초 동안 사과를 잡을 수 있다. 큰돌은 최대 M(1≤M≤30)번만 이동할 수 있다. 매 초마다 어느 나무에서 사과가 떨어질지에 대한 정보가 주어졌을 때, 큰돌이 받을 수 있는 사과의 최대 개수를 구하는 프로그램을 작성하시오. 큰돌은 처음에는 1번 사과나무 아래에 위치해 있다.

**입력**

첫째 줄에 두 정수 S와 M이 주어진다. 다음 S개의 줄에는 매 초마다 떨어지는 사과의 나무 번호가 1 또는 2로 주어진다.

**출력**

첫째 줄에 큰돌이 받을 수 있는 사과의 최대 개수를 출력한다.

**예제 입력 1**

7 2

2

1

1

2

2

1

1

**예제 출력 1**

6

이 문제를 보았을 때 idx를 증가시키면서 idx를 재귀함수에 넘겨서 다음 경우의 수를 탐색하는 것을 생각할 수 있습니다. 만약 이 문제를 완전탐색으로 푼다면 시간복잡도는 얼마일까요?

바로 2^30입니다.

굉장히 시간복잡도가 크죠? 이럴 때 DP로 해결할 수 있을까 생각해보아야 합니다. 그러면 메모이제이션이 가능한지를 보기 위해서 배열을 정의해야 하는데요. 배열을 어떻게 정의할 수 있을까요?

- 사과는 최대 100초동안 떨어진다고 하니 매초를 담을 100개짜리 배열이 필요합니다.

- 지금 내가 있는 상태가 1인지 2인지가 중요합니다. 2개짜리 배열이 필요합니다.
- 그리고 움직일 수 있는 카운트가 제한되어있네요. 그걸 담을 30개짜리 배열이 필요합니다.

즉, 다음과 같은 3차원 배열이 필요합니다.  이렇게 문제를 일고 핵심적인 부분을 담아야할 배열을 생각하다 보니 3차원 배열이 완성되는 것을 알 수 있습니다.

```
int dp[104][2][34];
```

### 코드

```
#include <bits/stdc++.h>
using namespace std;

int dp[104][2][34], n, m, b[104];

int go(int idx, int tree, int cnt) {
    if (cnt < 0) return -1e9;
    if (idx == n) return 0;
    int &ret = dp[idx][tree][cnt];
    // 이미 계산된 값이면 반환
    if (ret != -1) return ret;

    // 현재 나무에 머무르거나 다른 나무로 이동하는 경우 중 최대 값 선택
    return ret = max(go(idx + 1, tree ^ 1, cnt - 1), go(idx + 1, tree,
cnt)) + (tree == b[idx] - 1);
}

int main() {
    memset(dp, -1, sizeof(dp));
    cin >> n >> m;
    for (int i = 0; i < n; i++) cin >> b[i];

    // 두 가지 초기 상태(1번 나무에 시작, 2번 나무에 시작) 중 최대 값 출력
    cout << max(go(0, 1, m - 1), go(0, 0, m)) << '\n';
    return 0;
}
```

## DP의 구조

여기서 앞의 코드를 볼 때 중요한 것은 DP의 기본구조를 파악하는 것입니다. DP는 초기화, 기저사례, 메모이제이션, 로직 으로 되어있습니다.

## 1. 초기화 (Initialization)

```
memset(dp, -1, sizeof(dp));
```

dp 배열을 -1로 초기화하여 메모이제이션에 사용될 배열의 값이 아직 계산되지 않았음을 표시합니다.

초기화는 어떤 값으로 초기화를 해야할까요? 초기화는 문제의 값이 될 수 없는 값으로 초기화를 해야 합니다.이 문제의 경우 W의 범위가 최소 1이기 때문에 무조건 1번은 움직일 수 있어서 문제에서 답이 될 수 있는 범위는 1이상입니다. 즉, 0 이하로 초기화를 걸어도 됩니다.

그리고 DP에서 최소를 구할때는 초기 DP를 최대로 초기화하고 최대값을 구할 때는 최소로 초기화해서 최댓값을 구하는 것이 중요합니다.

## 2. 기저 사례 (Base Case)

기저 사례는 재귀 함수에서 재귀 호출이 종료되는 조건입니다. 이 코드에서는 두 가지 기저 사례가 존재합니다.

```
if (cnt < 0) return -1e9;
```

이동 횟수(cnt)가 음수가 되면 불가능한 경우로 간주하여 매우 작은 값(-1e9)을 반환합니다. 이는 불가능한 경로임을 표시하고, 최댓값을 계산할 때 해당 경우의 수가 제외되도록 합니다.

```
if (idx == n) return 0;
```

현재 인덱스(idx)가 전체 시간(n)과 같아졌다면 0을 반환합니다.

## 3. 메모이제이션 (Memoization)

```
int &ret = dp[idx][tree][cnt];
```

dp 배열을 사용하여 이미 계산된 값을 저장합니다. dp[idx][tree][cnt]는 idx 번째 시간에 tree (1번 나무 또는 2번 나무)에 위치해 있으며 남은 이동 횟수가 cnt일 때의 최대 사과 개수를 저장합니다.

```
if (ret != -1) return ret;
```

ret이 -1이 아니면 이미 계산된 값을 의미합니다. 이 때 그 값을 바로 반환합니다. "값이 있다면 리턴한다" 이것이 바로 메모이제이션입니다. 여기서 중요한 점이 있는데 DP는 어떠한 값을 리턴해서 쌓아가야 하는 알고리즘입니다. 때문에 **DP 함수의 리턴형은 무조건 void가 아닙니다.**

## 4. 로직 (Logic)

```
return ret = max(go(idx + 1, tree ^ 1, cnt - 1), go(idx + 1, tree, cnt))
+ (tree == b[idx] - 1);
```

현재 위치에서 다음 초(idx + 1)로 이동하는 두 가지 선택지를 고려합니다. 2가지 경우의 수밖에 없습니다.

- 현재 나무(tree) 아래에 그대로 머무를 경우 (go(idx + 1, tree, cnt)).
- 다른 나무로 이동할 경우 (go(idx + 1, tree ^ 1, cnt - 1)). 이 경우 이동 횟수가 하나 줄어듭니다.

이 두 가지 경우 중 더 큰 값을 선택하고, 현재 사과가 떨어지는 나무와 현재 나무(tree)가 같은 경우에는 1을 더해줍니다.

# DP의 종류

ex1) 피보나치 수열 점화식

$$F(n) = F(n-1) + F(n-2)$$

## 피보나치 수열 : 탑다운 방식

```
#include<bits/stdc++.h>
using namespace std;
int dp[1004];
int fibo(int n){
    if(n <= 1) return n;
    int & ret = dp[n];
    if(ret != -1) return ret;
    return ret = fibo(n - 1) + fibo(n - 2);
}
int main(){
    int n = 10;
```

```
    memset(dp, -1, sizeof(dp));
    int ret = fibo(n);
    cout << ret << '\n';
}
```

이 코드는 탑다운DP(재귀적 DP) 입니다. 재귀적인 구조를 가지고 있으며 큰 N에서 1 또는 0 등 작은 숫자로 향하게 함수가 호출되며 DP에 필요한 배열만을 만들며 코드가 직관적이라는 장점과 재귀호출에 대한 오버헤드가 있다는 단점이 있습니다.

### 피보나치 수열 : 바텀업 방식

```
#include<bits/stdc++.h>
using namespace std;
int dp[1004];
int main(){
    memset(dp, -1, sizeof(dp));
    int n = 10;
    dp[0] = 0;
    dp[1] = 1;
    for(int i = 2; i <= n; i++){
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    cout << dp[n] << '\n';
}
```

이 코드는 바텀업DP(반복적DP)입니다. 보통 0이나 1 작은 숫자에서 N까지 반복문을 통해 DP를 만들고 재귀호출에 대한 오버헤드가 없다는 장점이 있지만 문제에 필요한 배열뿐만 아니라 모든 배열을 다 만든다는 단점이 있습니다. 보통 탑다운DP보다 더 빠릅니다.

## 경우의 수를 푸는 DP

경우의 수를 풀 때 DP를 사용하는 경우가 있습니다. 이 때는 “그냥 다 더한다. “ 고 생각하면 됩니다.

## Q. 큰돌의 숫자 놀이

큰돌은 숫자와 놀이하는 것을 좋아한다. 그는 정수 하나를 선택한 다음, 그 숫자를 1, 2, 3의 합으로 표현하는 방법을 찾으려고 한다. 예를 들어, 숫자 4를 1, 2, 3의 합으로 나타내는 방법은 총 4가지가 있다. 합을 나타낼 때는 숫자를 1개 이상 사용해야 하며, 합을 이루고 있는 숫자의 순서는 중요하지 않다.

큰돌은 모든 가능한 경우의 수를 알고 싶어한다. 큰돌을 도와 주어진 숫자를 1, 2, 3의 합으로 나타낼 수 있는 방법의 수를 구하는 프로그램을 작성하시오.

---

**입력**

첫째 줄에 테스트 케이스의 개수 T가 주어진다. 각 테스트 케이스는 한 줄로 이루어져 있고, 정수 n이 주어진다. n은 양수이며 10,000보다 작거나 같다.

**출력**

각 테스트 케이스마다, n을 1, 2, 3의 합으로 나타낼 수 있는 방법의 수를 출력한다.

**예제 입력 1**

3

4

8

15

**예제 출력 1**

4

10

27

## 코드

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
ll t, n, dp[10001];
int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cin >> t;
    dp[0] = 1;
    for(int i = 1; i <= 3; i++){
        for(int j = 1; j <= 10000; j++){
            if(j - i >= 0)dp[j] += dp[j - i];
        }
    }
    while(t--){
        cin >> n;
        cout << dp[n] << "\n";
    }
```

```
    return 0;
}
```

## 코드설명

```
typedef long long ll;
```

보통 경우의 수 문제는 매우 큰 숫자가 나오기 때문에 long long 타입을 쓰는게 좋습니다. ll로 간단히 사용하기 위해 정의합니다.

```
dp[10001];
```

dp[i]는 정수 i를 1, 2, 3의 합으로 나타내는 방법의 수를 저장합니다.

```
dp[0] = 1;
```

dp[0]을 1로 설정합니다. 이는 숫자 0을 만들기 위한 경우의 수가 단 하나, "아무 숫자도 선택하지 않는 경우"이기 때문입니다.

```
for(int i = 1; i <= 3; i++){
    for(int j = 1; j <= 10000; j++){
        if(j - i >= 0) dp[j] += dp[j - i];
    }
}
```

- i는 사용 가능한 숫자 (1, 2, 3)를 나타냅니다. 이 반복문은 각 숫자를 사용할 때의 경우의 수를 DP 배열에 반영합니다.
- j는 목표 숫자입니다. j를 1부터 10000까지 순회하며, j를 i를 사용하여 만들 수 있는 경우의 수를 갱신합니다.
- dp[j] += dp[j - i]는 j에서 i를 뺀 값(j - i)에서 i를 추가하는 경우를 더해줍니다. 이를 통해 j를 만들 수 있는 모든 경우의 수를 누적해서 dp를 만듭니다.

# 8주차 개념

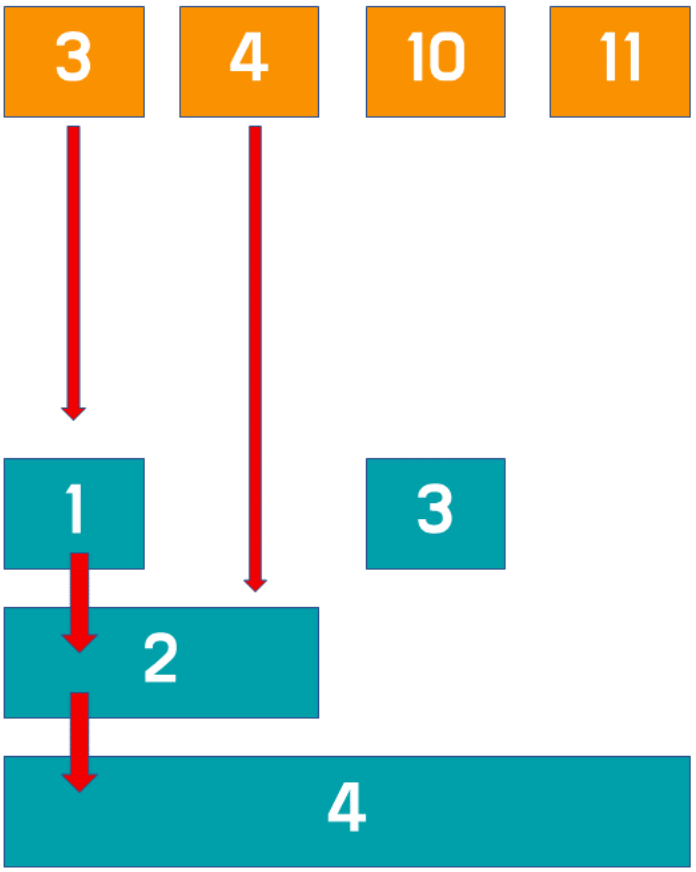
8주차에서는 펜윅트리와 최단거리알고리즘인 다익스트라, 벨만포드, 플로이드 워셜을 배워보겠습니다.

# 펜윅트리

펜윅 트리(Fenwick Tree)는 최하위 켜져있는 비트를 기반으로 트리를 만들어 동적배열에서 구간합을 효율적으로 구할 수 있는 자료구조입니다. 구간 합을 구하는 쿼리와 값을 갱신하는 작업을 O(logN)만에 할 수 있습니다.

참고 : 최하위 켜져있는 비트는 4주차 비트마스킹강의 : 최하위 켜져있는 비트 찾기 idx = (S & -S) 를 참고해주세요.

## 원리




펜윅트리는 {3, 4, 10, 11}이라는 배열이 주어졌을 때앞의 그림의 파란색 노드를 만들어 구간합을 구하는 트리라고 보면 됩니다. 3이라는 요소는 파란색노드 {1, 2, 4}에 더해집니다. 4라는 요소는 {2,

4}에 더해집니다. 10이라는 요소는 {3, 4}에 더해집니다. 11은 {4}에 더해집니다. 이렇게 더해져서 만들어진 파란색노드를 기반으로 보면 {4, 10, 11}의 합은 파란색노드 4 - 1이 되는 것을 볼 수 있습니다.

## 코드

```
#include <bits/stdc++.h>

using namespace std;

int n;
vector<int> tree;

void update(int index, int value) {
    while (index < tree.size()) {
        tree[index] += value;
        index += index & -index;
    }
}

int sum(int index) {
    int ret = 0;
    while (index > 0) {
        ret += tree[index];
        index -= index & -index;
    }
    return ret;
}

// 구간 [left, right]의 합 구하기
int rangeQuery(int left, int right) {
    return sum(right) - sum(left - 1);
}

int main() {
    vector<int> data = {3, 4, 10, 11};
    n = data.size();
    tree.resize(n + 1, 0);

    // 초기 배열을 기반으로 트리 구성
    // 트리의 인덱스는 1부터 시작합니다.
    for (int i = 0; i < n; ++i) {
        update(i + 1, data[i]);
    }
```

```
    cout << "1 ~ 4 sum :" << rangeQuery(1, 4) << "\n";  // 28
    cout << "2 ~ 3 sum :" << rangeQuery(2, 3) << "\n";  // 14
    // index 2에 5 더하기, 즉 data[1] = 4 + 5 = 9로 변경
    update(2, 5);
    cout << "1 ~ 4 sum :" << rangeQuery(1, 4) << "\n";  // 33
    cout << "2 ~ 3 sum :" << rangeQuery(2, 3) << "\n";  // 19

    return 0;
}
```

## update()

최하위 파란색 노드를 만드는 역할을 합니다. 예를 들어 data[1] = 5였는데 9로 변했다면 해당 차이점(+4)을 각각의 노드에 더하는 식으로 업데이트를 합니다.

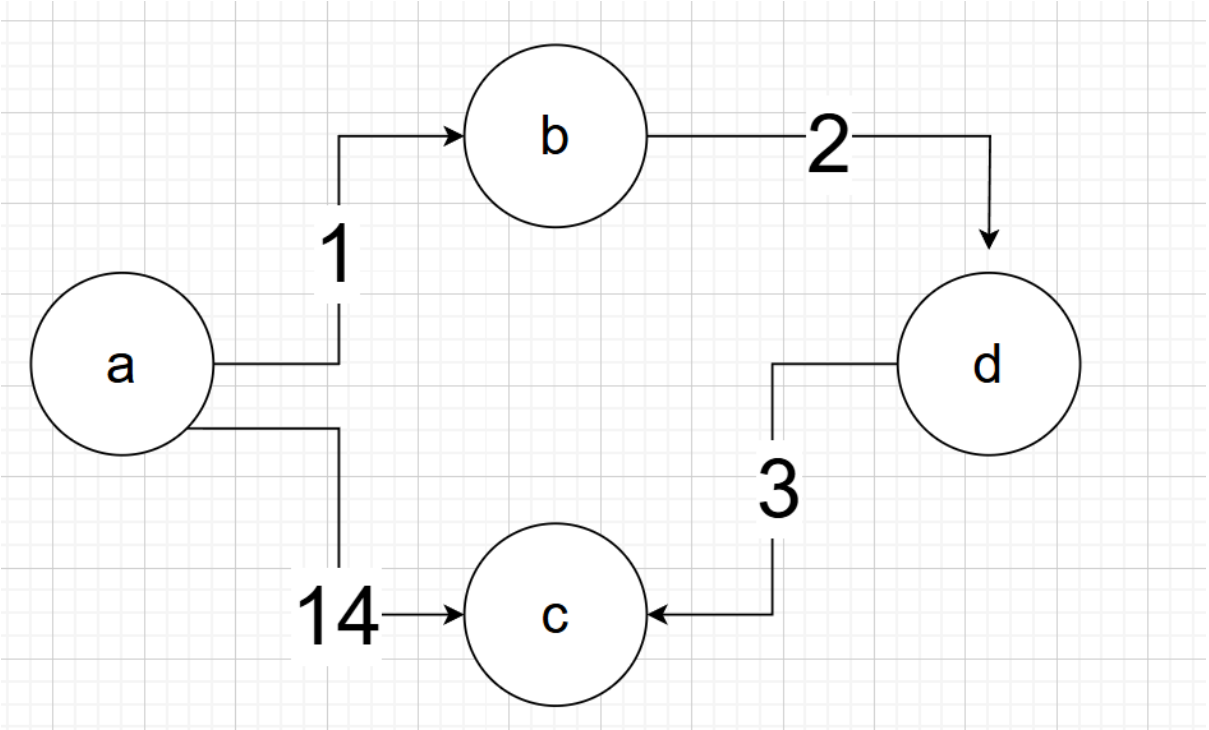
## sum(idx)

해당 idx까지의 합을 파란색노드를 기반으로 더해서 값을 리턴합니다.

# 최단거리 알고리즘

최단거리 알고리즘은 그래프 상의 두 정점 사이의 거리가 최소가 되는 경로를 찾는 알고리즘입니다.

앞서서 BFS는 가중치가 같은 그래프에서 최단거리 알고리즘으로 쓸 수 있지만 가중치가 다를 경우 최단거리 알고리즘을 써야 한다고 했었죠? 바로 그 최단거리 알고리즘입니다.




BFS로 최단거리를 못찾는 사례: a -> c 의 최단거리는 6입니다.

최단거리 알고리즘은 A*, 존슨 등 여러 알고리즘이 있지만 이 강의에서는 코딩테스트에 주로 나오는 알고리즘인 **다익스트라 알고리즘**, **벨만-포드 알고리즘**, **플로이드-워셜 알고리즘**을 배워보도록 하겠습니다.

## 1. 다익스트라 알고리즘 (Dijkstra's Algorithm)

다익스트라 알고리즘은 **양의 가중치**만을 가지는 그래프에서 **한 정점에서 다른 모든 정점까지의 최단 거리를 구하는 알고리즘**입니다. u -> {a, b, c} 우선순위 큐를 사용해 현재 방문할 정점 중 가장 비용이 작은 경로를 우선적으로 선택하고, 거리 배열(보통 dist[]라는 변수명을 씁니다.)을 갱신하여 최단 경로를 찾는 방식입니다.

## 원리

구체적으로, 알고리즘은 다음 과정을 반복합니다:

1. 우선순위 큐에서 가장 짧은 경로의 정점을 뽑아냅니다.
2. 선택한 정점을 통해 이동할 수 있는 정점들의 거리 값을 계산하여, 기존 거리보다 짧으면 dist[] 배열을 갱신합니다. 이 때 정점 u에서 정점 v까지 거리와 비교하여 더 짧아진 경우에 dist[u] + w(u, v) = dist[v]로 갱신하는데 이를 완화가 일어났다고 합니다.
3. 갱신된 최단거리와 정점의 정보를 다시 우선순위 큐에 넣고 1번부터 다시 반복합니다.

## Q. 시작점에서 다른 모든 교차로까지의 최단 경로 찾기

어떤 도시에는 N개의 교차로와 M개의 도로가 있습니다. 각 도로는 두 교차로를 연결하며, 이 도로를 지나는 데 걸리는 시간이 주어집니다. 특정 시작 교차로에서 다른 모든 교차로까지의 최단 시간을 구하는 프로그램을 작성하세요.

**문제 조건**

- N (교차로의 개수): 1 <= N <= 20,000
- M (도로의 개수): 1 <= M <= 100,000
- 모든 도로의 가중치는 양수이며, 연결되지 않은 교차로도 있을 수 있습니다.
- 입력 형식:
    - 첫 줄에 교차로의 수 N, 도로의 수 M, 그리고 시작 교차로 S가 주어집니다.
    - 그 다음 M개의 줄에 걸쳐 각 도로의 정보가 세 개의 정수로 주어집니다: u (출발 교차로), v (도착 교차로), w (도로를 지나는 데 걸리는 시간).

**출력 형식**

- 시작 교차로 S에서 각 교차로까지의 최단 거리를 출력합니다. 만약 도달할 수 없는 교차로가 있다면 "INF"를 출력합니다.

**예제 입력**

5 7 1

1 2 1

1 3 4

2 3 2

2 3 1

2 4 5

3 4 1

4 5 2

**예제 출력**

0

1

2

3

5

**코드**

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9; // 무한대 값 설정
vector<pair<int, int>> adj[20004]; // 그래프의 인접 리스트 표현
vector<int> dist(20004, INF); // 최단 거리 배열

void dijkstra(int start) {
    priority_queue<pair<int, int>, vector<pair<int, int>>,
greater<pair<int, int>>> pq;
    dist[start] = 0;
    pq.push({0, start});
```

```
    while (!pq.empty()) {
        int here_cost = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        // 느긋한 삭제
        if (dist[u] != here_cost) continue;

        for (auto &[v, weight] : adj[u]) {
            int new_cost = here_cost + weight;
            if (new_cost < dist[v]) {
                dist[v] = new_cost;
                pq.push({new_cost, v});
            }
        }
    }
}

int main() {
    int n, m, start;
    cin >> n >> m >> start;

    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
    }

    dijkstra(start);

    for (int i = 1; i <= n; i++) {
        if (dist[i] == INF) cout << "INF\n";
        else cout << dist[i] << "\n";
    }

    return 0;
}
```

## 코드구조

무한대 초기값 -> 초기값은 0

```
vector<int> dist(20001, INF);
dist[start] = 0;
```

최단거리배열은 최소값을 만드는 것이 핵심입니다. 최소값은 최대값부터 시작해야 하기 때문에 아주 큰 INF로 초기화를 해주고 시작점은 0부터 시작해서 최단거리 배열을 담습니다.

**완화**

```
int new_cost = here_cost + weight;
if (new_cost < dist[v]) {
      dist[v] = new_cost;
      pq.push({new_cost, v});
}
```

우선순위 큐를 사용하여 최단 거리가 가장 작은 순으로 뽑고  "내가 탐색하려는 이 정점"이 “지금의 정점” + “그 정점까지의 거리”가 더 작다면 갱신을 합니다.

**느긋한 삭제**

느긋한 삭제는 pq안에 여러개의 같은 정점이 저장될 수 있는데 이 중에서 가장 짧은 정점만을 처리하는 것을 말하며 그 외의 불필요한 정점을 바로바로 삭제 하지 않고 나중에 다음과 같이 continue를 통해 거르는 것을 말합니다. (최소값을 가진 정점 하나만을 기반으로 인접정점들을 탐색하는게 최적이기 때문입니다.)

```
        if (dist[u] != here_cost) continue;
        if (dist[u] < here_cost) continue;
```

이렇게 두개의 코드로 표현되며 둘 다 같은 의미입니다.

dist[]와 here_cost의 값이 다르다 라는 것은 pq에 집어 넣을 때의 값과 달라졌다는 것을 의미합니다. 즉, 집어넣은 이후에 앞서서 더 작은 값으로 완화되었다는 것을 의미하며 이미 더 작은 값으로 완화가 되었기 때문에 이 부분을 다시 탐색하는 것은 불필요합니다. 이 때 그냥 넘어가는 것을 말합니다.

## 시간복잡도

시간복잡도는 ElogE 또는 ElogV입니다.

최악의 경우 모든 간선에 대해 우선위큐에 집어넣어야 하며 이 때 E개의 간선에 대해 우선순위큐를 기반으로 최단거리 정점 추출에 logE가 들어 ElogE가 됩니다.

여기서 E는 방향성이 있는 완전그래프에서 E = V(V - 1)라는 특징을 가집니다.

이 때문에 ElogV^2 가 되고 이는 2ElogV가 되어 ElogV가 됩니다.

[참고] 다익스트라 알고리즘의 증명

우선, 다익스트라 알고리즘이 최단 경로를 찾지 못하는 경우가 있다고 가정해보겠습니다.

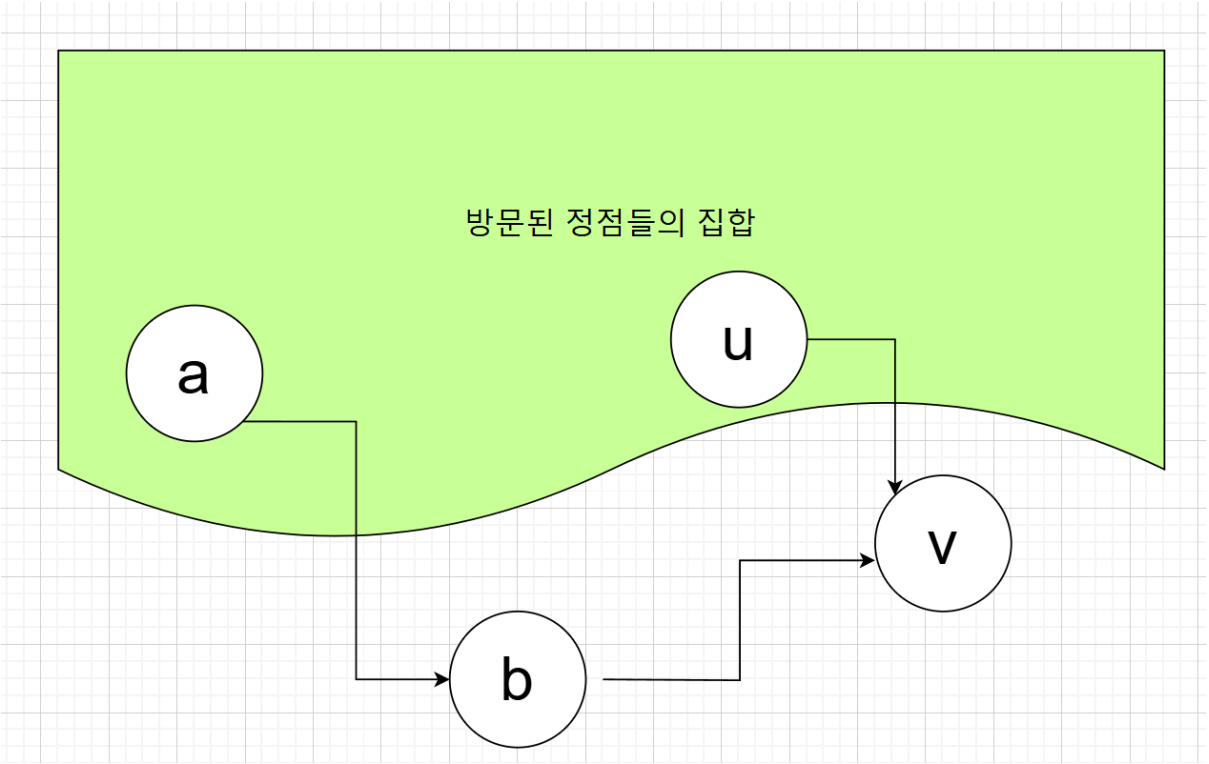
즉, 다익스트라 알고리즘으로 계산한 어떤 정점 v의 최단 거리 값 dist[v]가 실제 최단 거리보다 크다고 가정합니다. 이 말은 정점 v에 도달하는 더 짧은 경로가 존재함을 의미합니다.

2. 최단 거리보다 짧은 경로의 존재를 가정

이제 정점 v까지 도달하는 더 짧은 경로가 존재한다면, 그 경로는 두 가지 경우로 나눌 수 있습니다.

- 첫 번째 경우: 경로에 포함된 모든 정점이 이미 방문된 상태입니다. (즉, 큐에서 꺼낸 상태로, 다익스트라 알고리즘은 해당 정점들까지의 최단 거리를 확정한 상태입니다.)
- 두 번째 경우: 경로에 아직 방문하지 않은 정점이 포함되어 있습니다.

이제, 이 두 경우 중 두 번째 경우에 대해 살펴보겠습니다.




아직 방문하지 않은 정점을 b라고 정의하고, 이 경로 상에서 b 바로 전에 위치한 이미 방문한 정점을 a라고 합시다. 다익스트라 알고리즘은 a를 방문했기 때문에 dist[a]에는 최단거리가 들어가있습니다. 여기서 dist[b] = dist[a] + w(a, b)라고 할 수 있습니다.

다익스트라 알고리즘에서는 우선순위 큐의 특성상, 현재까지의 최단 거리가 낮은 순서대로 정점이 꺼내지므로 정점 v가 b보다 먼저 큐에서 꺼내졌다면 이는 다음의 조건을 만족합니다.

dist[v] ≤ dist[b]

하지만, 우리가 앞서 가정한 바에 따르면, v에 대해 dist[v]가 실제 최단 거리보다 더 크고, 따라서 b를 경유한 경로가 더 짧다는 가정을 했기 때문에 이 가정에 따르면 다음이 성립해야 합니다:

dist[v]>dist[b]

하지만 위 두 부등식이 서로 모순되는 상황이므로통 이를 통해 "다익스트라 알고리즘이 모든 정점에 대해 최단 경로를 찾지 못하는 경우가 있다"는 주장이 잘못되었음을 증명할 수 있습니다.

즉, 귀류법을 통해 다익스트라 알고리즘은 모든 정점에 대해 정확한 최단 경로를 찾는 것을 증명했습니다.

## 2. 플로이드 워셜(Floyd-Warshall)

플로이드 워셜 알고리즘은 모든 정점에서 모든 정점 까지의 최단 경로를 구하는 알고리즘입니다. {a, b, c} -> {a, b, c}

**음수 가중치가 있는 그래프에도 사용할 수 있지만, 음수 사이클이 존재하면 사용할 수 없습니다.**
모든 노드 쌍 (i,j)에 대해, 노드 k를 경유했을 때의 최단 경로를 구하고 갱신하는 방식으로 진행됩니다.

### 원리

노드 i에서 j까지의 경로가 존재할 때 노드 k를 경유하는 경우가 더 짧은 경로라면 해당 경로로 완화합니다.

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

### Q. 모든 도시 간 최단 경로

N개의 도시와 M개의 일방향 도로가 주어집니다. 각 도로는 특정 두 도시를 연결하고, 비용이 주어집니다. 모든 도시에 대해 다른 모든 도시까지의 최소 비용을 구하세요. 만약 경로가 존재하지 않는다면, INF를 출력합니다.

- 도시의 수 N은 최대 100개입니다.
- 도로의 수 M은 최대 10,000개입니다.
- 경로의 비용은 0 이상의 정수입니다.

## 코드

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;  // 도달할 수 없는 경로를 나타내는 큰 값
const int MAX_N = 204;

int dist[MAX_N][MAX_N];

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == j) {
                dist[i][j] = 0;
            } else {
                dist[i][j] = INF;
            }
        }
    }
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        dist[u][v] = min(dist[u][v], w);
    }
    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (dist[i][k] != INF && dist[k][j] != INF) {
                    dist[i][j] = min(dist[i][j], dist[i][k] +
dist[k][j]);
                }
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (dist[i][j] == INF) {
                cout << "INF ";
```

```
            } else {
                cout << dist[i][j] << " ";
            }
        }
        cout << "\n";
    }

    return 0;
}
/*
4 4
1 2 4
1 3 1
3 2 2
2 4 1


0 3 1 4
INF 0 INF 1
INF 2 0 3
INF INF INF 0
*/
```

### 시간복잡도

시간복잡도는 주어진 V에 대해 3개의 중첩 반복문을 통해 최단거리를 결정하므로 O(V^3)을 가집니다. 시간복잡도가 크기 때문에 보통은 V가 400이하인 문제에서만 사용할 수 있습니다. 다만, 문제의 시간초과상한선이 클 경우 400을 넘어가더라도 간혹 되는 경우가 있습니다.

## 3.벨만-포드(Bellman-Ford)

**벨만-포드 알고리즘은 음수 가중치가 있고 음수 사이클이 있는 그래프에서 단일 출발점 최단 경로를 구하는 알고리즘**이며 음수 사이클이 존재하는 것도 판별할 수 있습니다.

u -> {a, b, c}

만약 문제에서 타임머신, 블랙홀, 과거로 돌아갈 수도 있다 이런 워딩이 있다면 벨만포드를 써야 하지 않을까? 생각해봐야 합니다.

## 원리

벨만-포드는 모든 간선을 반복적으로 확인하며 **최단 경로를 갱신합니다.**

1. **초기화**:출발 노드에서 출발 노드까지의 거리는 0으로 설정하고, 나머지 노드들은 무한대(∞)로 초기화합니다.
2. **최단 경로 갱신**: 모든 간선을 V−1번 반복하여 검사하며 **현재 경로가 기존 경로보다 짧다면 해당 경로로 완화**합니다.
3. **음수 사이클 확인**: 최단 경로가 확정된 후에도, 다시 모든 간선을 검사하여 최단 경로가 더 짧아지는 경로가 있다면, 이는 **음수 사이클**이 존재함을 의미합니다.

## Q. 음수 사이클 감지

주어진 그래프에서 출발 도시 1 로부터 다른 모든 도시에 대한 최단 경로를 구하고, 음수 사이클이 존재하는 경우 이를 감지하세요. 음수 사이클이 있는 경우 -1을 출력하고, 없는 경우 각 도시까지의 최단 거리를 출력하세요.

**입력**

- 첫 번째 줄에 노드의 수 n (2 ≤ n ≤ 1000)과 간선의 수 m (1 ≤ m ≤ 5000)이 주어집니다.
- 다음 m개의 줄에는 세 정수 a, b, c가 주어지며, 이는 도시 a에서 도시 b로 가는 가중치 c의 단방향 간선이 있음을 의미합니다. (-10000 ≤ c ≤ 10000)

**출력**

- 만약 음수 사이클이 존재한다면, -1을 출력합니다.
- 음수 사이클이 없으면 첫 번째 도시에 대해 나머지 모든 도시에 대한 최단 거리를 출력합니다. 도달할 수 없는 경우 -1을 출력합니다.

**예제입력1**

4 5

1 2 3

1 3 4

2 3 -2

2 4 5

4 1 -10

**예제출력1**

-1

**예제입력2**

4 4

1 2 4

1 3 1

2 3 2

3 4 5

**예제출력2**

0

4

1

6

## 코드

```
#include<bits/stdc++.h>
using namespace std;
long long t, n, m, a, b, c, dist[1004], INF = 987654321;
int main(){
   ios_base::sync_with_stdio(false); cin.tie(NULL); cout.tie(NULL);
   cin >> n >> m;
   fill(dist, dist + 1004, INF);
   vector<pair<int,int>> adj[1004];
   for(int i = 0; i < m; i++){
       cin >> a >> b >> c;
```

```
        adj[a].push_back({b, c});
    }
    dist[1] = 0; queue<int> q;
    for(int i = 0; i < n; i++){
        for(int here = 1; here <= n; here++){
            for(auto there : adj[here]){
                int d  = there.second;
                int to = there.first;
                if(dist[here] != INF && dist[here] + d < dist[to]){
                    if(i == n - 1)q.push(to);
                    dist[to] = dist[here] + d;
                    cout << here << " : " << to << "\n";
                }
            }
        }
    }
    if(q.size()) cout << -1 << "\n";
    else for(int i = 1; i <= n; i++) cout << (dist[i] == INF ? -1 :
dist[i]) << "\n";
}
```

## 코드 분석

1. dist배열을 최대값으로 설정합니다.

```
fill(dist, dist + n + 1, INF);
```

2. 시작지점을 0으로 놓습니다.

```
dist[0] = 0;
```

3. 완화를 하면서 dist배열을 갱신합니다. 하지만 우선순위큐를 쓰지 않습니다.

```
if(dist[here] != INF && dist[here] + d < dist[to]){
  if(i == n - 1)q.push(to);
  dist[to] = dist[here] + d;
}
```

4. 또한 아래와 같이 마지막 i 번째 또한 완화가 일어났다면 사이클이 존재한다"라는 것을 파악합니다.

```
if(i == n - 1)q.push(to);
```

V개의 노드가 있는 그래프에서 최단경로는 최대 V - 1개의 간선으로 구성됩니다. 이런 간선들을 기반으로 최단 경로를 구했는데 다시 갱신된다는 의미는 음수사이클이 있다는 것임을 의미합니다.

### 시간복잡도

시간복잡도는 모든 간선에 대해 V 번 확인하기 때문에 O(V * E)의 시간복잡도를 갖습니다.

## 4. 최단거리 알고리즘 비교

이렇게 최단거리 알고리즘을 알아봤습니다. 그렇다면 어떨 때 다익스트라를 쓰고 어떨 때 플로이드워셜을 써야할까요? 다음과 같습니다.

1. 다익스트라 : 시작점부터 끝점까지 최단거리를 구해야하고 N의 크기가 크고 음의 가중치를 가지지 않은 문제

2. 플로이드워셜 : 모든 정점의 쌍의 거리를 구해야 하는 문제이며 N의 크기가 400이하로 작은 문제

3. 벨만포드 : 시작점부터 끝점까지 최단거리를 구해야하고 음의 사이클이 있을 만한 문제

# 강의를 마무리하며.

이렇게 8주차까지 코딩테스트 대비 모든 알고리즘개념강의를 끝이 났습니다.

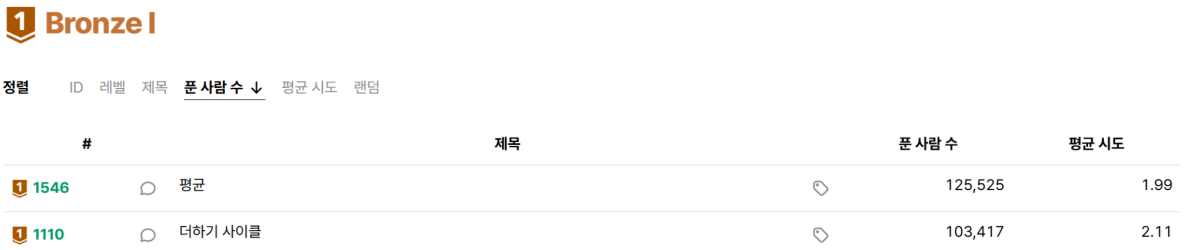
이제 해야 할일은 다음과 같습니다.

1. 지금까지 푼 문제 중 틀리거나 오랫동안 고민한 문제를 맞출 때까지 다시 풉니다. (1, 3, 7, 21일 마다) 틀린 문제는 어떤 부분에서 막히는가를 정확히 파악하는데 집중하고 처음부터 푼다는 생각으로 접근을 하는게 좋습니다.

2. 실버, 브론즈 수준의 문제를 빨리 풀어봅니다. ex) 1시간내에 10문제 풀기 / 이를 통해 쉬운 문제를 풀 때의 실수를 줄입니다.

이 때, 다음그림처럼 푼사람수 내림차순으로 푸는게 좋습니다.

https://solved.ac/problems/level/5?sort=solved&direction=desc&page=1




3. 내가 어려워하는 개념 또는 약한개념의 문제를 다시 풉니다. (solved.ac 사이트를 참고)

4. 다음의 프로그래머스 기출문제를 풉니다.

https://school.programmers.co.kr/learn/challenges?order=recent&page=1&partIds=49442%2C49441%2C6174%2C17615%2C18952%2C21163%2C23708%2C301%2C300%2C17931%2C18498%2C22586%2C9317%2C12286%2C17214%2C20069%2C21366%2C25448%2C31236%2C37527%2C58464

5. 코딩테스트에 자주 나오지는 않지만 가끔 등장하는 개념을 공부합니다.

다음의 개념을 배우는 것을 추천합니다. {유니온파인드, MST, LCS, 위상정렬}

**그동안 여기까지 오느라 수고 많으셨습니다.**
**좋은 수강평과 별점 5점은 제게 큰 힘이 됩니다. :)**
**감사합니다.**
**강사 큰돌 올림.**

# 올솔브특강

## 1.유니온파인드

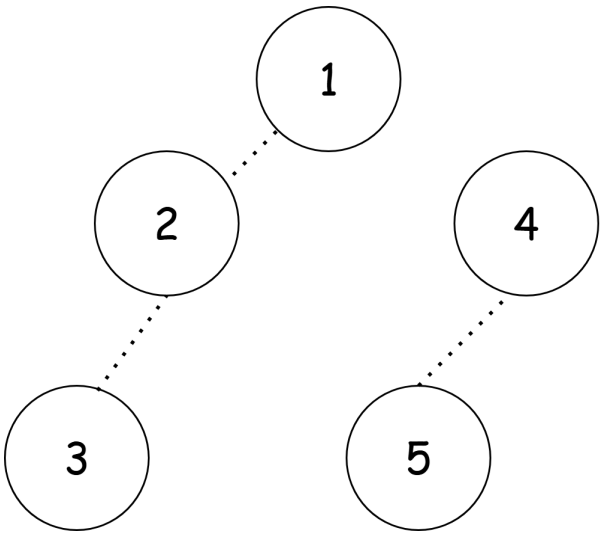
마지막까지 오신 수강생분들을 위한 선물입니다. 코딩테스트에 자주 나오지는 않지만, 가끔 나왔을 때 해당 개념을 모르면 고생하는 개념들을 배워보겠습니다.

유니온파인드(Union-Find)는 서로 겹치지 않는 집합들을 관리할 때 쓰이는 알고리즘이며 상호배타적집합(Disjoint Set)이라고도 불립니다. 이는 다음의 용도로 쓰입니다.

- 그래프에서 사이클 검출
- 한 노드가 노드와 연결되어있는지를 확인
- MST (최소 신장 트리)

Q. 3번노드와 5번노드가 같은 집합임을 알려면 어떻게 해야할까요?

DFS와 BFS를 돌리면 되겠죠? 좋은 생각입니다. 하지만 이런 그래프가 더 복잡해지고 매번 이런 쿼리들이 있다면 더 나은 생각을 해야 합니다. 이 때 쓰이는 것이 바로 유니온 파인드입니다.




이 알고리즘은  딱 2가지 함수만 기억하면 됩니다. 바로 find와 merge 함수입니다. (이를 UF-find, UF-union연산이라고도 부릅니다.)

### find(x)

find(x)는 특정 원소 x가 속한 집합의 루트노드를 찾아 해당 루트노드를 할당하는 함수입니다.

```
int find(int x) {
    if(p[x] == -1) {
        return x;  // x가 루트 노드인 경우
    }
    return p[x] = find(p[x]);  // 경로 압축
}
```

## merge()

merge()는 두 개의 집합을 하나로 합치는 함수입니다.

```
bool merge(int a, int b){
    a = find(a); b = find(b);
    if(a == b) return 1;
    else {
        p[b] = a; return 0;
    }
}
```

내부적으로는 두 원소의 루트 노드를 찾아, 그 중 하나를 다른 하나의 부모로 설정하여 트리 2개를 트리 1개로 합치는 함수입니다.

[참고] 사실 union()이라는 함수명을 쓰는게 맞지만 놀랍게도 union이라는 C++의 키워드가 있기 때문에 눈물을 머금고 merge()라고 써야 합니다.

```
union kundol{
    int age;
    string name;
};
```

# Q. 큰돌의 친구 그룹 만들기

여기는 알고리즘 명문학교 "큰돌의 터전"! 이 학교의 학생들은 엄청난 친화력을 자랑하며, 하루가 멀다 하고 친구를 사귑니다. 학생들은 1번부터 N번까지 번호가 붙어 있고, 친구가 되면 같은 그룹으로 묶입니다.친구의 친구도 친구! 즉, 연결되어 있으면 전부 같은 그룹으로 간주됩니다.

큰돌은 요즘 궁금한 게 생겼습니다."도대체 얘랑 쟤는 같은 친구 그룹일까?"

그래서 큰돌은 여러분에게 Q개의 질문을 던집니다.  친구 관계 정보와 함께, 주어진 두 학생이 같은 그룹인지 빠르게 판단해 주세요!

문제 범위

- N: 학생 수 (1 ≤ N ≤ 100,000)
- M: 친구 관계 수
- Q: 큰돌의 궁금한 질문 수
- 이후 M줄: 학생 ai, bi는 친구가 되었음 (1 ≤ ai, bi ≤ N)
- 이후 Q줄: 학생 xi, yi가 같은 친구 그룹인지 판단

출력 형식

- 각 쿼리마다 YES 또는 NO 출력

예제 입력

8 5 2

1 2

2 3

4 5

6 7

7 8

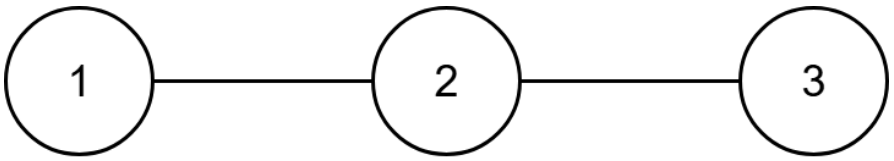
1 3

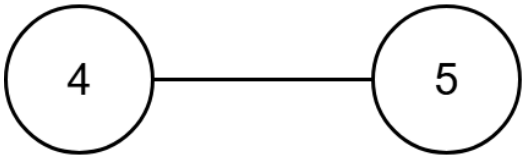
3 4

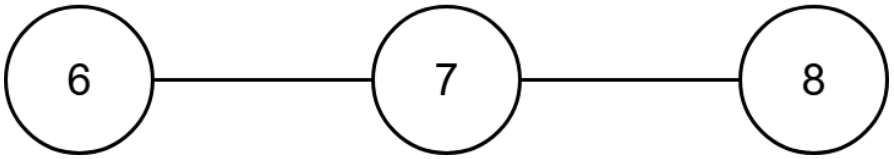
예제 출력

YES

NO




## 코드

```
#include <bits/stdc++.h>
using namespace std;

const int MAX = 100004;

int p[MAX];
int uf_rank[MAX];
int N, M, Q, a, b,x, y;
int find(int x) {
    if(p[x] == -1) {
        return x;  // x가 루트 노드인 경우
    }
    return p[x] = find(p[x]);  // 경로 압축
}

void merge(int a, int b) {
    int rootA = find(a);
```

```
    int rootB = find(b);
    if (rootA == rootB) return;
    // rank가 더 작은 쪽을 부모로 설정
    if (uf_rank[rootA] < uf_rank[rootB]) {
        p[rootA] = rootB;
    } else {
        p[rootB] = rootA;
        if (uf_rank[rootA] == uf_rank[rootB]) {
            uf_rank[rootA]++;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> N >> M >> Q;
    memset(p, -1, sizeof(p));
    for (int i = 0; i < M; i++) {
        cin >> a >> b;
        merge(a, b);
    }
    for (int i = 0; i < Q; i++) {
        cin >> x >> y;
        if (find(x) == find(y)) {
            cout << "YES\n";
        } else {
            cout << "NO\n";
        }
    }

    return 0;
}
```

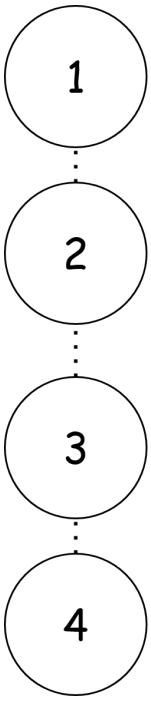
## 경로압축

이 문제를 해결하기 위해 경로압축(path compression)을 합니다.

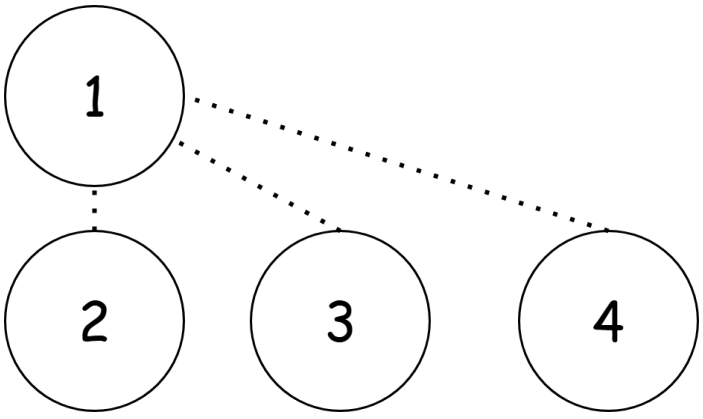
```
int find(int x) {
    if (parent[x] == x) return x;
    return parent[x] = find(parent[x]);
}
```

매번 find()가 호출되면 계속해서 O(N)의 시간복잡도로 루트를 찾게 됩니다. 하지만 경로압축을 하면 경로 상의 모든 노드들이 루트 노드를 직접 가리키도록 업데이트하게 되어 다음의 find() 호출시 더 빠르게 수행되도록 합니다

before




after




## 트리의 깊이 최적화

```
if (uf_rank[rootA] < uf_rank[rootB])
```

- rank는 트리의 높이를 나타냄

- 작은 트리를 큰 트리에 붙이는 전략 → 깊이를 최소화

예시:

- rank 1짜리 트리를 rank 3짜리 트리에 붙이면, 전체 깊이는 3 그대로 유지됨
- 반대로 rank 3짜리를 rank 1짜리에 붙이면 깊이가 4가 되어 비효율

```
p[rootA] = rootB; 또는 p[rootB] = rootA;
```

- 루트가 rank 더 큰 쪽이 되도록 부모를 설정

```
if (uf_rank[rootA] == uf_rank[rootB]) uf_rank[rootA]++;
```

- 두 루트의 rank가 같을 경우에만 붙이는 쪽의 rank를 증가
  왜냐하면 같은 높이의 트리를 하나로 붙이면 트리 높이가 1 증가하기 때문입니다.

이외에도 문제에 따라 각 트리가 가지고 있는 노드의 수를 기반으로 해당 서브트리의 크기를 더해가면서 로직을 구축할 수도 있습니다.

ex) size[rootB] += size[rootA]

## 시간 복잡도

앞에서 설명드린 최적화된 find() 함수와 merge() 의 시간 복잡도는O(α(N))입니다. 이는 아커만(Ackermann) 함수의 역함수이며 사실상 O(1)로 간주할 수 있습니다.

[참고] 진격의 거인 - 아커만 리바이 이라고 외우시면 쉽습니다.

백준 추천문제: 1717, 1976, 16562, 4195, 20040

# 2.LCS

DP 알고리즘 중 하나인 최장공통부분수열, LCS(Longest Common Subsequence)은 두개의 수열 중 가장 긴 공통 부분 수열 알고리즘을 공부해봅시다.

## 예시

A = "**ACA**Y**K**P"

B = "C**A**P**CAK**"

→ 최장 공통 부분 수열(LCS) 길이: 4 (“ACAK")가 됩니다. 앞의 보는 것처럼 연속적으로 ACAK일 필요는 없고 순서대로의 문자열이기만하면 공통부분수열이 됩니다.

DP 정의

LCS[i][j] = A의 i번째까지, B의 j번째까지 비교했을 때 LCS의 길이

- A[i-1] == B[j-1] 이면
  → LCS[i][j] = LCS[i-1][j-1] + 1
- A[i-1] ≠ B[j-1] 이면
  → LCS[i][j] = max(LCS[i-1][j], LCS[i][j-1])

- LCS[i-1][j] : A의 마지막 문자 하나 빼고 계산

- LCS[i][j-1] : B의 마지막 문자 하나 빼고 계산

실제 다음의 문자열로 LCS를 만들어 봅시다.

A = "**ACA**Y**K**P"

B = "C**A**P**CAK**"

| | 0 | C | A | P | C | A | K |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| A | 0 | 0 | 1 | 1 | 1 | 1 | 1 |
| C | 0 | 1 | 1 | 1 | 2 | 2 | 2 |
| A | 0 | 1 | 2 | 2 | 2 | 3 | 3 |
| Y | 0 | 1 | 2 | 2 | 2 | 3 | 3 |
| K | 0 | 1 | 2 | 2 | 2 | 3 | 4 |
| P | 0 | 1 | 2 | 3 | 3 | 3 | 4 |

```
#include <bits/stdc++.h>
#define MAX_N 1004
using namespace std;

string A, B;
int lcs[MAX_N][MAX_N];
int N, M;
string traceLCS() {
    int i = N, j = M;
    string ret_lcs = "";
    while (i > 0 && j > 0) {
        if (A[i - 1] == B[j - 1]) {
            ret_lcs = A[i - 1] + ret_lcs;
            i--; j--;
```

```
        } else if (lcs[i - 1][j] >= lcs[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }
    return ret_lcs;
}

int main() {
    cin >> A >> B;
    N = A.size();
    M = B.size();
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= M; j++) {
            if (A[i - 1] == B[j - 1]) {
                lcs[i][j] = lcs[i - 1][j - 1] + 1;
            } else {
                lcs[i][j] = max(lcs[i - 1][j], lcs[i][j - 1]);
            }
        }
    }

    cout << lcs[N][M] << "\n";
    for (int i = 0; i <= N; i++) {
        for (int j = 0; j <= M; j++) {
            cout << lcs[i][j] << " ";
        }
        cout << "\n";
    }

    cout << traceLCS() << "\n";

    return 0;
}

ACAYKP
CAPCAK
```

시간복잡도: O(N × M)

백준 추천문제: 9251